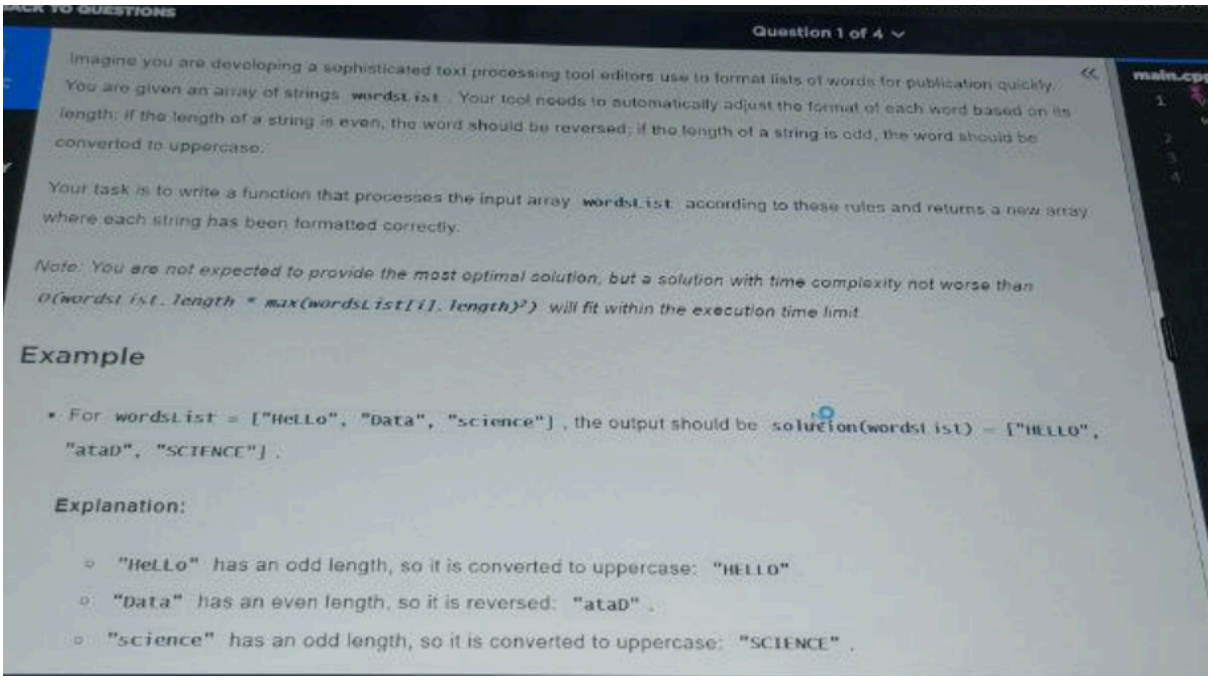
VISA



Question 1 of 4

Imagine you are developing a sophisticated text processing tool editors use to format lists of words for publication quickly. You are given an array of strings `wordsList`. Your tool needs to automatically adjust the format of each word based on its length: if the length of a string is even, the word should be reversed; if the length of a string is odd, the word should be converted to uppercase.

Your task is to write a function that processes the input array `wordsList` according to these rules and returns a new array where each string has been formatted correctly.

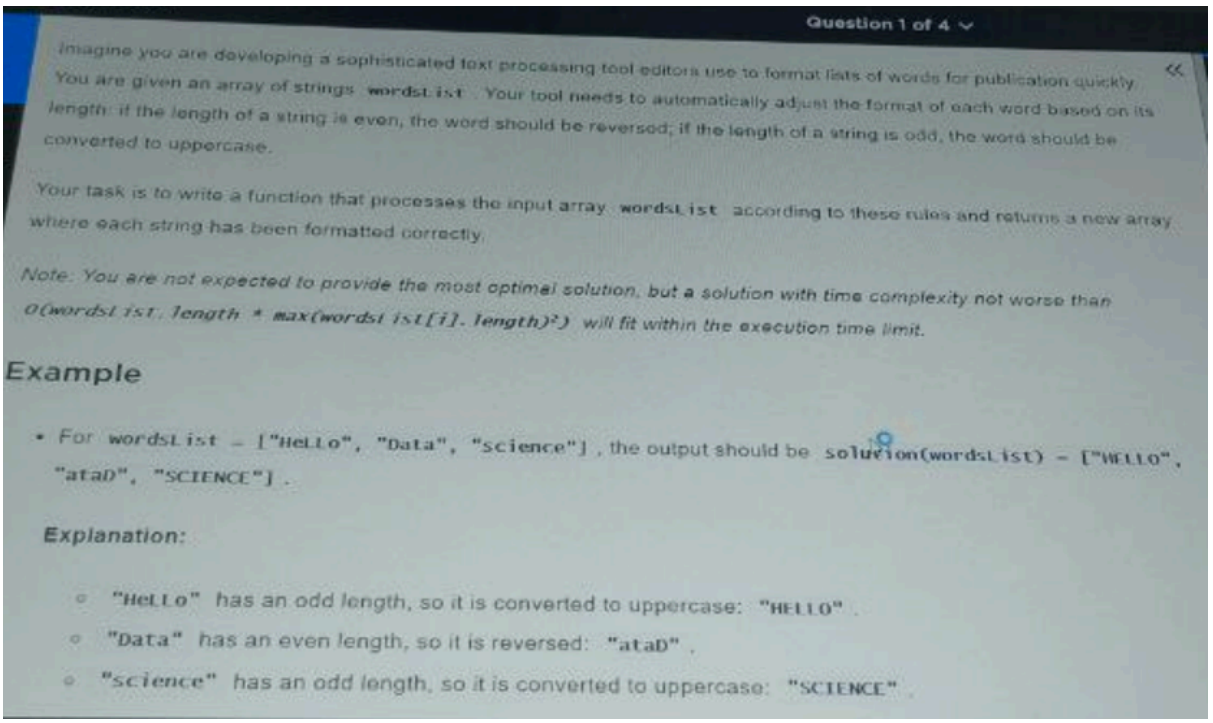
Note: You are not expected to provide the most optimal solution, but a solution with time complexity not worse than `O(wordsList.length * max(wordsList[i].length)²)` will fit within the execution time limit.

Example

- For `wordsList = ["Hello", "Data", "science"]`, the output should be `solution(wordsList) = ["HELLO", "ataD", "SCIENCE"]`.

Explanation:

- `"Hello"` has an odd length, so it is converted to uppercase: `"HELLO"`.
- `"Data"` has an even length, so it is reversed: `"ataD"`.
- `"science"` has an odd length, so it is converted to uppercase: `"SCIENCE"`.



Question 1 of 4

Imagine you are developing a sophisticated text processing tool editors use to format lists of words for publication quickly. You are given an array of strings `wordsList`. Your tool needs to automatically adjust the format of each word based on its length: if the length of a string is even, the word should be reversed; if the length of a string is odd, the word should be converted to uppercase.

Your task is to write a function that processes the input array `wordsList` according to these rules and returns a new array where each string has been formatted correctly.

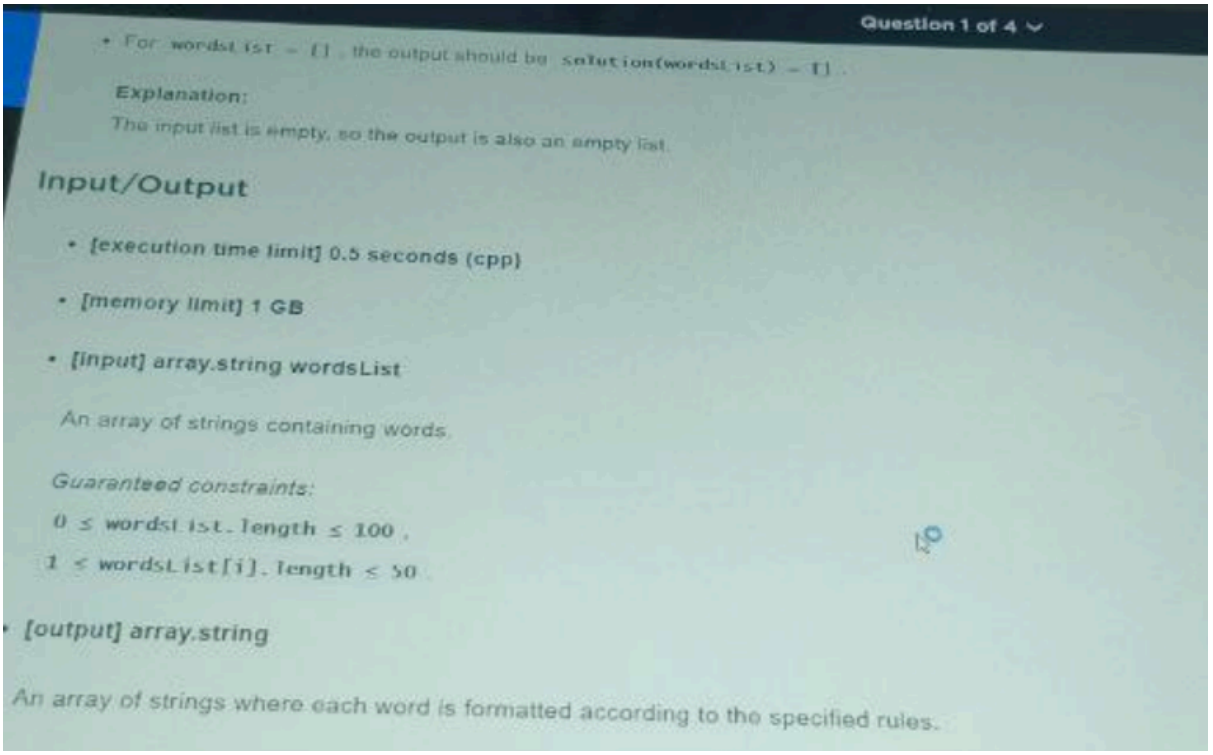
Note: You are not expected to provide the most optimal solution, but a solution with time complexity not worse than `O(wordsList.length * max(wordsList[i].length)²)` will fit within the execution time limit.

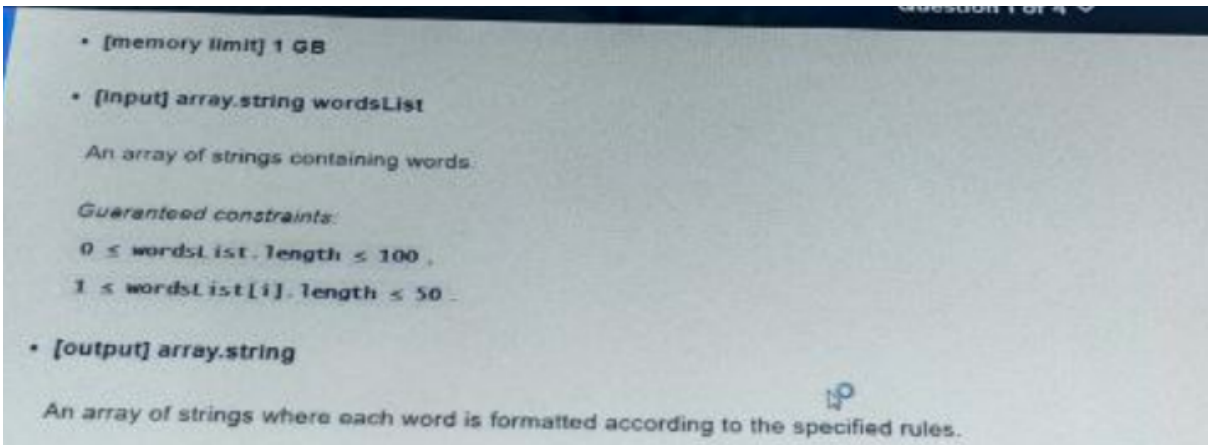
Example

- For `wordsList = ["Hello", "Data", "science"]`, the output should be `solution(wordsList) = ["HELLO", "ataD", "SCIENCE"]`.

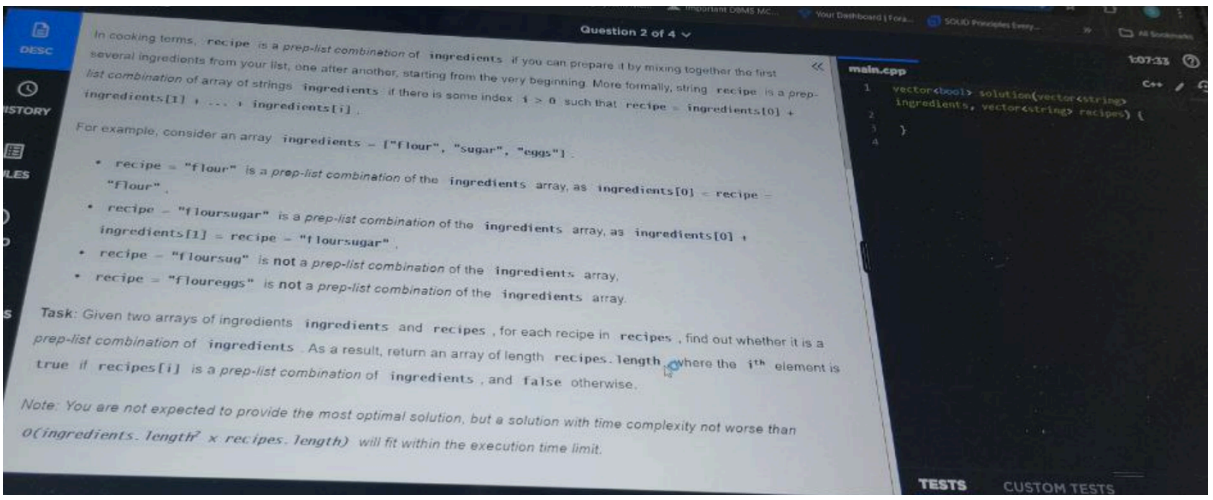
Explanation:

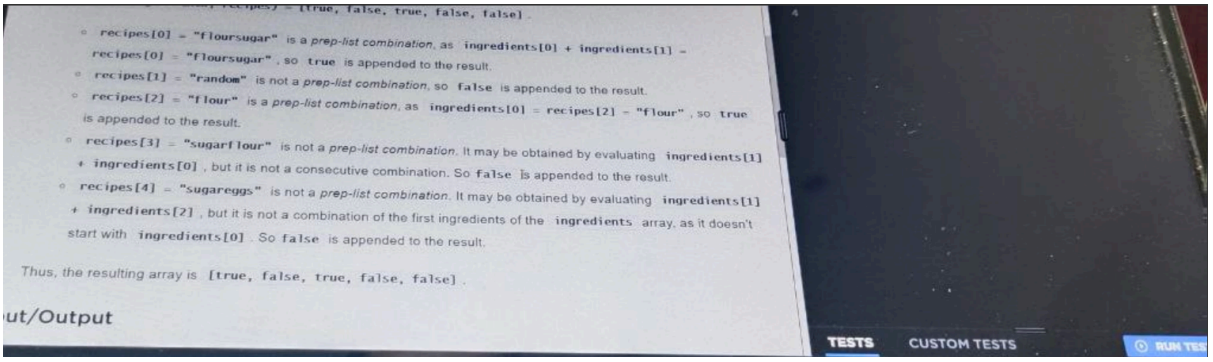
- `"Hello"` has an odd length, so it is converted to uppercase: `"HELLO"`.
- `"Data"` has an even length, so it is reversed: `"ataD"`.
- `"science"` has an odd length, so it is converted to uppercase: `"SCIENCE"`.

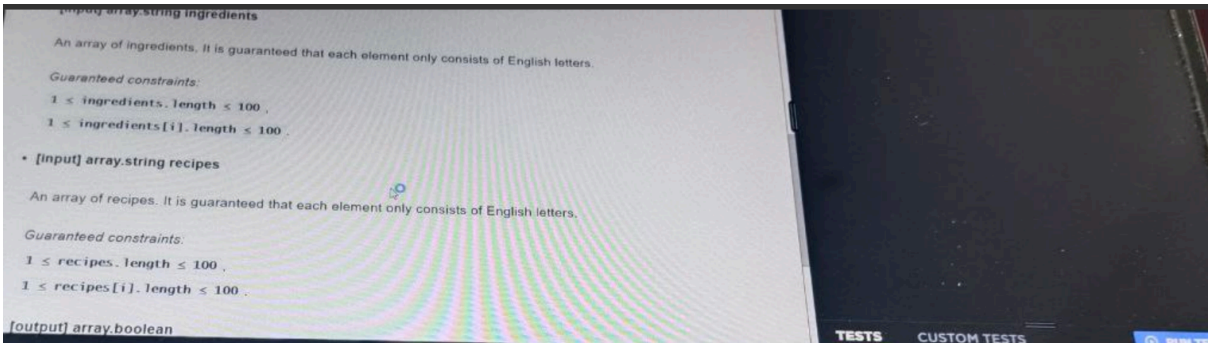

Question 1 of 4
• For wordsList = [], the output should be solution(wordsList) = [].
Explanation:
The input list is empty, so the output is also an empty list.
Input/Output
• [execution time limit] 0.5 seconds (cpp)
• [memory limit] 1 GB
• [input] array.string wordsList
An array of strings containing words.
Guaranteed constraints:
0 ≤ wordsList.length ≤ 100,
1 ≤ wordsList[i].length ≤ 50.
• [output] array.string
An array of strings where each word is formatted according to the specified rules.


• [memory limit] 1 GB
• [input] array.string wordsList
An array of strings containing words.
Guaranteed constraints:
0 ≤ wordsList.length ≤ 100,
1 ≤ wordsList[i].length ≤ 50.
• [output] array.string
An array of strings where each word is formatted according to the specified rules.

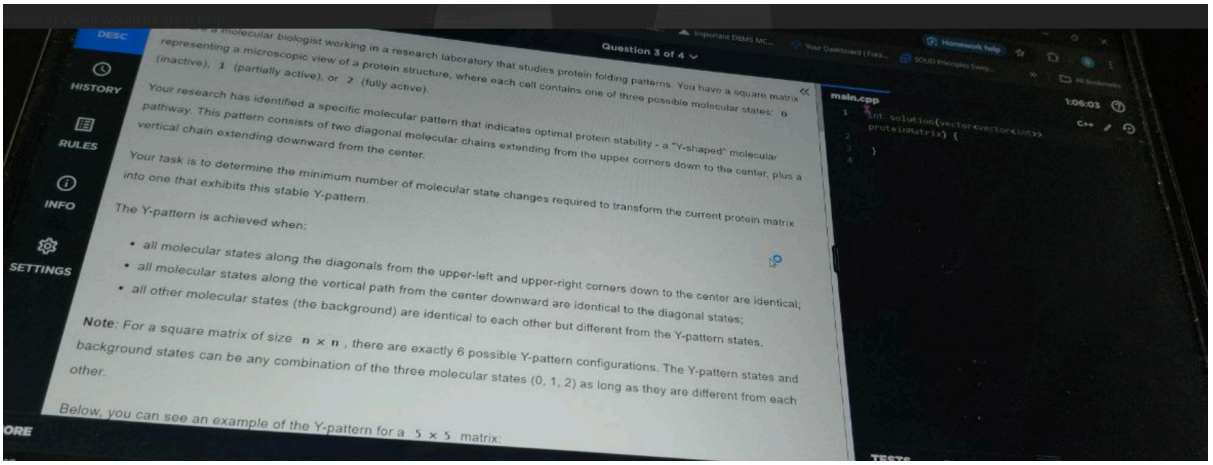
Q2.

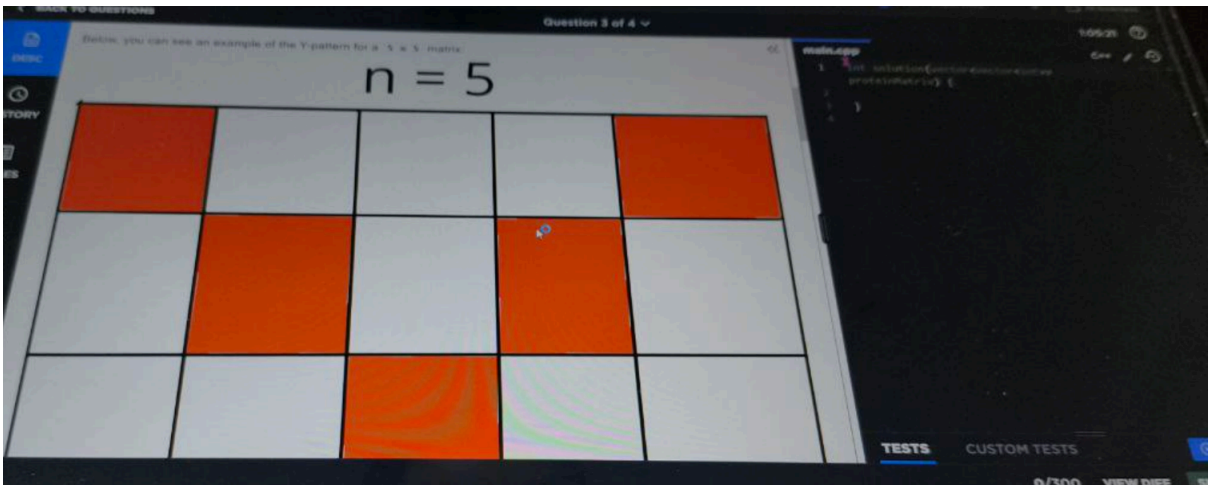

Question 2 of 4
DESC
HISTORY
RULES
In cooking terms, recipe is a prep-list combination of ingredients if you can prepare it by mixing together the first several ingredients from your list, one after another, starting from the very beginning. More formally, string recipe is a prep-list combination of array of strings ingredients if there is some index i > 0 such that recipe = ingredients[0] + ingredients[1] + ... + ingredients[i].
For example, consider an array ingredients = ["flour", "sugar", "eggs"].
recipe = "flour" is a prep-list combination of the ingredients array, as ingredients[0] = recipe = "flour".
recipe = "floursugar" is a prep-list combination of the ingredients array, as ingredients[0] + ingredients[1] = recipe = "floursugar".
recipe = "floursug" is not a prep-list combination of the ingredients array.
recipe = "floureggs" is not a prep-list combination of the ingredients array.
Task: Given two arrays of ingredients ingredients and recipes, for each recipe in recipes, find out whether it is a prep-list combination of ingredients. As a result, return an array of length recipes.length where the i^th element is true if recipes[i] is a prep-list combination of ingredients, and false otherwise.
Note: You are not expected to provide the most optimal solution, but a solution with time complexity not worse than O(ingredients.length^2 * recipes.length) will fit within the execution time limit.
main.cpp
vector<bool> solution(vector<string> ingredients, vector<string> recipes) {
}
TESTS
CUSTOM TESTS

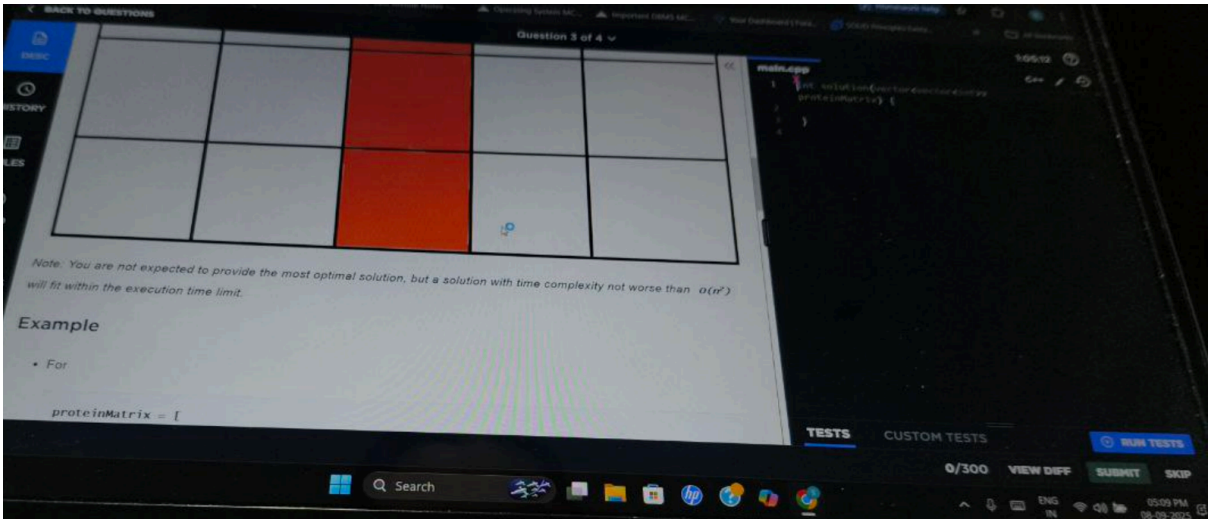

[true, false, true, false, false].
recipes[0] = "floursugar" is a prep-list combination, as ingredients[0] + ingredients[1] = recipes[0] = "floursugar", so true is appended to the result.
recipes[1] = "random" is not a prep-list combination, so false is appended to the result.
recipes[2] = "flour" is a prep-list combination, as ingredients[0] = recipes[2] = "flour", so true is appended to the result.
recipes[3] = "sugarflour" is not a prep-list combination. It may be obtained by evaluating ingredients[1] + ingredients[0], but it is not a consecutive combination. So false is appended to the result.
recipes[4] = "sugareggs" is not a prep-list combination. It may be obtained by evaluating ingredients[1] + ingredients[2], but it is not a combination of the first ingredients of the ingredients array, as it doesn't start with ingredients[0]. So false is appended to the result.
Thus, the resulting array is [true, false, true, false, false].
ut/Output
TESTS
CUSTOM TESTS

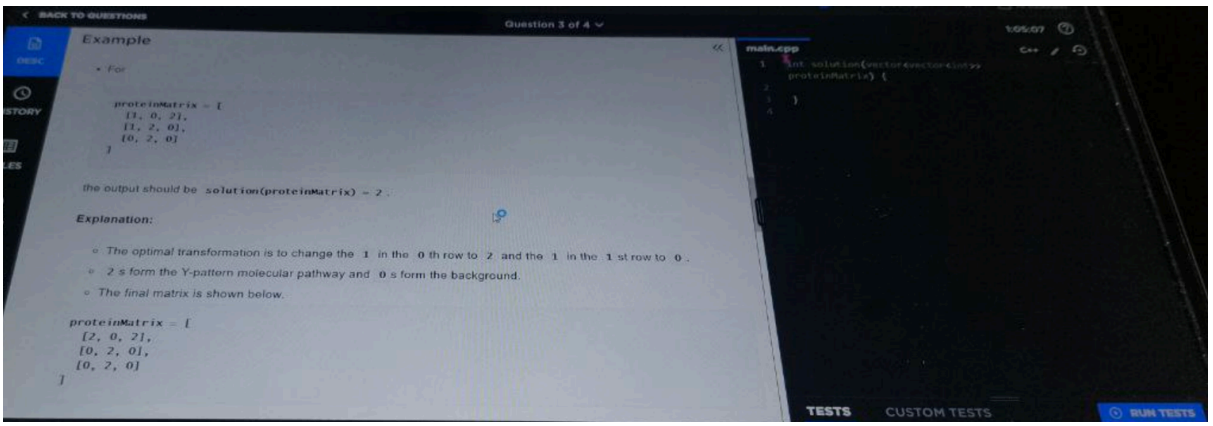

[input] array.string ingredients
An array of ingredients. It is guaranteed that each element only consists of English letters.
Guaranteed constraints:
1 ≤ ingredients.length ≤ 100,
1 ≤ ingredients[i].length ≤ 100.
[input] array.string recipes
An array of recipes. It is guaranteed that each element only consists of English letters.
Guaranteed constraints:
1 ≤ recipes.length ≤ 100,
1 ≤ recipes[i].length ≤ 100.
[output] array.boolean
TESTS
CUSTOM TESTS

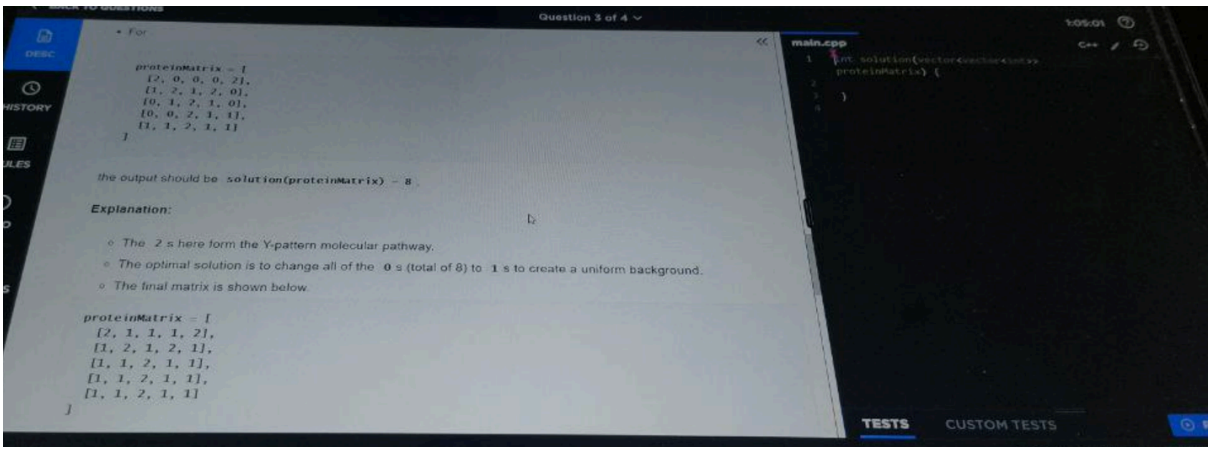
Q3.

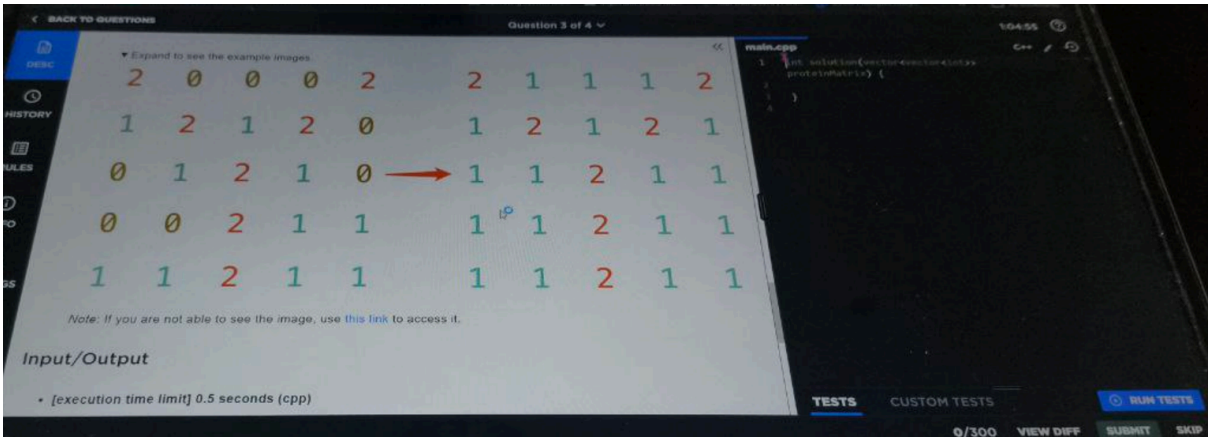

Question 3 of 4
DESC
HISTORY
RULES
INFO
SETTINGS
a molecular biologist working in a research laboratory that studies protein folding patterns. You have a square matrix representing a microscopic view of a protein structure, where each cell contains one of three possible molecular states: 0 (inactive), 1 (partially active), or 2 (fully active).
Your research has identified a specific molecular pattern that indicates optimal protein stability - a "Y-shaped" molecular pathway. This pattern consists of two diagonal molecular chains extending from the upper corners down to the center, plus a vertical chain extending downward from the center.
Your task is to determine the minimum number of molecular state changes required to transform the current protein matrix into one that exhibits this stable Y-pattern.
The Y-pattern is achieved when:
all molecular states along the diagonals from the upper-left and upper-right corners down to the center are identical;
all molecular states along the vertical path from the center downward are identical to the diagonal states;
all other molecular states (the background) are identical to each other but different from the Y-pattern states.
Note: For a square matrix of size n × n, there are exactly 6 possible Y-pattern configurations. The Y-pattern states and background states can be any combination of the three molecular states (0, 1, 2) as long as they are different from each other.
Below, you can see an example of the Y-pattern for a 5 × 5 matrix:
main.cpp
int solution(vector<vector<int>> proteinMatrix) {
}
TESTS

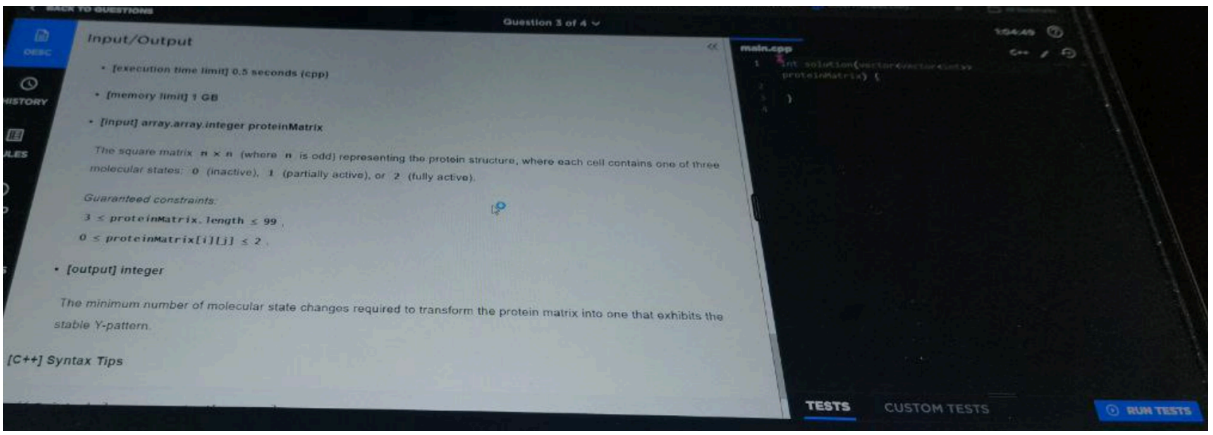

Question 3 of 4
n = 5
TESTS
CUSTOM TESTS


Question 3 of 4
Note: You are not expected to provide the most optimal solution, but a solution with time complexity not worse than O(n²) will fit within the execution time limit.
Example
• For
proteinMatrix = [
TESTS
CUSTOM TESTS
0/300
VIEW DIFF
SUBMIT
SKIP
Search

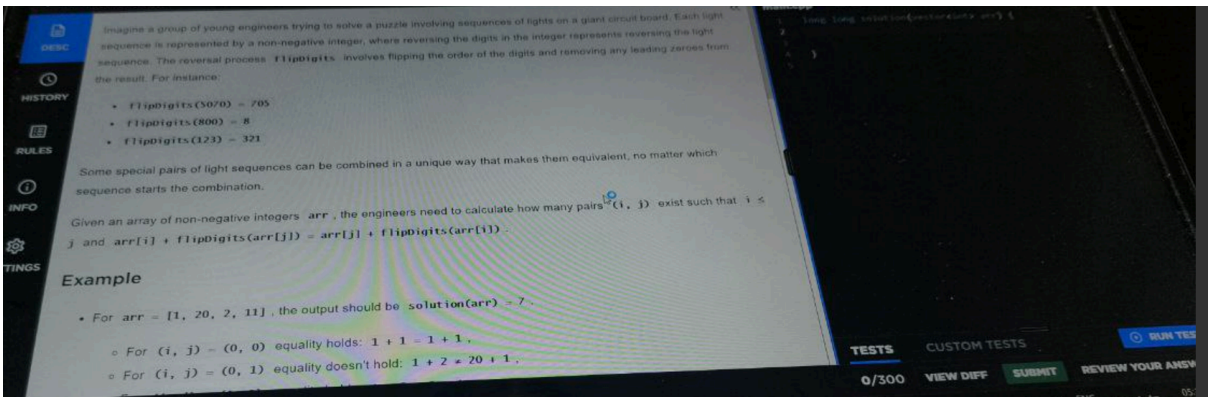

Question 3 of 4
Example
• For
proteinMatrix = [
[1, 0, 2],
[1, 2, 0],
[0, 2, 0]
]
the output should be solution(proteinMatrix) = 2.
Explanation:
The optimal transformation is to change the 1 in the 0 th row to 2 and the 1 in the 1 st row to 0.
2 s form the Y-pattern molecular pathway and 0 s form the background.
The final matrix is shown below.
proteinMatrix = [
[2, 0, 2],
[0, 2, 0],
[0, 2, 0]
]
TESTS
CUSTOM TESTS

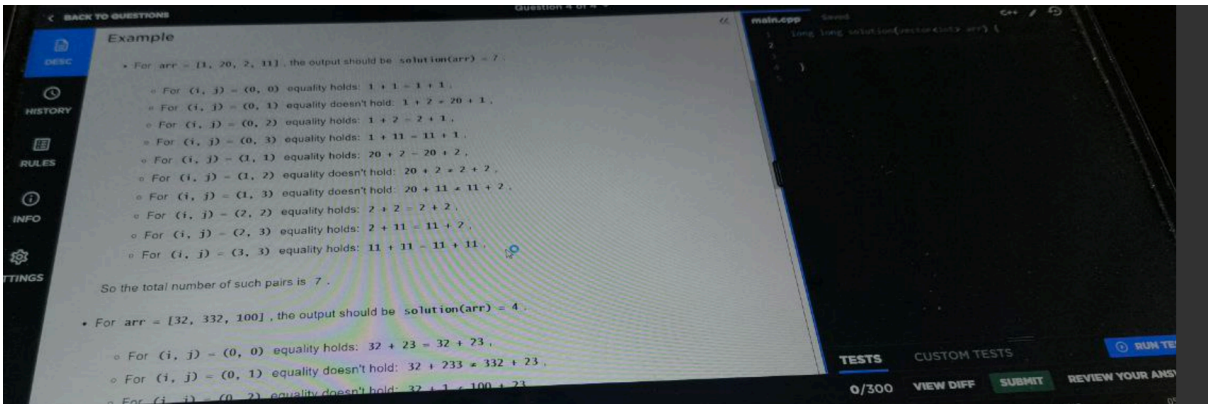

Question 3 of 4
proteinMatrix = [
[2, 0, 0, 0, 2],
[1, 2, 1, 2, 0],
[0, 1, 2, 1, 0],
[0, 0, 2, 1, 1],
[1, 1, 2, 1, 1]
]
the output should be solution(proteinMatrix) = 8
Explanation:
The 2 s here form the Y-pattern molecular pathway.
The optimal solution is to change all of the 0 s (total of 8) to 1 s to create a uniform background.
The final matrix is shown below.
proteinMatrix = [
[2, 1, 1, 1, 2],
[1, 2, 1, 2, 1],
[1, 1, 2, 1, 1],
[1, 1, 2, 1, 1],
[1, 1, 2, 1, 1]
]
main.cpp
TESTS
CUSTOM TESTS

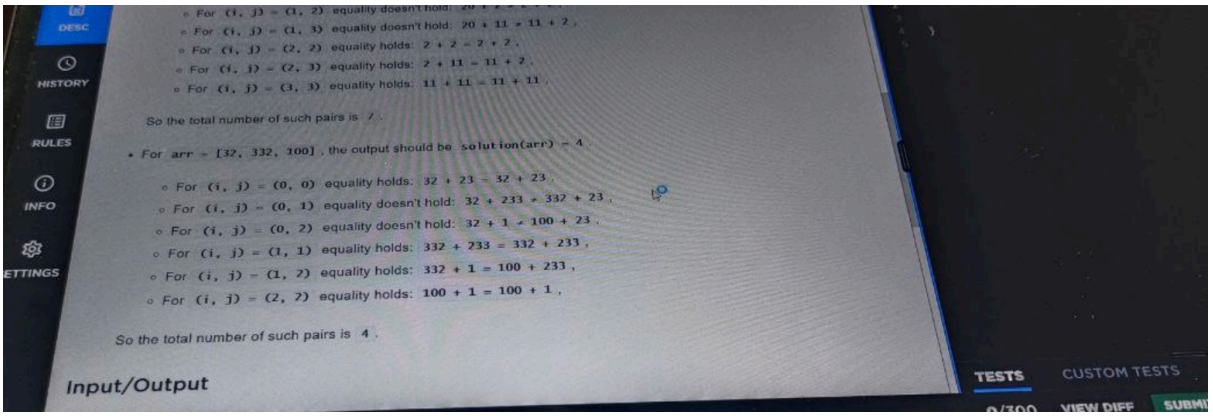

BACK TO QUESTIONS
Question 3 of 4
Expand to see the example images
2 0 0 0 2
1 2 1 2 0
0 1 2 1 0
0 0 2 1 1
1 1 2 1 1
2 1 1 1 2
1 2 1 2 1
1 1 2 1 1
1 1 2 1 1
1 1 2 1 1
Note: If you are not able to see the image, use this link to access it.
Input/Output
[execution time limit] 0.5 seconds (cpp)
main.cpp
TESTS
CUSTOM TESTS
RUN TESTS
0/300
VIEW DIFF
SUBMIT
SKIP

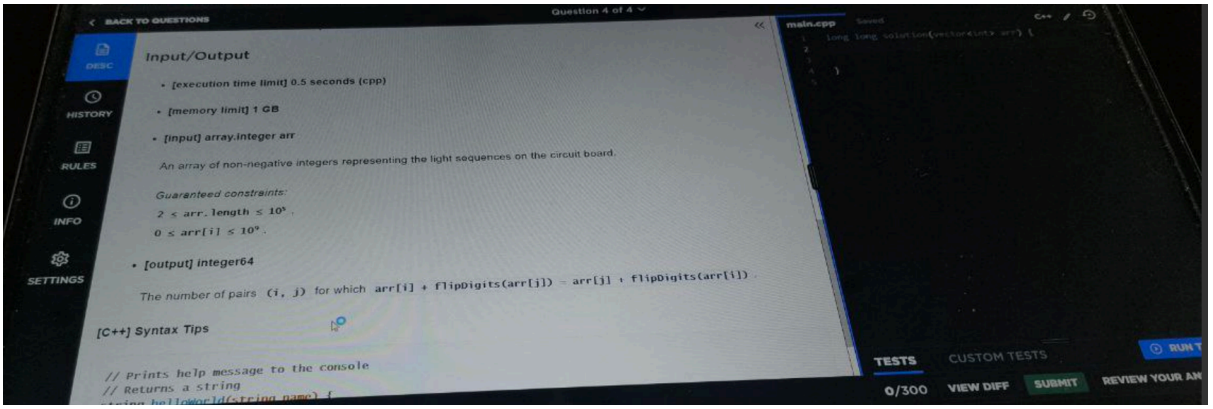

Question 3 of 4
Input/Output
[execution time limit] 0.5 seconds (cpp)
[memory limit] 1 GB
[input] array.array.integer proteinMatrix
The square matrix n × n (where n is odd) representing the protein structure, where each cell contains one of three molecular states: 0 (inactive), 1 (partially active), or 2 (fully active).
Guaranteed constraints:
3 ≤ proteinMatrix.length ≤ 99,
0 ≤ proteinMatrix[i][j] ≤ 2.
[output] integer
The minimum number of molecular state changes required to transform the protein matrix into one that exhibits the stable Y-pattern.
[C++] Syntax Tips
main.cpp
TESTS
CUSTOM TESTS
RUN TESTS

Q4.

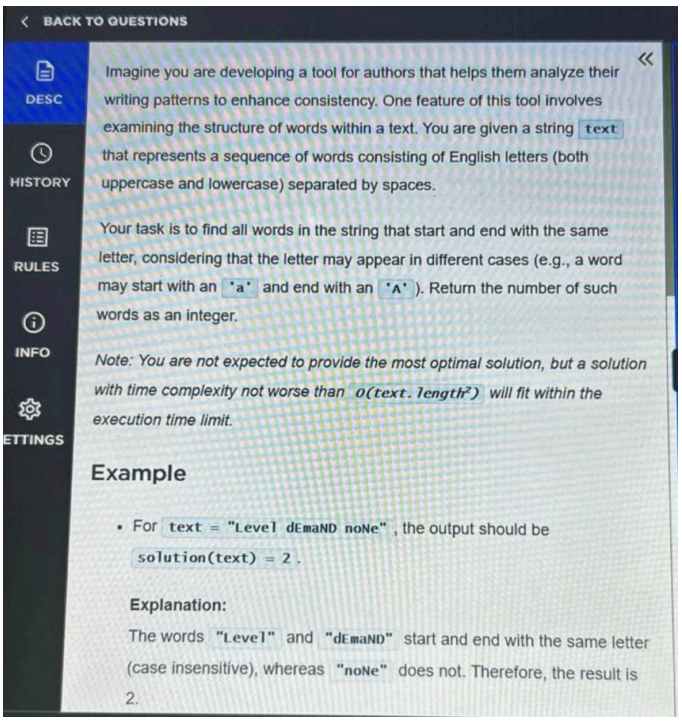

DESC
HISTORY
RULES
INFO
SETTINGS
Imagine a group of young engineers trying to solve a puzzle involving sequences of lights on a giant circuit board. Each light sequence is represented by a non-negative integer, where reversing the digits in the integer represents reversing the light sequence. The reversal process flipDigits involves flipping the order of the digits and removing any leading zeroes from the result. For instance:
flipDigits(3070) = 705
flipDigits(800) = 8
flipDigits(123) = 321
Some special pairs of light sequences can be combined in a unique way that makes them equivalent, no matter which sequence starts the combination.
Given an array of non-negative integers arr, the engineers need to calculate how many pairs (i, j) exist such that i ≤ j and arr[i] + flipDigits(arr[j]) = arr[j] + flipDigits(arr[i]).
Example
For arr = [1, 20, 2, 11], the output should be solution(arr) = 7.
For (i, j) = (0, 0) equality holds: 1 + 1 = 1 + 1.
For (i, j) = (0, 1) equality doesn't hold: 1 + 2 ≠ 20 + 1.
TESTS
CUSTOM TESTS
0/300
VIEW DIFF
SUBMIT
REVIEW YOUR ANSW

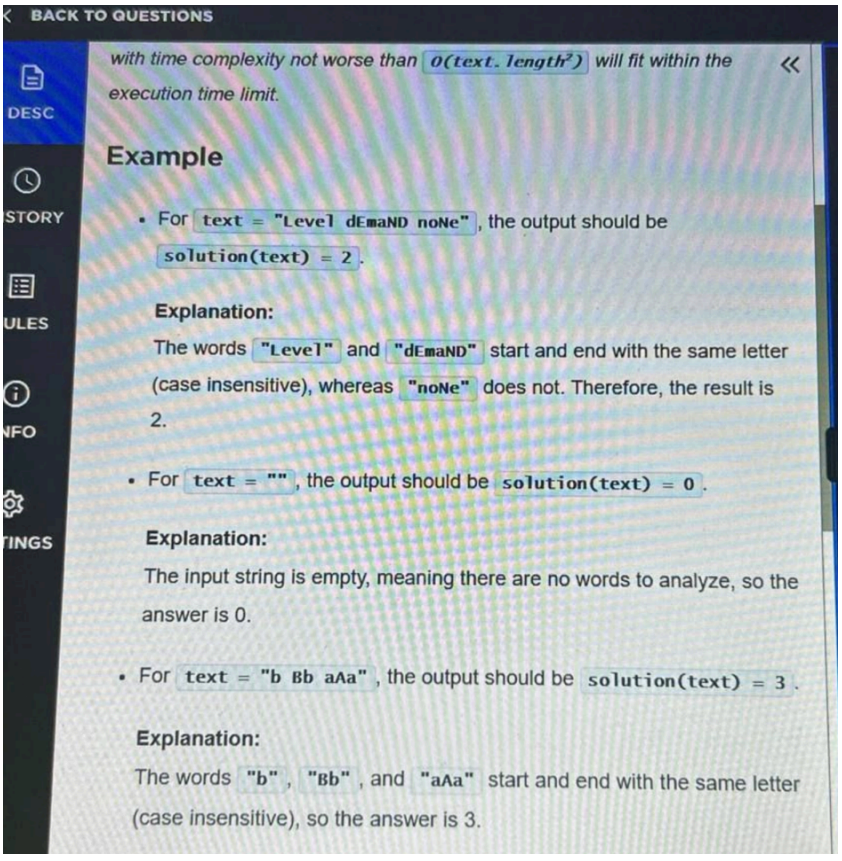

BACK TO QUESTIONS
DESC
HISTORY
RULES
INFO
TTINGS
Example
For arr = [1, 20, 2, 11], the output should be solution(arr) = 7.
For (i, j) = (0, 0) equality holds: 1 + 1 = 1 + 1.
For (i, j) = (0, 1) equality doesn't hold: 1 + 2 ≠ 20 + 1.
For (i, j) = (0, 2) equality holds: 1 + 2 = 2 + 1.
For (i, j) = (0, 3) equality holds: 1 + 11 = 11 + 1.
For (i, j) = (1, 1) equality holds: 20 + 2 = 20 + 2.
For (i, j) = (1, 2) equality doesn't hold: 20 + 2 ≠ 2 + 2.
For (i, j) = (1, 3) equality doesn't hold: 20 + 11 ≠ 11 + 2.
For (i, j) = (2, 2) equality holds: 2 + 2 = 2 + 2.
For (i, j) = (2, 3) equality holds: 2 + 11 = 11 + 2.
For (i, j) = (3, 3) equality holds: 11 + 11 = 11 + 11.
So the total number of such pairs is 7.
For arr = [32, 332, 100], the output should be solution(arr) = 4.
For (i, j) = (0, 0) equality holds: 32 + 23 = 32 + 23.
For (i, j) = (0, 1) equality doesn't hold: 32 + 233 ≠ 332 + 23.
main.cpp
TESTS
CUSTOM TESTS
0/300
VIEW DIFF
SUBMIT
REVIEW YOUR ANSW

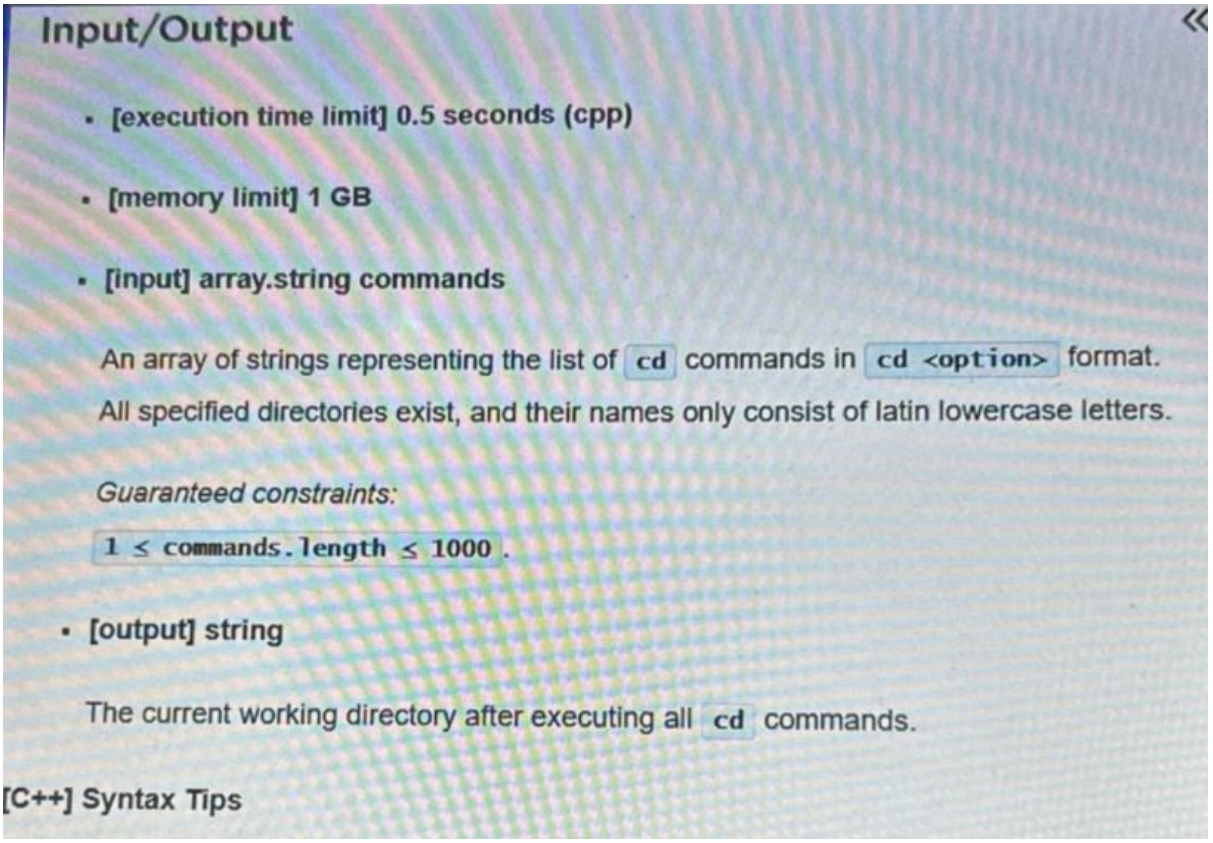

DESC
HISTORY
RULES
INFO
ETTINGS
For (i, j) = (1, 2) equality doesn't hold: 20 + 2 ≠ 2 + 2.
For (i, j) = (1, 3) equality doesn't hold: 20 + 11 ≠ 11 + 2.
For (i, j) = (2, 2) equality holds: 2 + 2 = 2 + 2.
For (i, j) = (2, 3) equality holds: 2 + 11 = 11 + 2.
For (i, j) = (3, 3) equality holds: 11 + 11 = 11 + 11.
So the total number of such pairs is 7.
For arr = [32, 332, 100], the output should be solution(arr) = 4.
For (i, j) = (0, 0) equality holds: 32 + 23 = 32 + 23.
For (i, j) = (0, 1) equality doesn't hold: 32 + 233 ≠ 332 + 23.
For (i, j) = (0, 2) equality doesn't hold: 32 + 1 ≠ 100 + 23.
For (i, j) = (1, 1) equality holds: 332 + 233 = 332 + 233.
For (i, j) = (1, 2) equality holds: 332 + 1 = 100 + 233.
For (i, j) = (2, 2) equality holds: 100 + 1 = 100 + 1.
So the total number of such pairs is 4.
Input/Output
TESTS
CUSTOM TESTS


BACK TO QUESTIONS
Question 4 of 4
DESC
HISTORY
RULES
INFO
SETTINGS
Input/Output
[execution time limit] 0.5 seconds (cpp)
[memory limit] 1 GB
[input] array.integer arr
An array of non-negative integers representing the light sequences on the circuit board.
Guaranteed constraints:
2 ≤ arr.length ≤ 10^5.
0 ≤ arr[i] ≤ 10^9.
[output] integer64
The number of pairs (i, j) for which arr[i] + flipDigits(arr[j]) = arr[j] + flipDigits(arr[i]).
[C++] Syntax Tips
// Prints help message to the console
// Returns a string
main.cpp
TESTS
CUSTOM TESTS
0/300
VIEW DIFF
SUBMIT
REVIEW YOUR AN

Q5.


BACK TO QUESTIONS
DESC
HISTORY
RULES
INFO
ETTINGS
Imagine you are developing a tool for authors that helps them analyze their writing patterns to enhance consistency. One feature of this tool involves examining the structure of words within a text. You are given a string text that represents a sequence of words consisting of English letters (both uppercase and lowercase) separated by spaces.
Your task is to find all words in the string that start and end with the same letter, considering that the letter may appear in different cases (e.g., a word may start with an 'a' and end with an 'A'). Return the number of such words as an integer.
Note: You are not expected to provide the most optimal solution, but a solution with time complexity not worse than O(text.length²) will fit within the execution time limit.
Example
For text = "Level dEmaND noNe", the output should be solution(text) = 2.
Explanation:
The words "Level" and "dEmaND" start and end with the same letter (case insensitive), whereas "noNe" does not. Therefore, the result is 2.


BACK TO QUESTIONS
DESC
STORY
ULES
NFO
TINGS
with time complexity not worse than O(text.length²) will fit within the execution time limit.
Example
For text = "Level dEmaND noNe", the output should be solution(text) = 2.
Explanation:
The words "Level" and "dEmaND" start and end with the same letter (case insensitive), whereas "noNe" does not. Therefore, the result is 2.
For text = "", the output should be solution(text) = 0.
Explanation:
The input string is empty, meaning there are no words to analyze, so the answer is 0.
For text = "b Bb aAa", the output should be solution(text) = 3.
Explanation:
The words "b", "Bb", and "aAa" start and end with the same letter (case insensitive), so the answer is 3.


Input/Output
[execution time limit] 0.5 seconds (cpp)
[memory limit] 1 GB
[input] array.string commands
An array of strings representing the list of cd commands in cd <option> format.
All specified directories exist, and their names only consist of latin lowercase letters.
Guaranteed constraints:
1 ≤ commands.length ≤ 1000.
[output] string
The current working directory after executing all cd commands.
[C++] Syntax Tips

Q6.

Your task is to implement a simulation of a `change directory` command. This command changes the current working directory to the specified one.

The initial working directory is root i.e. `/`. You are given a list of `cd` commands `commands`.

There are multiple options for command arguments.

- `cd /` - changes the working directory to the root directory.
- `cd .` - stays in the current directory.
- `cd ..` - moves the working directory one level up. In the root directory, `cd ..` does nothing.
- `cd <subdirectory>` - moves to the specified subdirectory within the current working directory. `<subdirectory>` is a string consisting of only lowercase English letters.

All specified directories exist. Return the absolute path from the root to the working directory after executing all `cd` commands in the given order. `/` should be used as separators.

Note: You are not expected to provide the most optimal solution, but a solution with time complexity not worse than $O(commands.length^2 \times max(commands[i].length))$ *will fit within the execution time limit.*

Example

- For

Example

- For

```
commands = ["cd users",
            "cd codesignal",
            "cd ..",
            "cd admin"]
```

the output should be `solution(commands) = "/users/admin"`.

The absolute path is changing in the following way:

`/` -> `/users` -> `/users/codesignal` -> `/users` -> `/users/admin`.

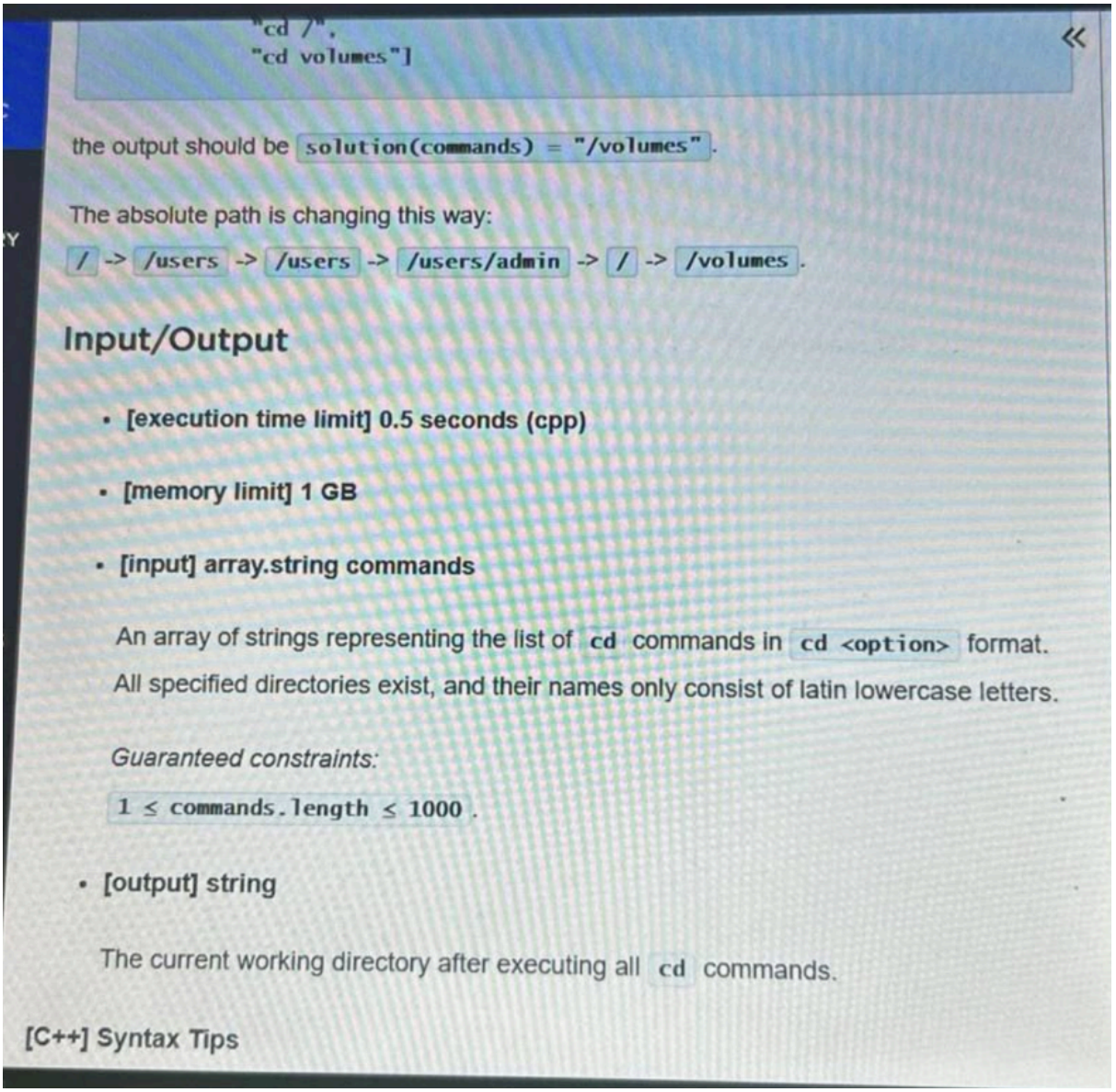
- For

```
commands = ["cd users",
            "cd .",
            "cd admin",
            "cd /",
            "cd volumes"]
```

the output should be `solution(commands) = "/volumes"`.

The absolute path is changing this way:

`/` -> `/users` -> `/users` -> `/users/admin` -> `/` -> `/volumes`.



```
"cd /",
"cd volumes"]
```

the output should be `solution(commands) = "/volumes"`.

The absolute path is changing this way:

`/` -> `/users` -> `/users` -> `/users/admin` -> `/` -> `/volumes`.

Input/Output

- **[execution time limit] 0.5 seconds (cpp)**
- **[memory limit] 1 GB**
- **[input] array.string commands**

  An array of strings representing the list of `cd` commands in `cd <option>` format. All specified directories exist, and their names only consist of latin lowercase letters.

  Guaranteed constraints:

  `1 ≤ commands.length ≤ 1000`.

- **[output] string**

  The current working directory after executing all `cd` commands.

[C++] Syntax Tips

Q7.

Imagine that you're exploring a mysterious labyrinth in the shape of a rectangular matrix, which contains *obstacles* and *teleports*. Starting from the upper-left corner, your goal is to reach the lower-right corner by **only moving to the right**.

You are given integers `n` and `m` representing the dimensions of the labyrinth. You are also given `obstacles` and `teleports`, which are lists of the cells that contain all the obstacles and teleports, respectively.

Here are details about the labyrinth:

- An *obstacle* cannot be traversed - you must stop immediately if you reach a cell containing an obstacle.
- A *teleport* is a pair of cells `(start, end)`. If you reach the `start` cell, you immediately move to the `end` cell.
  - Note that this doesn't work backwards: you cannot teleport from the `end` cell to the `start` cell.
  - It is guaranteed that all teleports have unique `start` cells (i.e. each cell in the labyrinth has one teleport at most).
  - It is guaranteed that the `end` cell for a teleport cannot be a `start` cell for another teleport.
  - It is also guaranteed that both the `start` and `end` cells of a teleport do not contain obstacles.
- Any cell that doesn't contain an obstacle or a teleport is considered a *free cell*, and you can travel through it normally.

contain obstacles.

- Any cell that doesn't contain an obstacle or a teleport is considered a *free cell*, and you can travel through it normally.

You start at the upper-left corner cell with coordinates `(0, 0)`, and the goal is to reach the exit located at the cell with coordinates `(n - 1, m - 1)`. You move according to the following rules:

- You will always move to the right: if you're currently on the cell with coordinates `(row, col)`, you will try moving to the cell with coordinates `(row, col + 1)`.
  - If you move to a cell that's the starting point of a *teleport*, proceed to the coordinates of the `end` cell.
  - If you try to move to a cell that either contains an *obstacle* or is outside the bounds of the labyrinth, stop moving and stay where you are.

Your task is to check whether you can reach the goal (exit of the labyrinth) by following the algorithm above, and to return the total number of cells you travelled through to reach the exit. Note that you should count all cells travelled, including the starting cell `(0, 0)`, and both the `start` and `end` cells of all teleports. If it is not possible to reach the exit, return `-1` if it's because of an obstacle or due to trying to go outside the bounds of the labyrinth, or `-2` if it's because of teleportation (i.e., an infinite teleport loop).

It's guaranteed that the starting cell `(0, 0)` and the exit cell `(n - 1, m - 1)` do not contain an *obstacle*, or be the starting point of a *teleport*.

Note: You are not expected to provide the most optimal solution, but a solution with time complexity not worse than `O(n · m · (obstacles.length + teleports.length))` *will fit within the execution time limit.*

Example

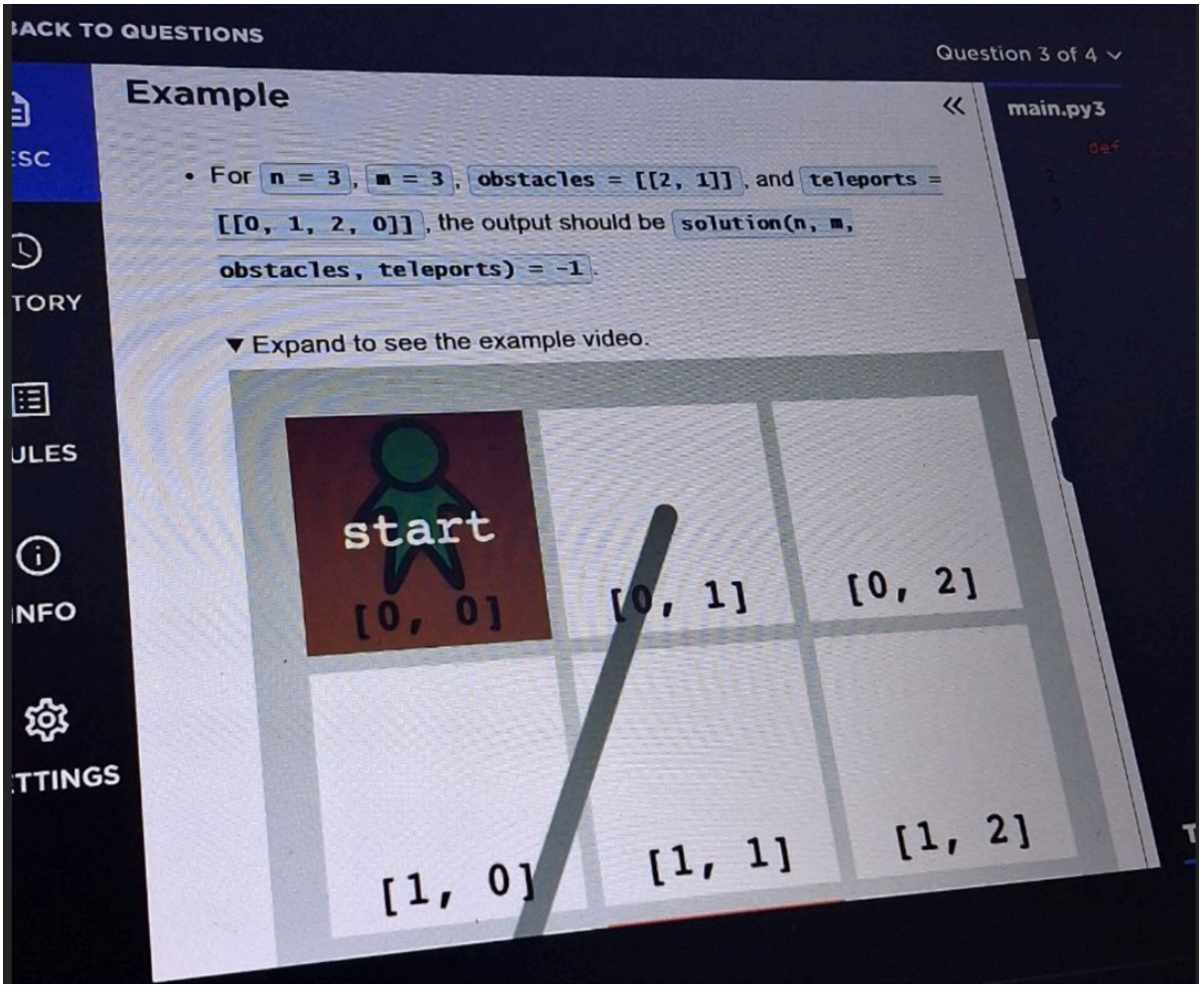
- For `n = 3`, `m = 3`, `obstacles = [[2, 1]]`, and `teleports = [[0, 1, 2, 0]]`, the output should be `solution(n, m, obstacles, teleports) = -1`.

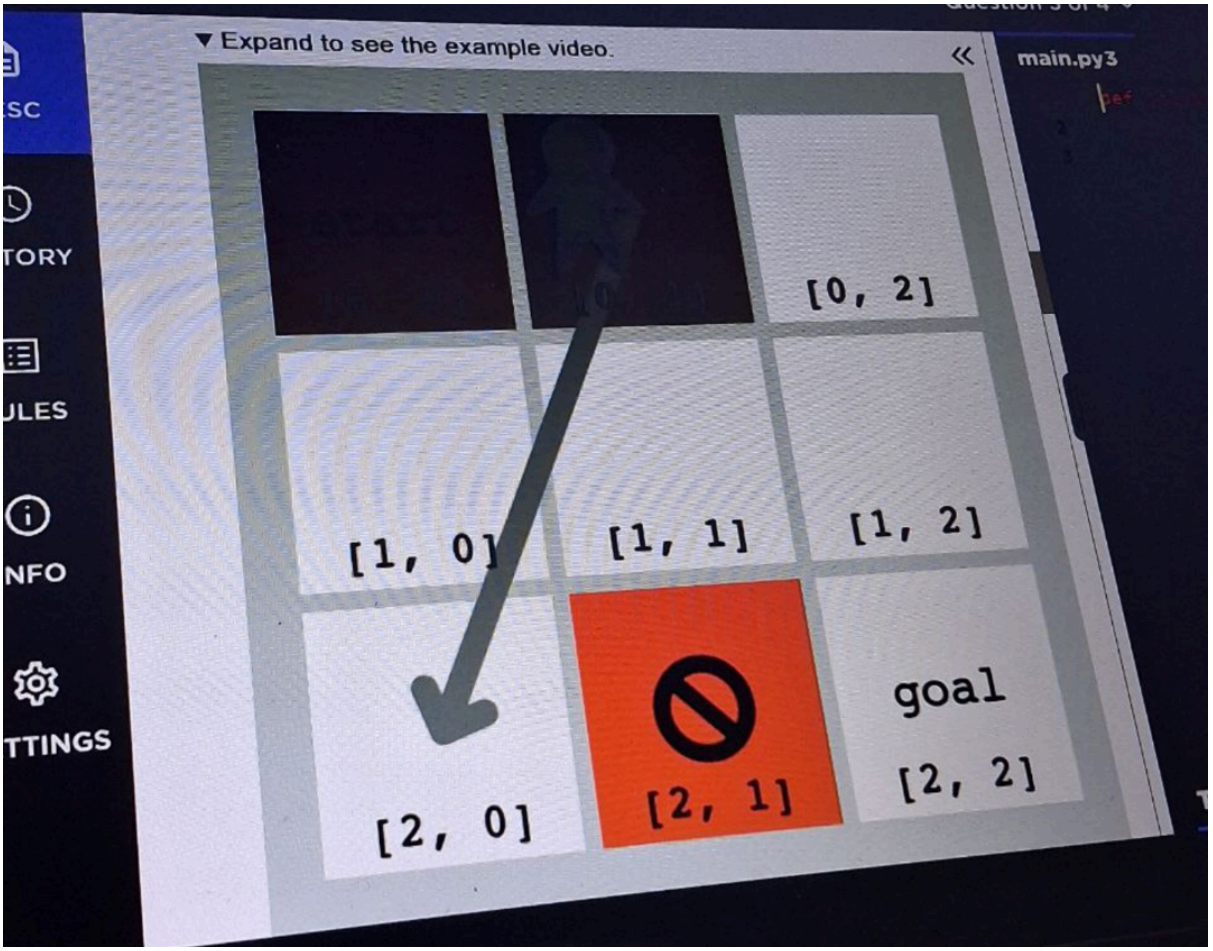
▼ Expand to see the example video.

Example

- For `n = 3`, `m = 3`, `obstacles = [[2, 1]]`, and `teleports = [[0, 1, 2, 0]]`, the output should be `solution(n, m, obstacles, teleports) = -1`.

▼ Expand to see the example video.


BACK TO QUESTIONS
Question 3 of 4
Example
main.py3
For n = 3, m = 3, obstacles = [[2, 1]], and teleports = [[0, 1, 2, 0]], the output should be solution(n, m, obstacles, teleports) = -1.
▼ Expand to see the example video.
start
[0, 0]
[0, 1]
[0, 2]
[1, 0]
[1, 1]
[1, 2]

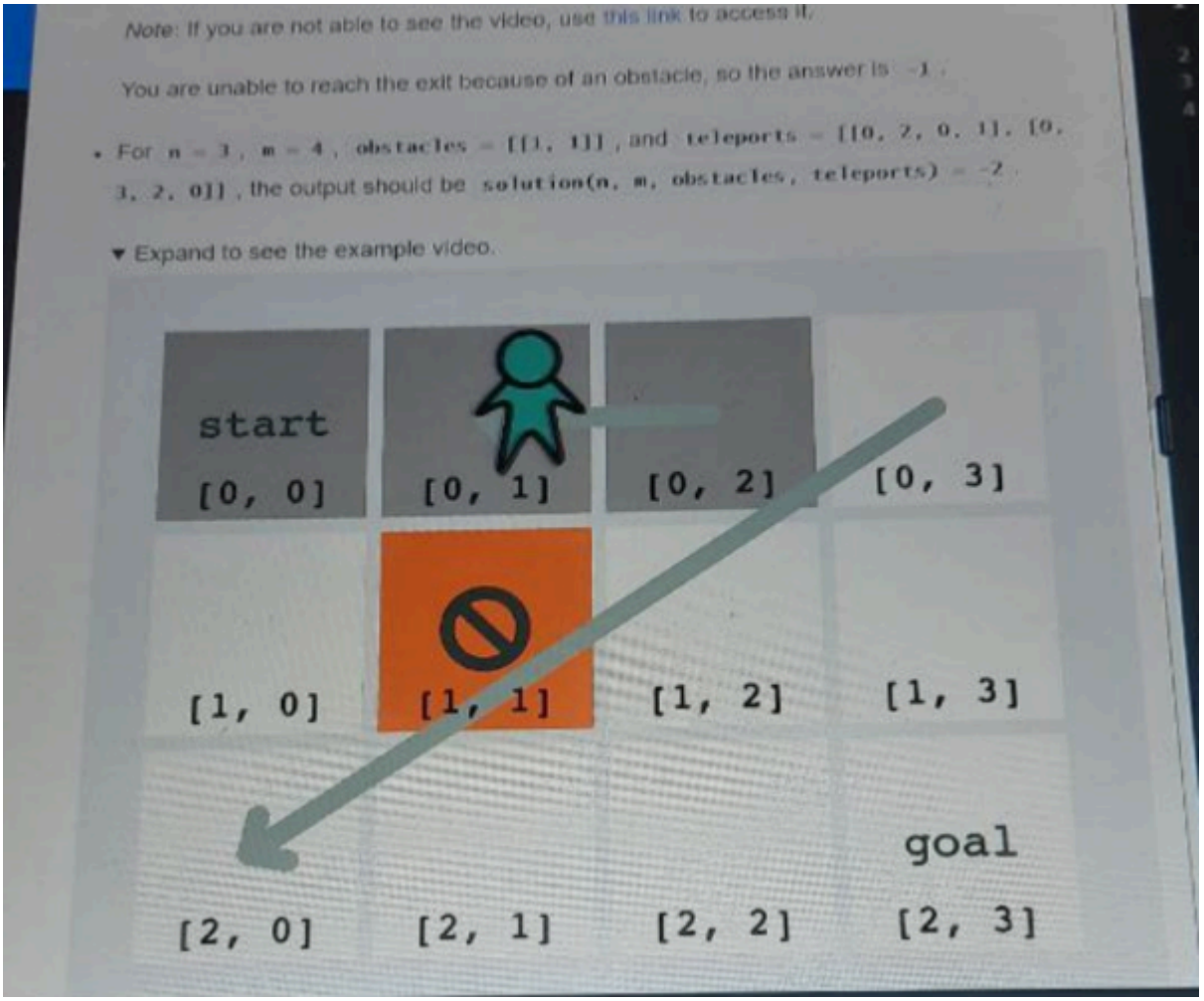

Expand to see the example video.
main.py3
[0, 2]
[1, 0]
[1, 1]
[1, 2]
[2, 0]
[2, 1]
goal
[2, 2]

Note: If you are not able to see the video, use this link to access it.

You are unable to reach the exit because of an obstacle, so the answer is `-1`.

- For `n = 3`, `m = 4`, `obstacles = [[1, 1]]`, and `teleports = [[0, 2, 0, 1], [0, 3, 2, 0]]`, the output should be `solution(n, m, obstacles, teleports) = -2`.

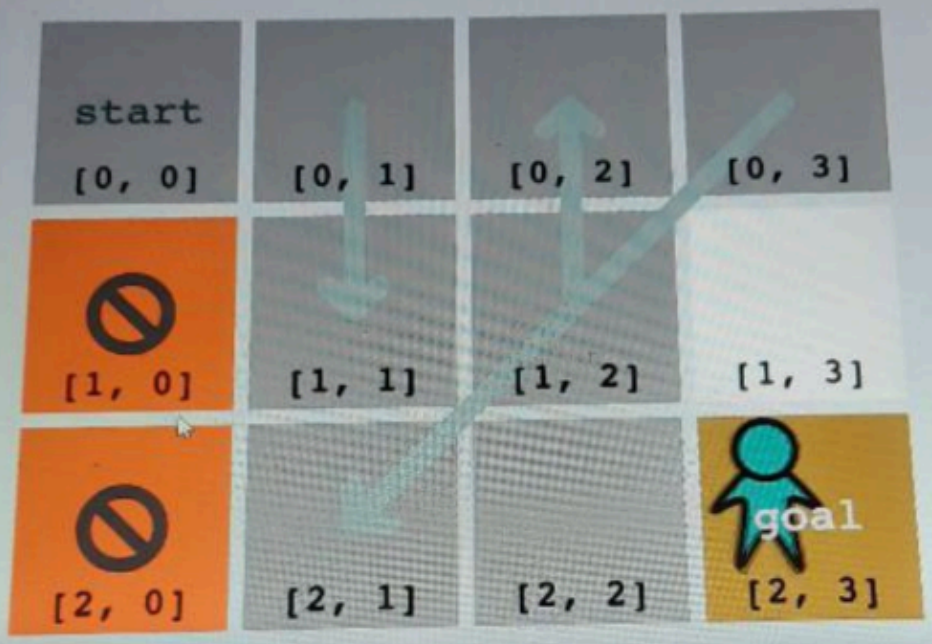
▼ Expand to see the example video.



You are unable to reach the exit because of an infinite teleport loop, so the answer is `-2`.

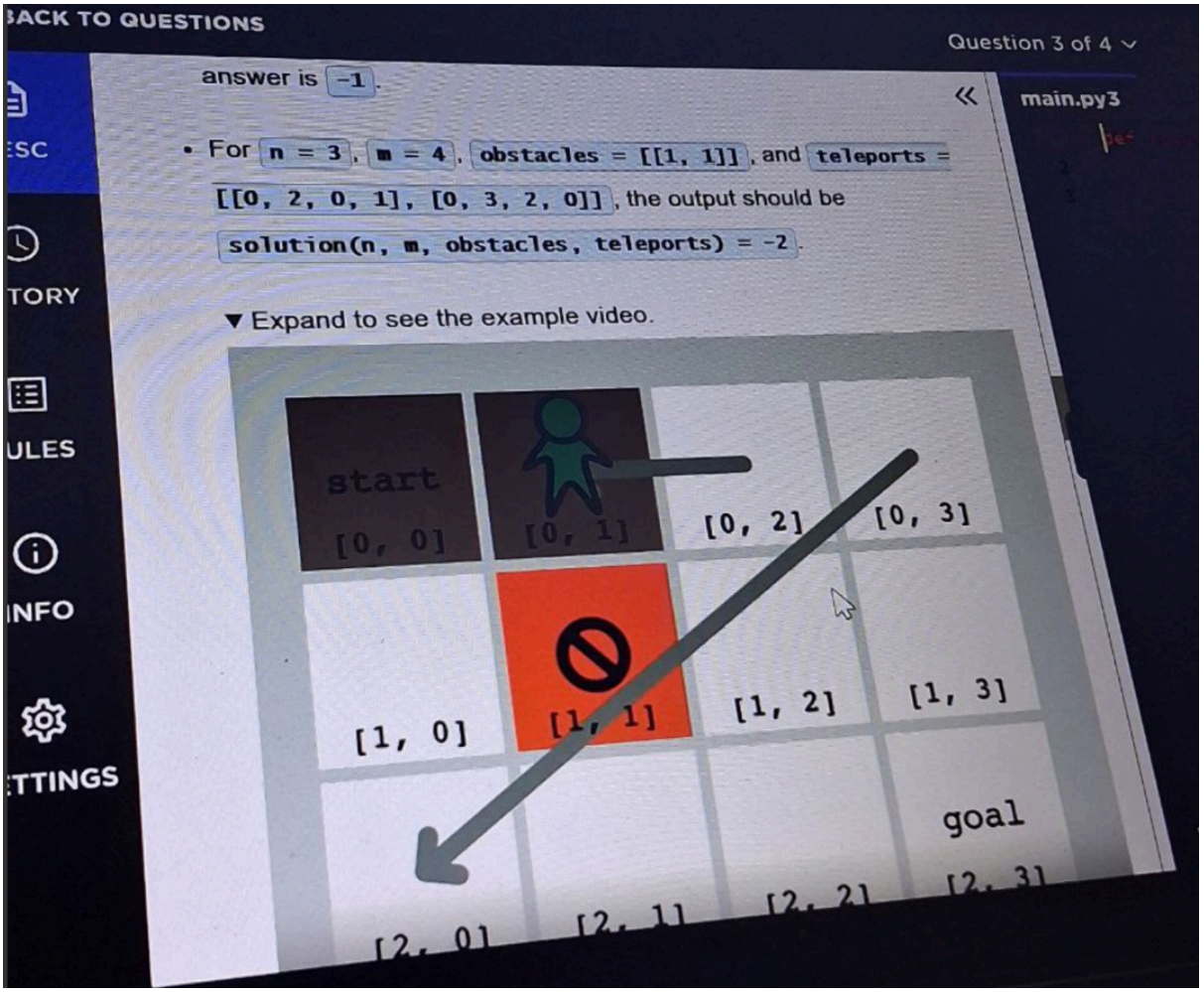
- For `n = 3`, `m = 4`, `obstacles = [[2, 0], [1, 0]]`, and `teleports = [[0, 1, 1, 1], [1, 2, 0, 2], [0, 3, 2, 1]]`, the output should be `solution(n, m, obstacles, teleports) = 9`.

▼ Expand to see the example video.

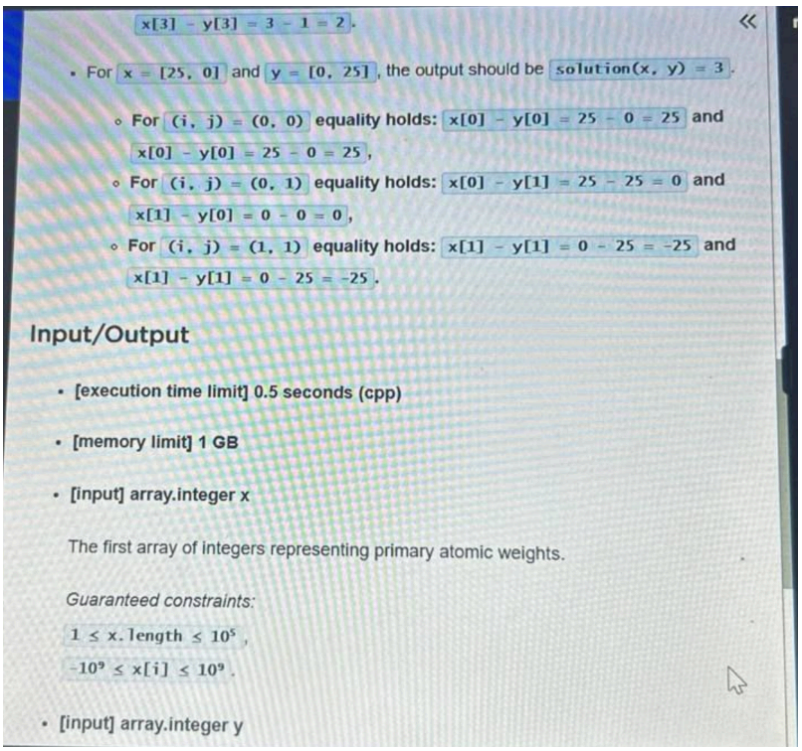


0:13 / 0:14

```
1
2
3
4
```


BACK TO QUESTIONS
Question 3 of 4
answer is -1.
main.py3
For n = 3, m = 4, obstacles = [[1, 1]], and teleports = [[0, 2, 0, 1], [0, 3, 2, 0]], the output should be solution(n, m, obstacles, teleports) = -2.
▼ Expand to see the example video.
start
[0, 0]
[0, 1]
[0, 2]
[0, 3]
[1, 0]
[1, 1]
[1, 2]
[1, 3]
goal
[2, 0]
[2, 1]
[2, 2]
[2, 3]

Q7.



`x[3] - y[3] = 3 - 1 = 2`.

- For `x = [25, 0]` and `y = [0, 25]`, the output should be `solution(x, y) = 3`.
  - For `(i, j) = (0, 0)` equality holds: `x[0] - y[0] = 25 - 0 = 25` and `x[0] - y[0] = 25 - 0 = 25`,
  - For `(i, j) = (0, 1)` equality holds: `x[0] - y[1] = 25 - 25 = 0` and `x[1] - y[0] = 0 - 0 = 0`,
  - For `(i, j) = (1, 1)` equality holds: `x[1] - y[1] = 0 - 25 = -25` and `x[1] - y[1] = 0 - 25 = -25`.

Input/Output

- **[execution time limit] 0.5 seconds (cpp)**
- **[memory limit] 1 GB**
- **[input] array.integer x**

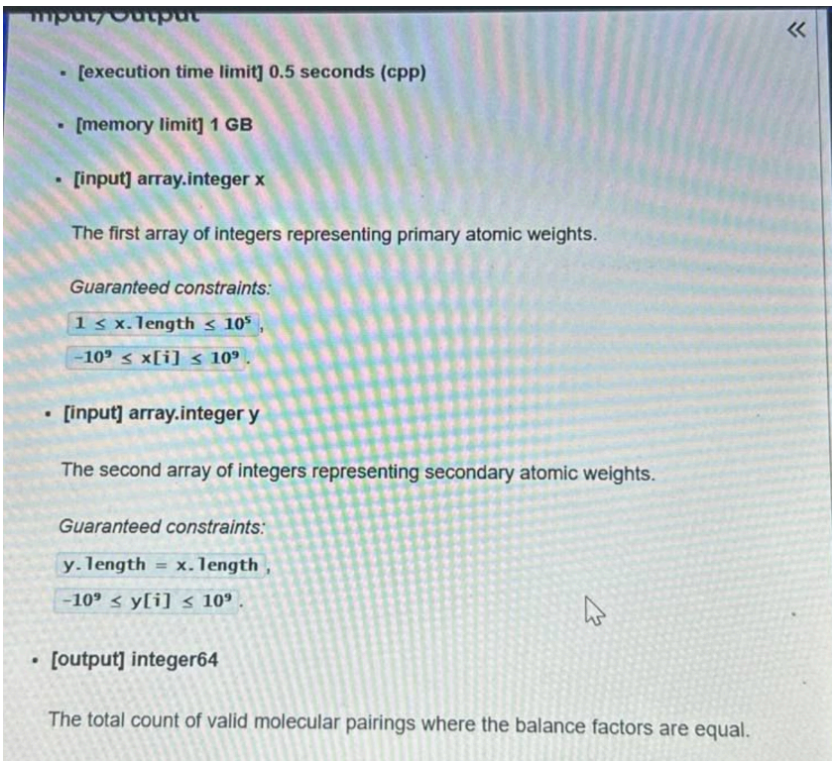
  The first array of integers representing primary atomic weights.

  Guaranteed constraints:

  $1 \le x.length \le 10^5$,

  $-10^9 \le x[i] \le 10^9$.

- [input] array.integer y



- **[execution time limit] 0.5 seconds (cpp)**
- **[memory limit] 1 GB**
- **[input] array.integer x**

  The first array of integers representing primary atomic weights.

  Guaranteed constraints:

  $1 \le x.length \le 10^5$,

  $-10^9 \le x[i] \le 10^9$.

- **[input] array.integer y**

  The second array of integers representing secondary atomic weights.

  Guaranteed constraints:

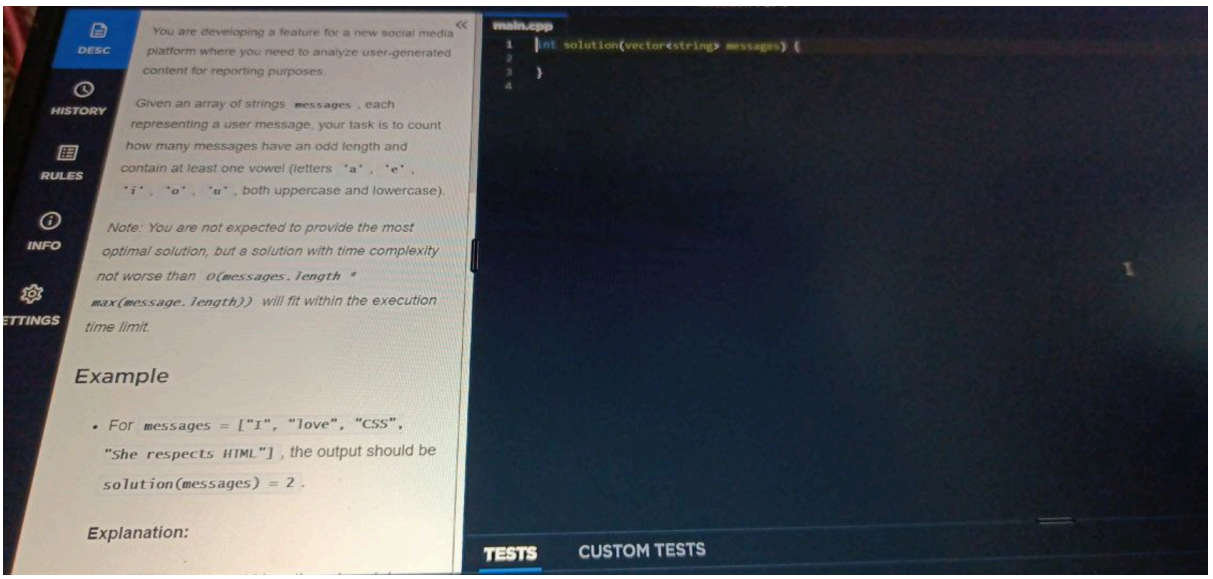
  $y.length = x.length$,

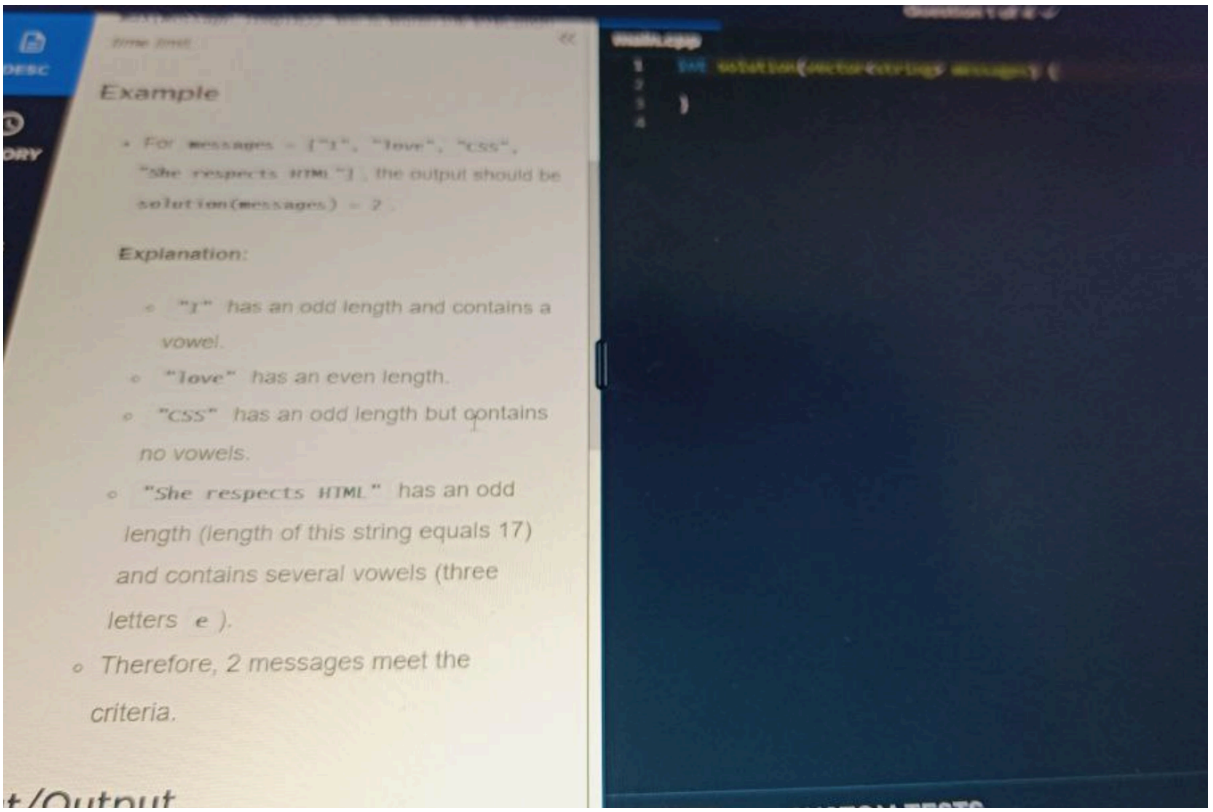
  $-10^9 \le y[i] \le 10^9$.

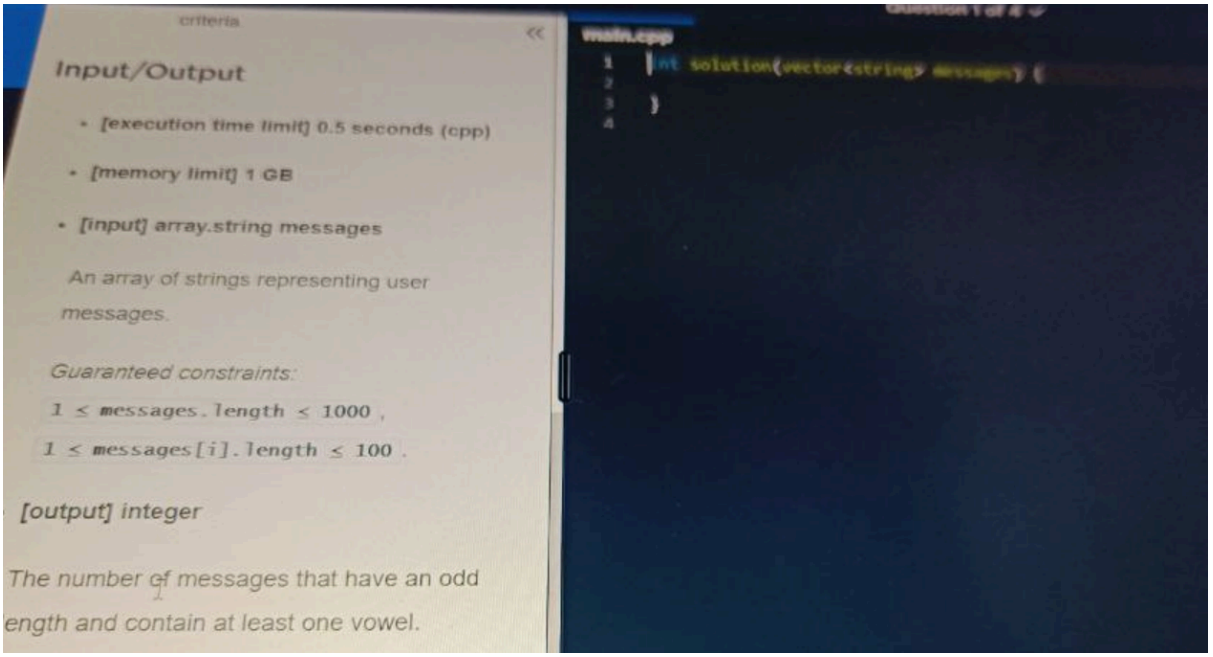
- **[output] integer64**

  The total count of valid molecular pairings where the balance factors are equal.

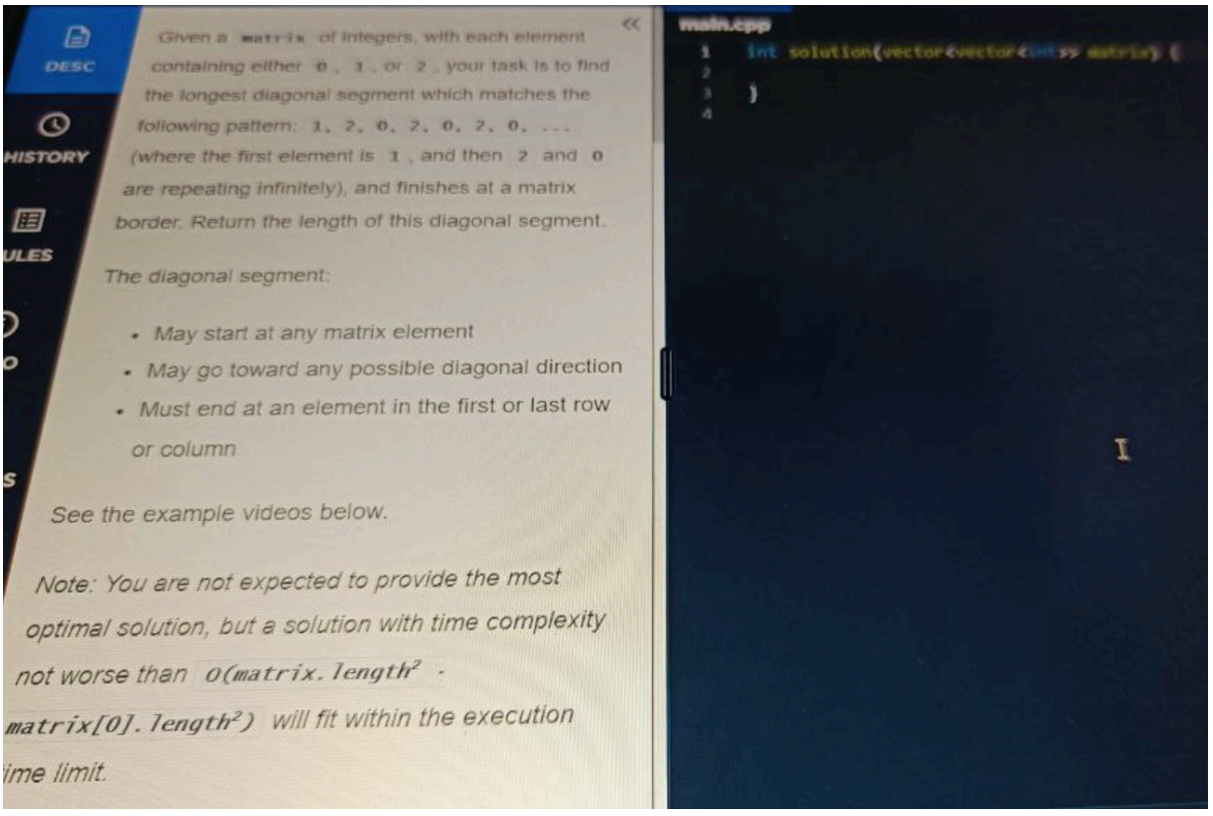
Q8.

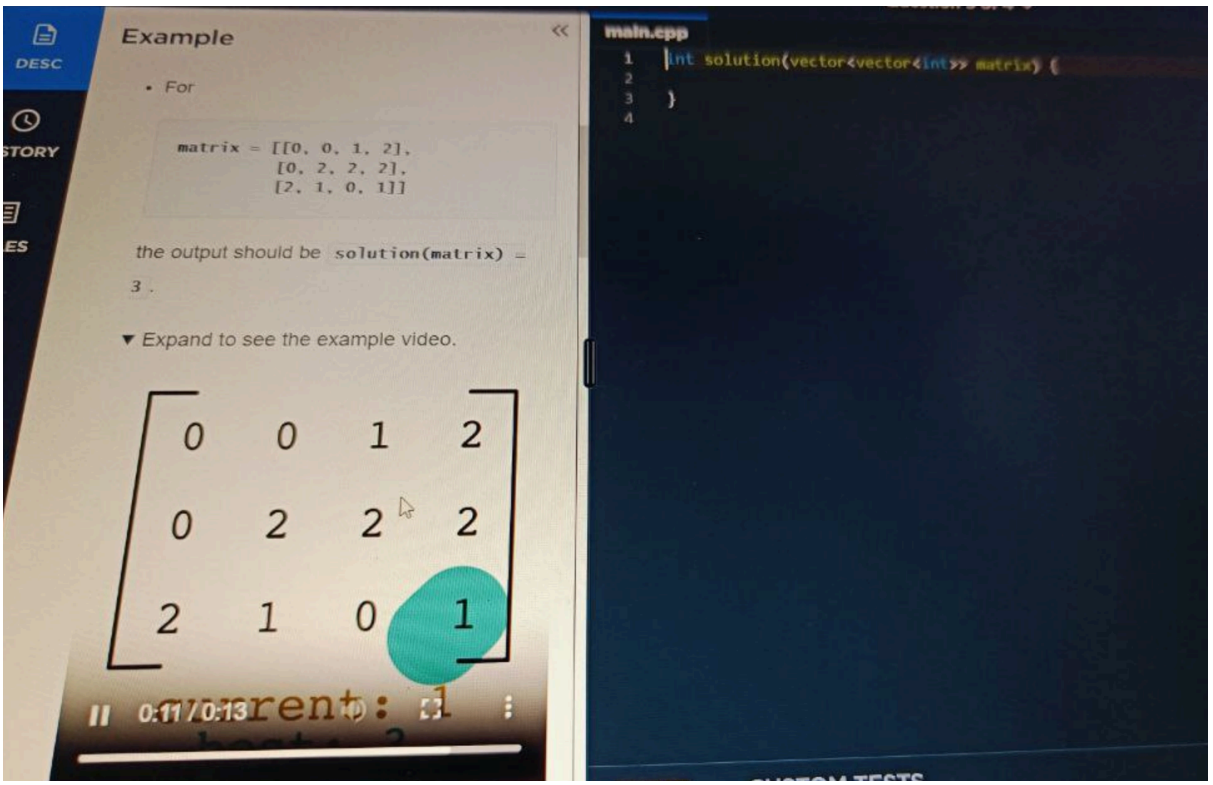

DESC
HISTORY
RULES
INFO
SETTINGS
You are developing a feature for a new social media platform where you need to analyze user-generated content for reporting purposes.
Given an array of strings messages, each representing a user message, your task is to count how many messages have an odd length and contain at least one vowel (letters 'a', 'e', 'i', 'o', 'u', both uppercase and lowercase).
Note: You are not expected to provide the most optimal solution, but a solution with time complexity not worse than O(messages.length * max(message.length)) will fit within the execution time limit.
Example
For messages = ["I", "love", "CSS", "She respects HTML"], the output should be solution(messages) = 2.
Explanation:
main.cpp
int solution(vector<string> messages) {
}
TESTS
CUSTOM TESTS

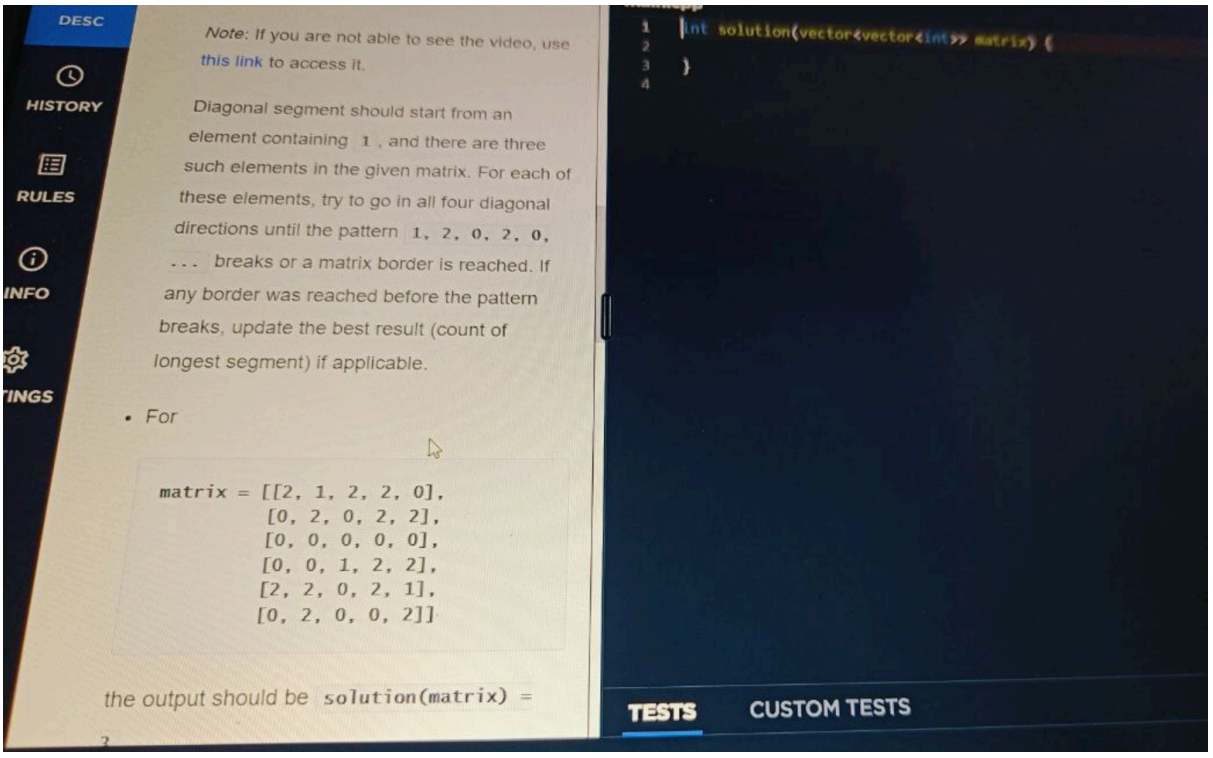

Example
For messages = ["I", "love", "CSS", "She respects HTML"], the output should be solution(messages) = 2.
Explanation:
"I" has an odd length and contains a vowel.
"love" has an even length.
"CSS" has an odd length but contains no vowels.
"She respects HTML" has an odd length (length of this string equals 17) and contains several vowels (three letters e).
Therefore, 2 messages meet the criteria.
main.cpp

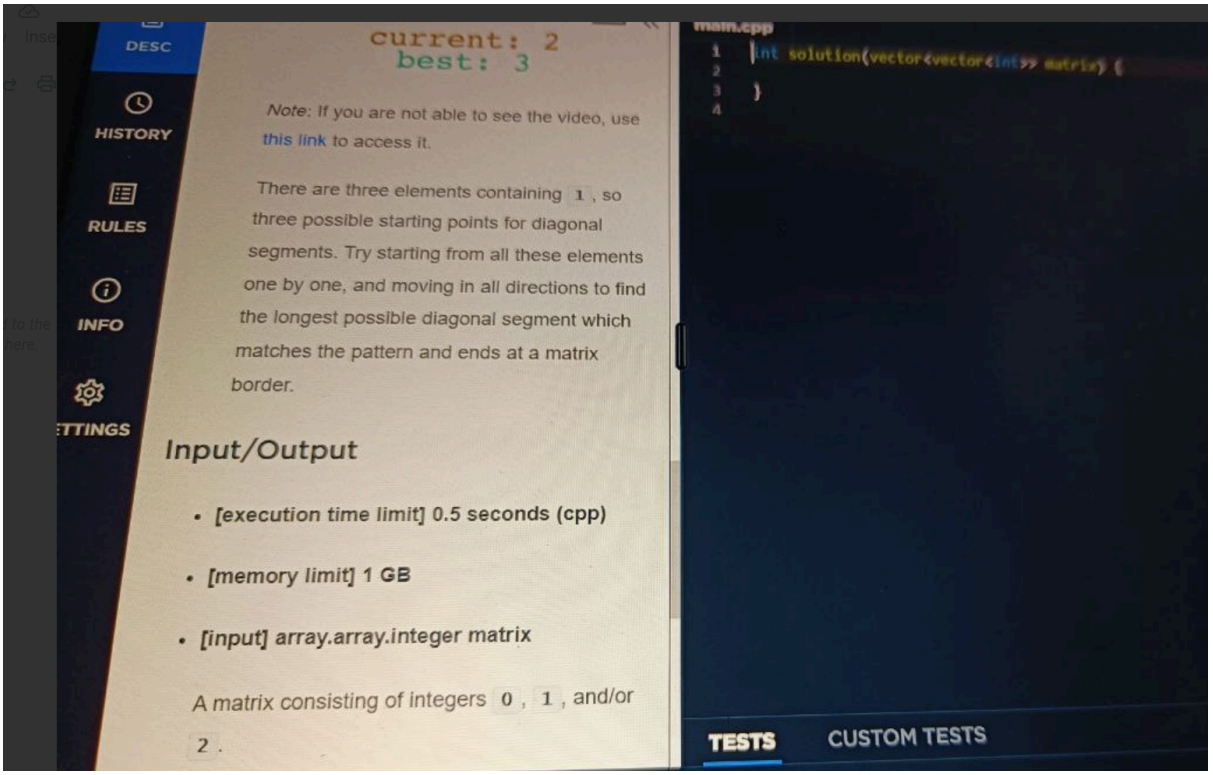

criteria
Input/Output
• [execution time limit] 0.5 seconds (cpp)
• [memory limit] 1 GB
• [input] array.string messages
An array of strings representing user messages.
Guaranteed constraints:
1 ≤ messages.length ≤ 1000,
1 ≤ messages[i].length ≤ 100.
[output] integer
The number of messages that have an odd length and contain at least one vowel.
main.cpp
int solution(vector<string> messages) {
}

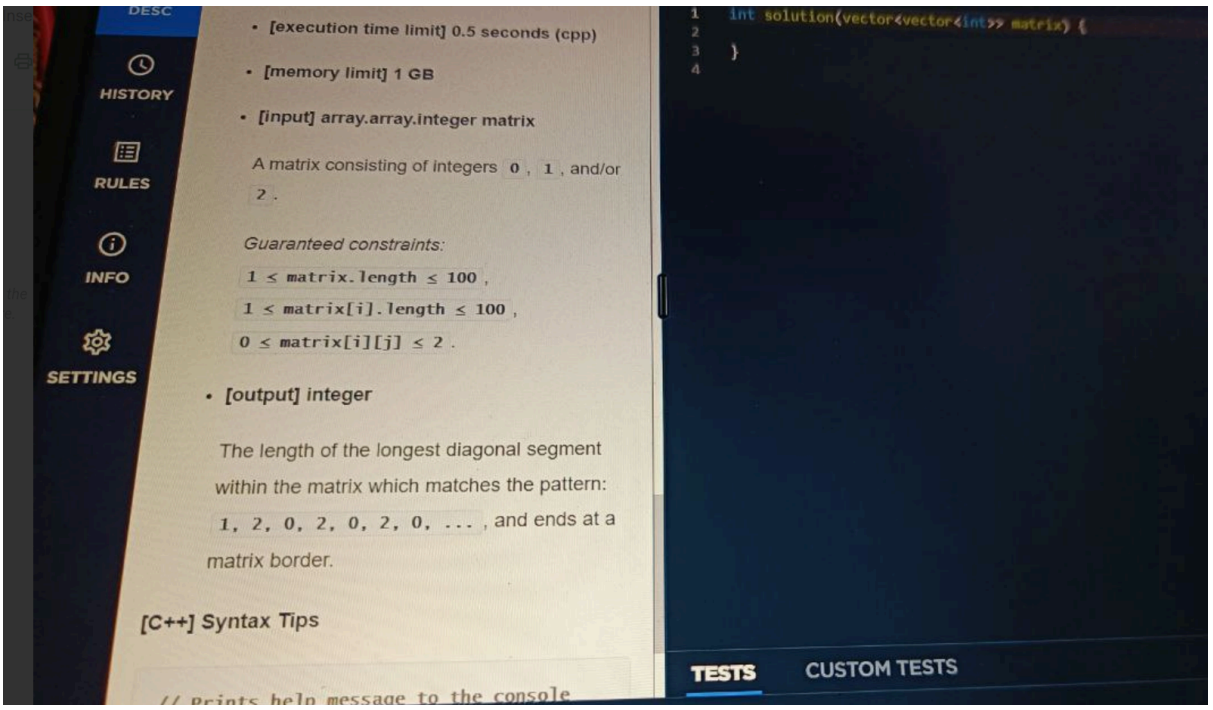
Q10.


DESC
HISTORY
Given a matrix of integers, with each element containing either 0, 1, or 2, your task is to find the longest diagonal segment which matches the following pattern: 1, 2, 0, 2, 0, 2, 0, ... (where the first element is 1, and then 2 and 0 are repeating infinitely), and finishes at a matrix border. Return the length of this diagonal segment.
The diagonal segment:
• May start at any matrix element
• May go toward any possible diagonal direction
• Must end at an element in the first or last row or column
See the example videos below.
Note: You are not expected to provide the most optimal solution, but a solution with time complexity not worse than O(matrix.length² · matrix[0].length²) will fit within the execution time limit.
main.cpp
int solution(vector<vector<int>> matrix) {
}

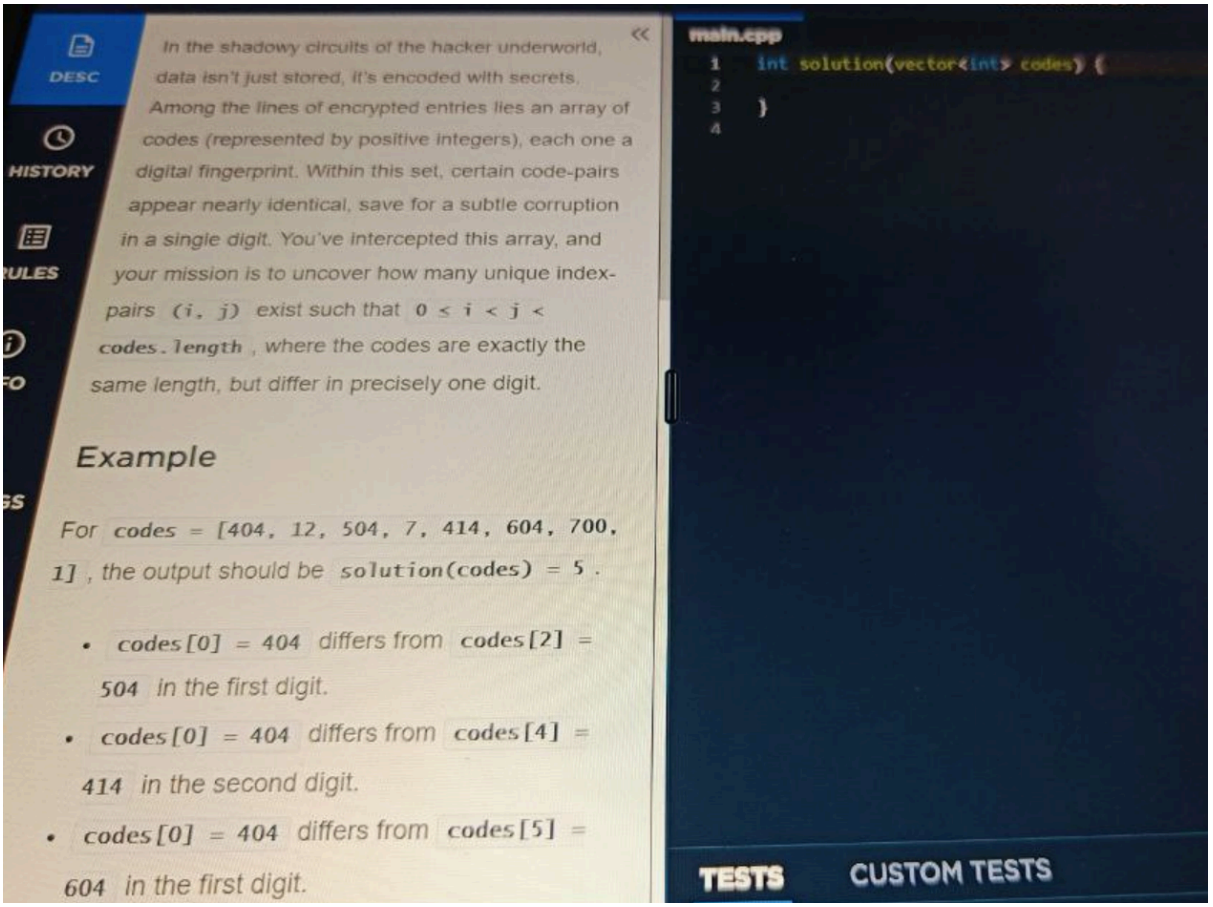

DESC
Example
For
matrix = [[0, 0, 1, 2],
[0, 2, 2, 2],
[2, 1, 0, 1]]
the output should be solution(matrix) = 3.
Expand to see the example video.
0 0 1 2
0 2 2 2
2 1 0 1
0:11 / 0:13
main.cpp
int solution(vector<vector<int>> matrix) {
}

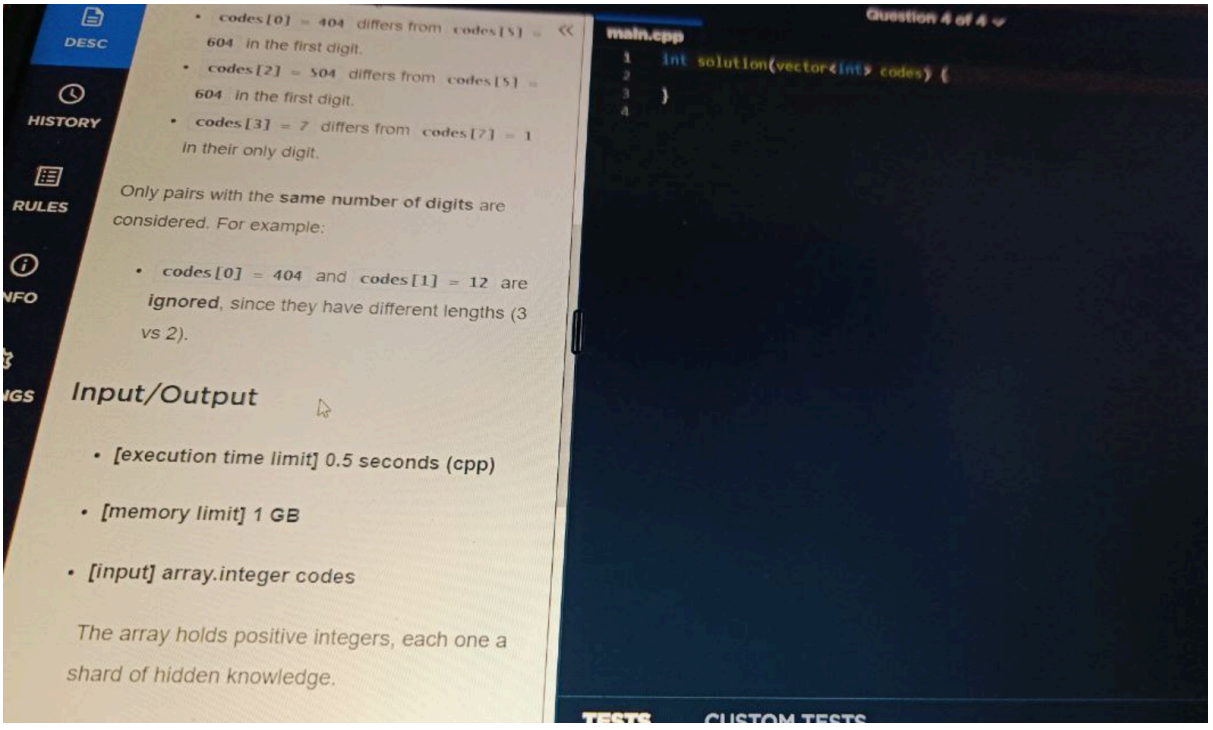

DESC
HISTORY
RULES
INFO
Note: If you are not able to see the video, use this link to access it.
Diagonal segment should start from an element containing 1, and there are three such elements in the given matrix. For each of these elements, try to go in all four diagonal directions until the pattern 1, 2, 0, 2, 0, ... breaks or a matrix border is reached. If any border was reached before the pattern breaks, update the best result (count of longest segment) if applicable.
For
matrix = [[2, 1, 2, 2, 0],
[0, 2, 0, 2, 2],
[0, 0, 0, 0, 0],
[0, 0, 1, 2, 2],
[2, 2, 0, 2, 1],
[0, 2, 0, 0, 2]]
the output should be solution(matrix) =
3
int solution(vector<vector<int>> matrix) {
}
TESTS
CUSTOM TESTS

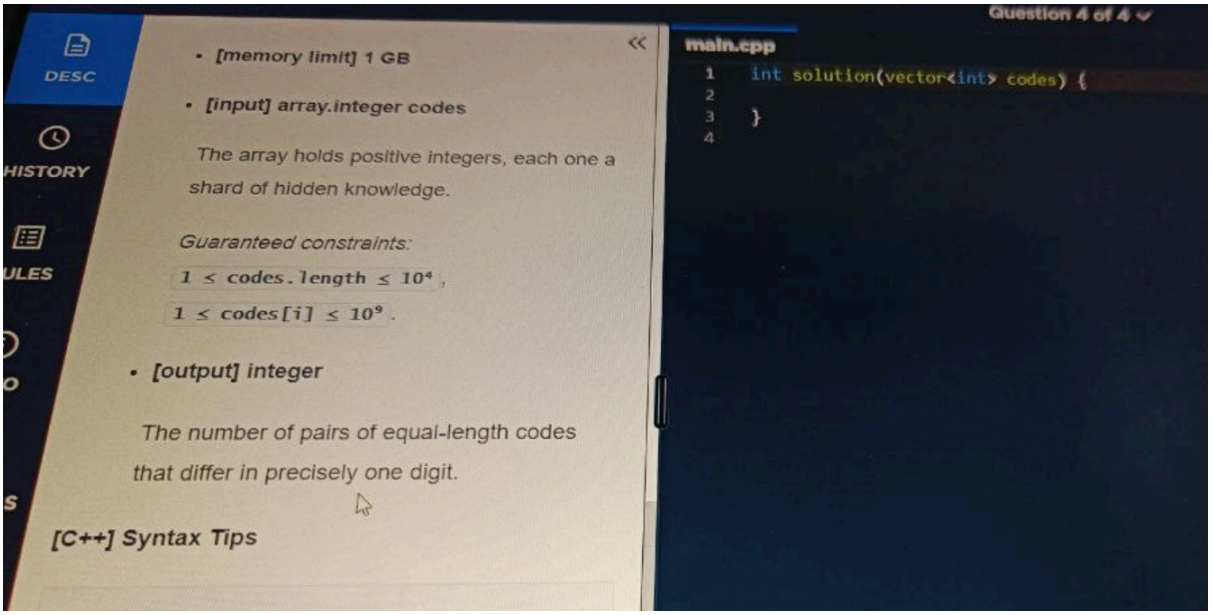

DESC
HISTORY
RULES
INFO
SETTINGS
current: 2
best: 3
Note: If you are not able to see the video, use this link to access it.
There are three elements containing 1, so three possible starting points for diagonal segments. Try starting from all these elements one by one, and moving in all directions to find the longest possible diagonal segment which matches the pattern and ends at a matrix border.
Input/Output
[execution time limit] 0.5 seconds (cpp)
[memory limit] 1 GB
[input] array.array.integer matrix
A matrix consisting of integers 0, 1, and/or 2.
main.cpp
int solution(vector<vector<int>> matrix) {
}
TESTS
CUSTOM TESTS


DESC
HISTORY
RULES
INFO
SETTINGS
[execution time limit] 0.5 seconds (cpp)
[memory limit] 1 GB
[input] array.array.integer matrix
A matrix consisting of integers 0, 1, and/or 2.
Guaranteed constraints:
1 ≤ matrix.length ≤ 100,
1 ≤ matrix[i].length ≤ 100,
0 ≤ matrix[i][j] ≤ 2.
[output] integer
The length of the longest diagonal segment within the matrix which matches the pattern: 1, 2, 0, 2, 0, 2, 0, ..., and ends at a matrix border.
[C++] Syntax Tips
// Prints help message to the console
int solution(vector<vector<int>> matrix) {
}
TESTS
CUSTOM TESTS

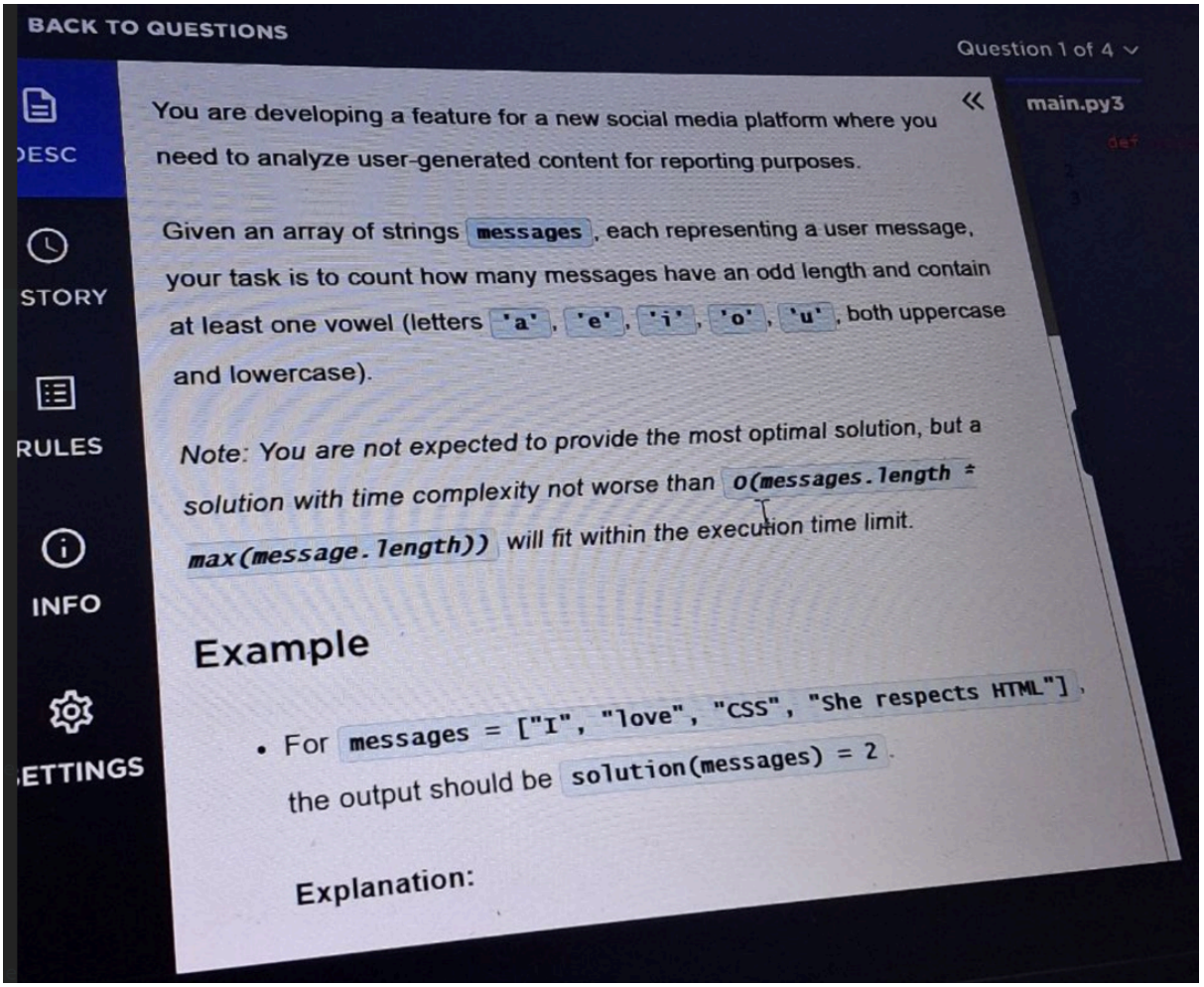
Q11.

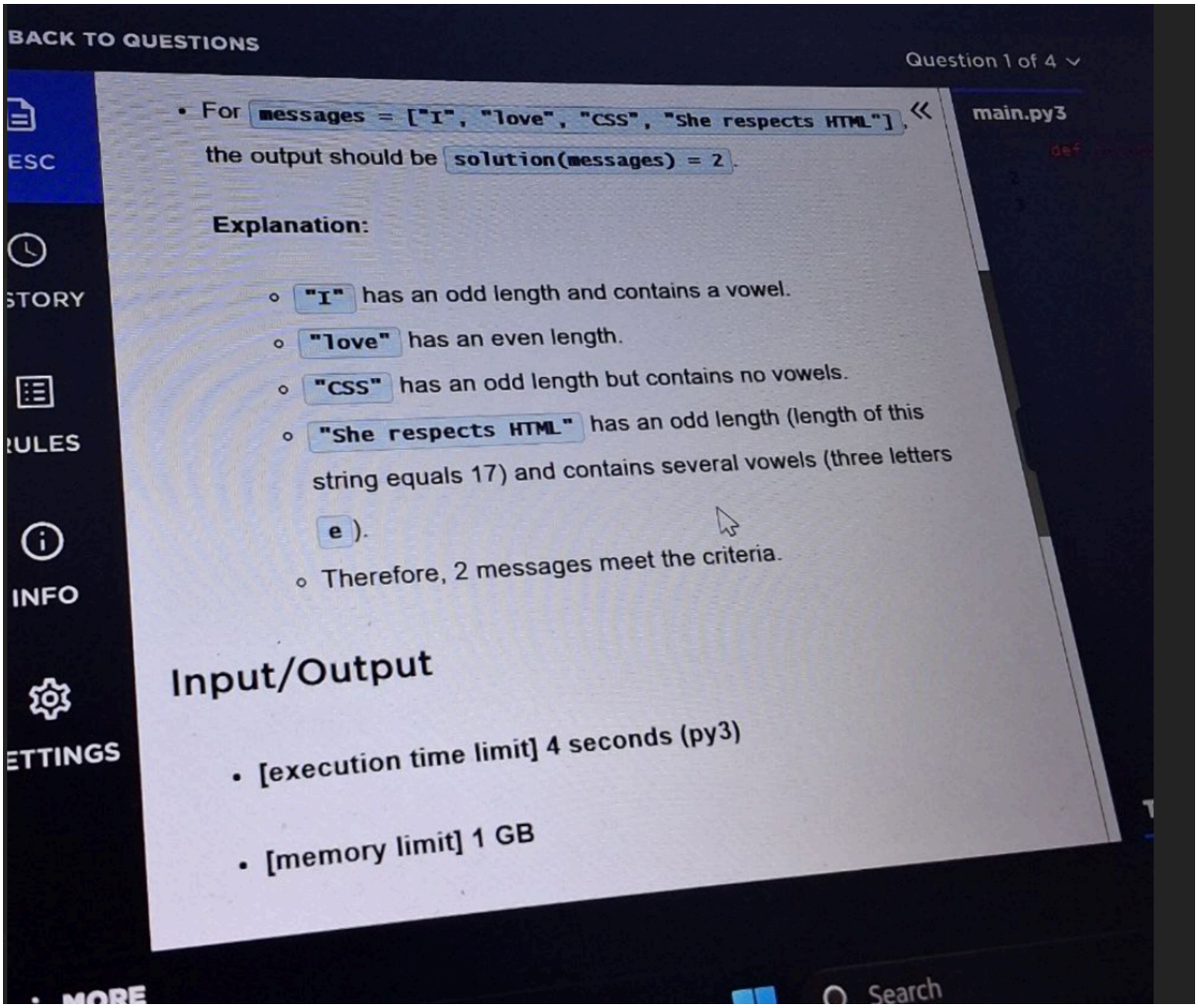

DESC
HISTORY
RULES
In the shadowy circuits of the hacker underworld, data isn't just stored, it's encoded with secrets. Among the lines of encrypted entries lies an array of codes (represented by positive integers), each one a digital fingerprint. Within this set, certain code-pairs appear nearly identical, save for a subtle corruption in a single digit. You've intercepted this array, and your mission is to uncover how many unique index-pairs (i, j) exist such that 0 ≤ i < j < codes.length, where the codes are exactly the same length, but differ in precisely one digit.
Example
For codes = [404, 12, 504, 7, 414, 604, 700, 1], the output should be solution(codes) = 5.
codes[0] = 404 differs from codes[2] = 504 in the first digit.
codes[0] = 404 differs from codes[4] = 414 in the second digit.
codes[0] = 404 differs from codes[5] = 604 in the first digit.
main.cpp
int solution(vector<int> codes) {
}
TESTS
CUSTOM TESTS

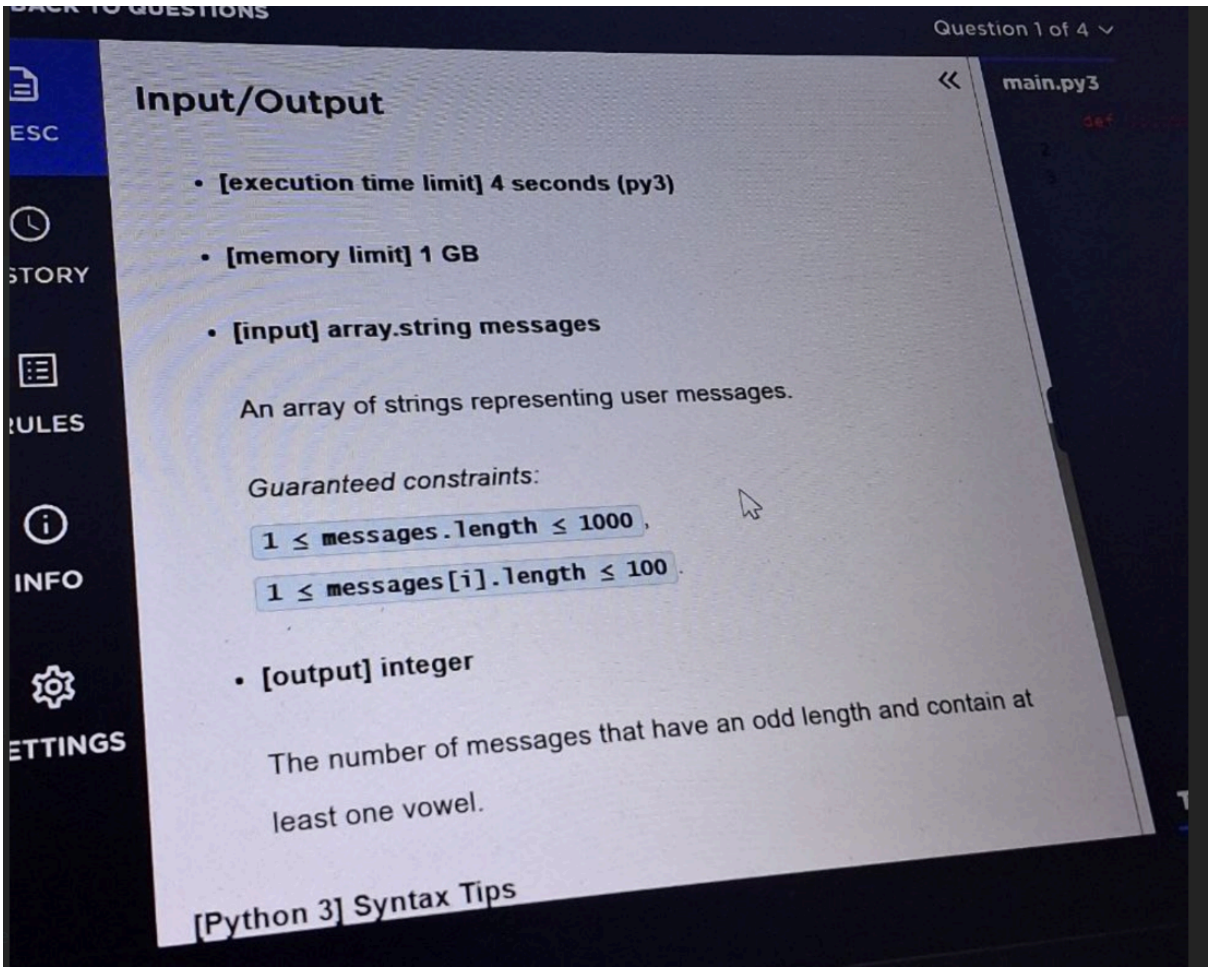

DESC
HISTORY
RULES
codes[0] = 404 differs from codes[5] = 604 in the first digit.
codes[2] = 504 differs from codes[5] = 604 in the first digit.
codes[3] = 7 differs from codes[7] = 1 in their only digit.
Only pairs with the same number of digits are considered. For example:
codes[0] = 404 and codes[1] = 12 are ignored, since they have different lengths (3 vs 2).
Input/Output
[execution time limit] 0.5 seconds (cpp)
[memory limit] 1 GB
[input] array.integer codes
The array holds positive integers, each one a shard of hidden knowledge.
Question 4 of 4
main.cpp
int solution(vector<int> codes) {
}


Question 4 of 4
DESC
HISTORY
ULES
• [memory limit] 1 GB
• [input] array.integer codes
The array holds positive integers, each one a shard of hidden knowledge.
Guaranteed constraints:
1 ≤ codes.length ≤ 10⁴,
1 ≤ codes[i] ≤ 10⁹.
• [output] integer
The number of pairs of equal-length codes that differ in precisely one digit.
[C++] Syntax Tips
main.cpp
1 int solution(vector<int> codes) {
2
3 }
4

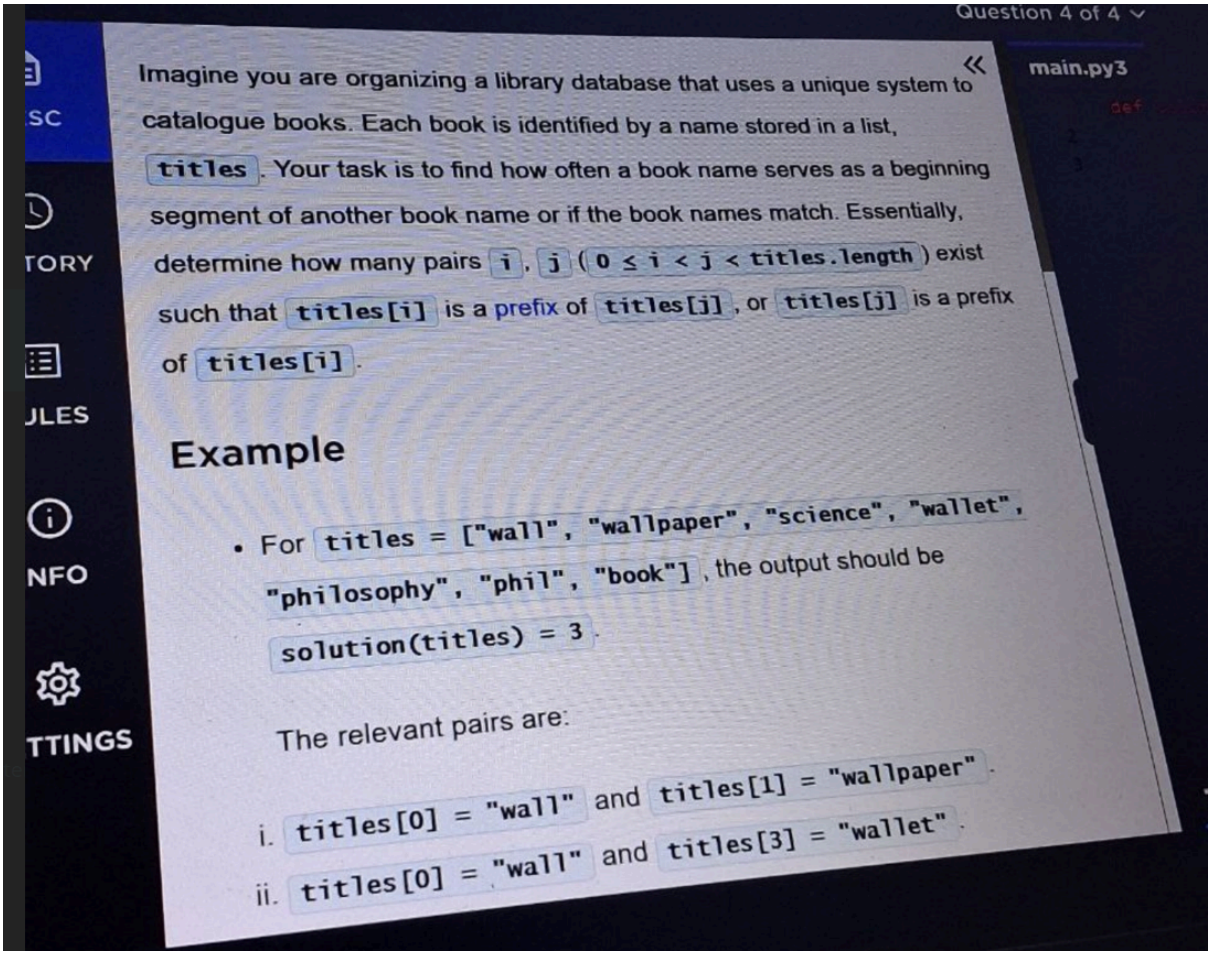
Q12.

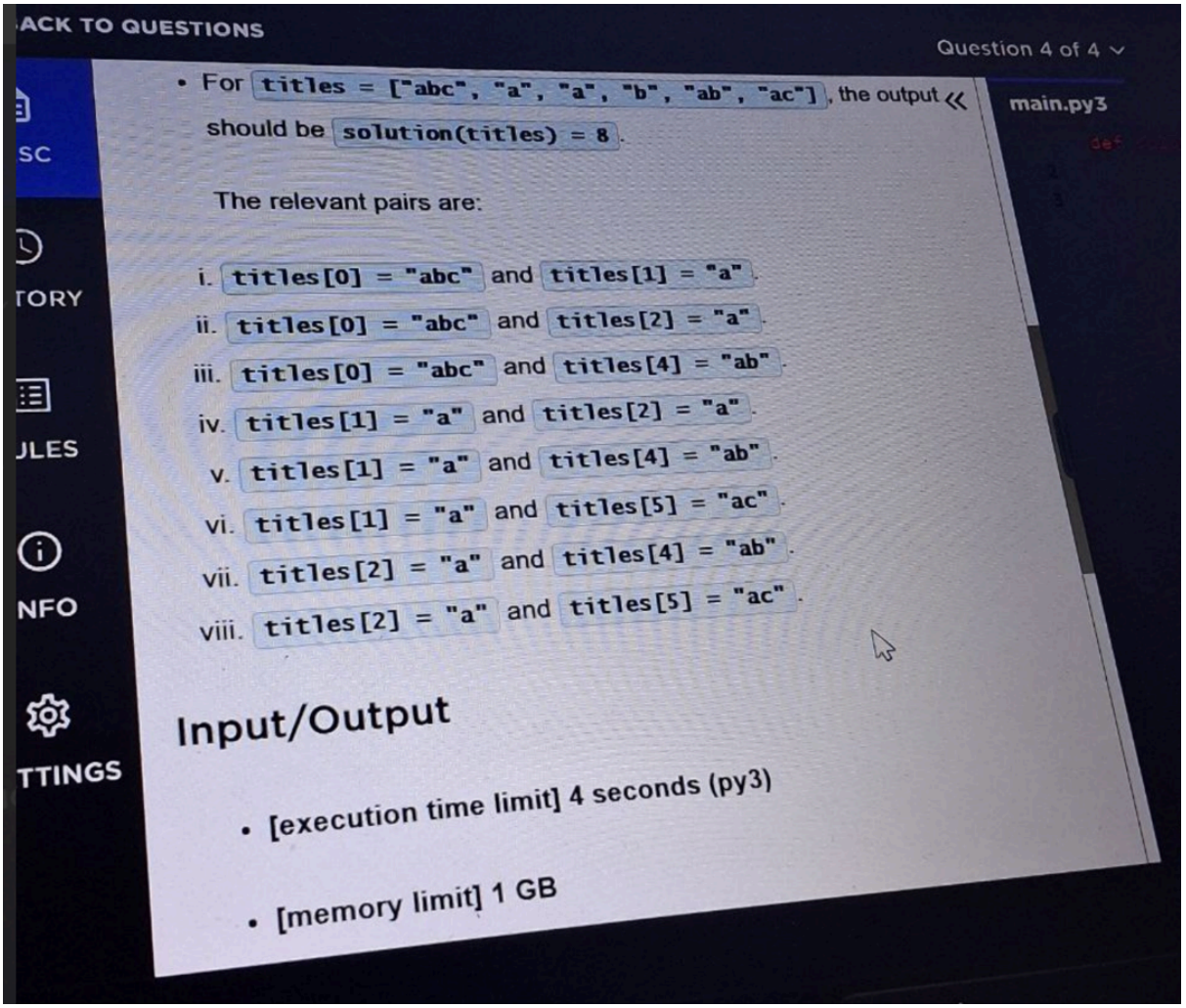

BACK TO QUESTIONS
Question 1 of 4
DESC
STORY
RULES
INFO
ETTINGS
main.py3
You are developing a feature for a new social media platform where you need to analyze user-generated content for reporting purposes.
Given an array of strings messages, each representing a user message, your task is to count how many messages have an odd length and contain at least one vowel (letters 'a', 'e', 'i', 'o', 'u', both uppercase and lowercase).
Note: You are not expected to provide the most optimal solution, but a solution with time complexity not worse than O(messages.length * max(message.length)) will fit within the execution time limit.
Example
• For messages = ["I", "love", "CSS", "She respects HTML"], the output should be solution(messages) = 2.
Explanation:

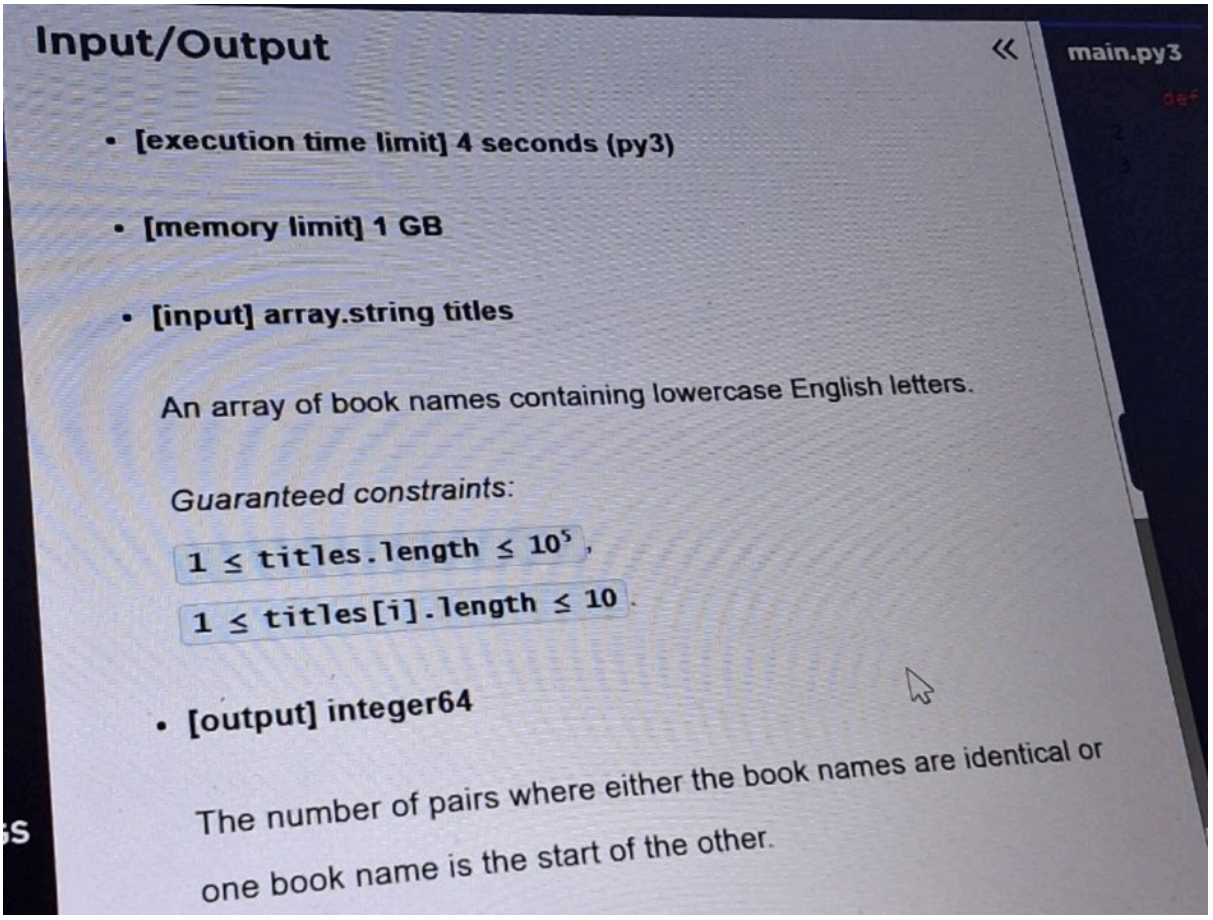

BACK TO QUESTIONS
Question 1 of 4
ESC
STORY
RULES
INFO
ETTINGS
main.py3
• For messages = ["I", "love", "CSS", "She respects HTML"], the output should be solution(messages) = 2.
Explanation:
"I" has an odd length and contains a vowel.
"love" has an even length.
"CSS" has an odd length but contains no vowels.
"She respects HTML" has an odd length (length of this string equals 17) and contains several vowels (three letters e).
Therefore, 2 messages meet the criteria.
Input/Output
• [execution time limit] 4 seconds (py3)
• [memory limit] 1 GB
MORE
Search


Question 1 of 4
main.py3
ESC
STORY
RULES
INFO
ETTINGS
Input/Output
• [execution time limit] 4 seconds (py3)
• [memory limit] 1 GB
• [input] array.string messages
An array of strings representing user messages.
Guaranteed constraints:
1 ≤ messages.length ≤ 1000,
1 ≤ messages[i].length ≤ 100.
• [output] integer
The number of messages that have an odd length and contain at least one vowel.
[Python 3] Syntax Tips

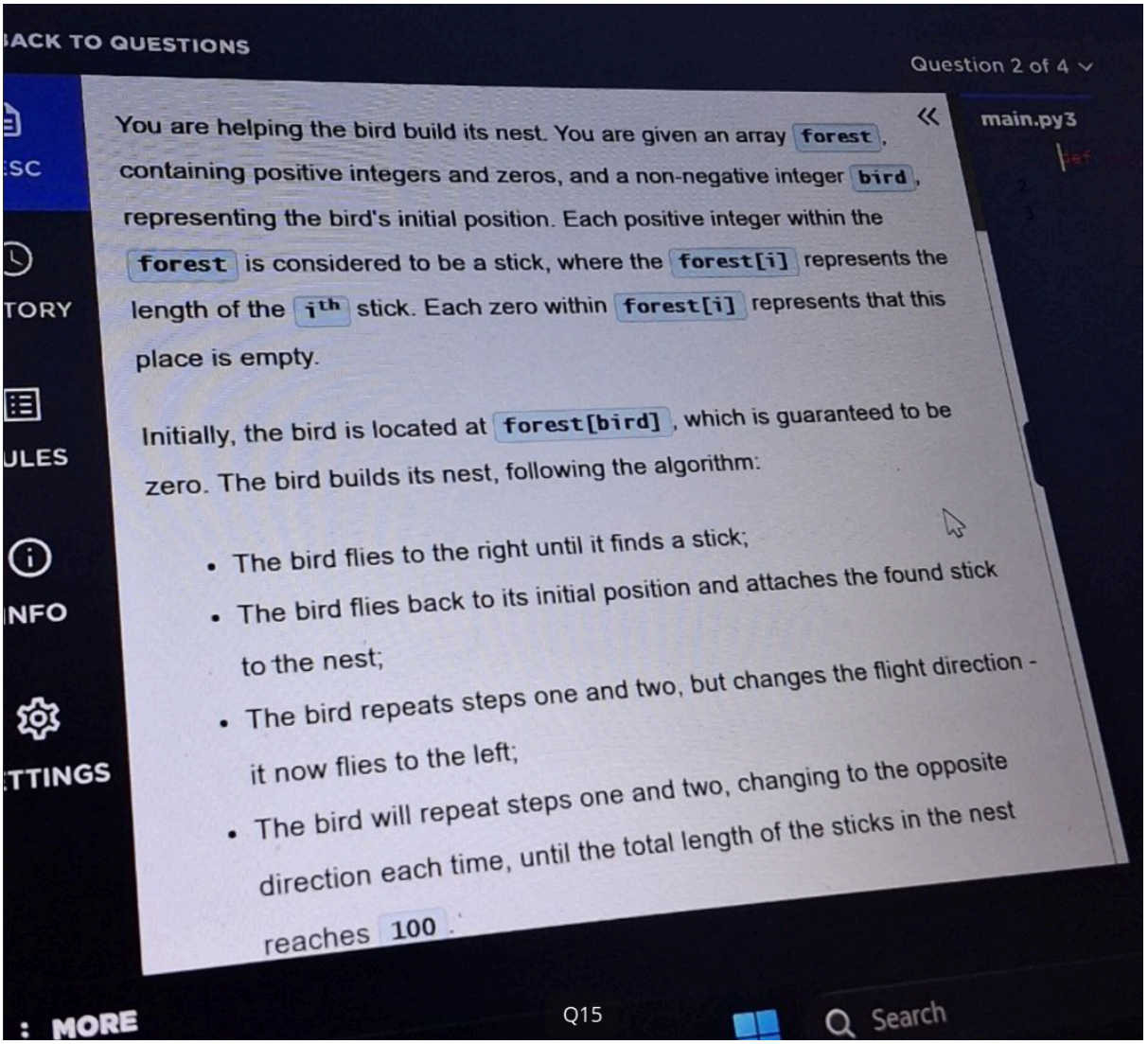
Q13.


Question 4 of 4
main.py3
SC
TORY
JLES
NFO
TTINGS
Imagine you are organizing a library database that uses a unique system to catalogue books. Each book is identified by a name stored in a list, titles. Your task is to find how often a book name serves as a beginning segment of another book name or if the book names match. Essentially, determine how many pairs i, j (0 ≤ i < j < titles.length) exist such that titles[i] is a prefix of titles[j], or titles[j] is a prefix of titles[i].
Example
For titles = ["wall", "wallpaper", "science", "wallet", "philosophy", "phil", "book"], the output should be solution(titles) = 3.
The relevant pairs are:
i. titles[0] = "wall" and titles[1] = "wallpaper".
ii. titles[0] = "wall" and titles[3] = "wallet".

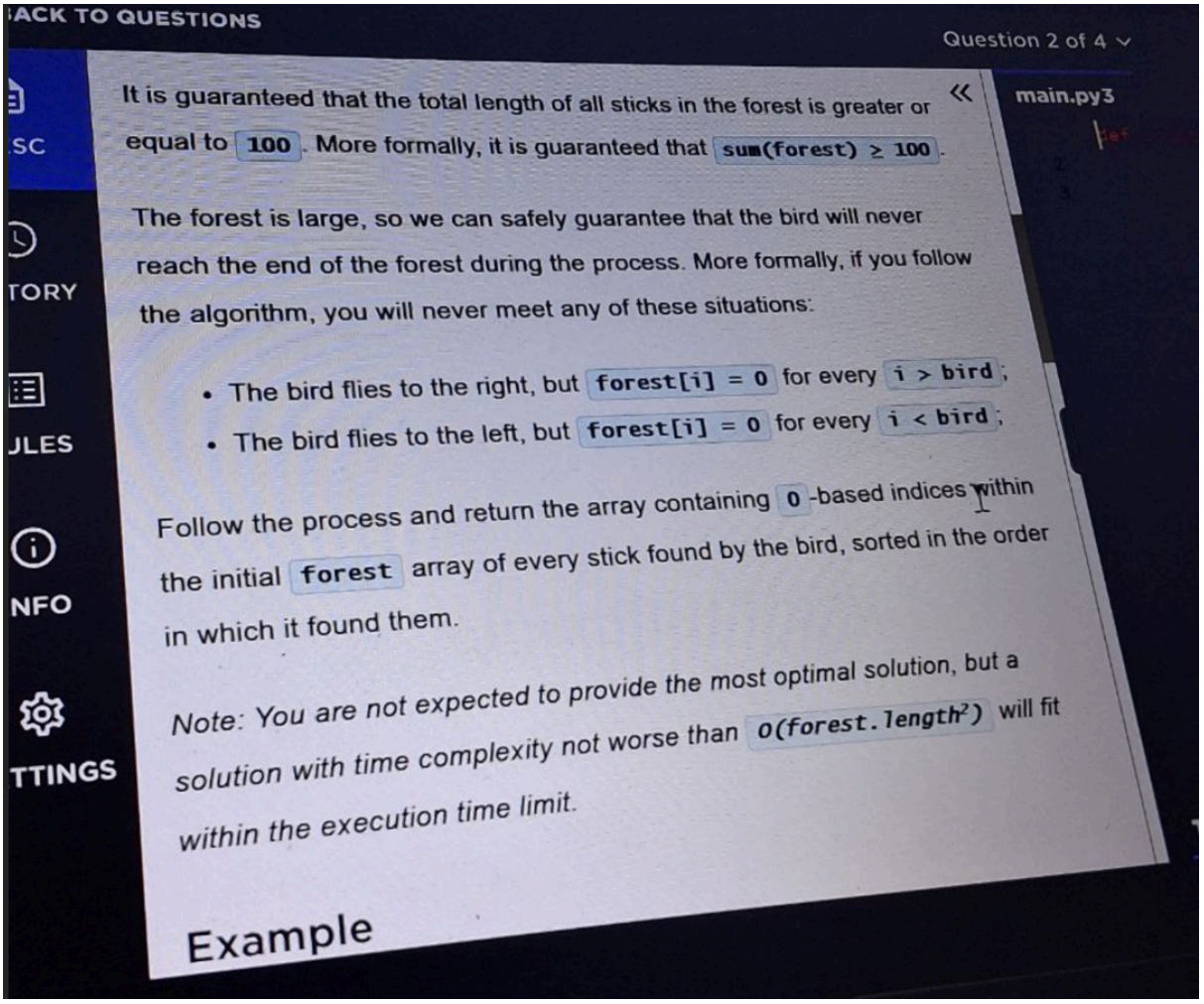

ACK TO QUESTIONS
Question 4 of 4
SC
TORY
JLES
NFO
TTINGS
main.py3
• For titles = ["abc", "a", "a", "b", "ab", "ac"], the output should be solution(titles) = 8.
The relevant pairs are:
i. titles[0] = "abc" and titles[1] = "a".
ii. titles[0] = "abc" and titles[2] = "a".
iii. titles[0] = "abc" and titles[4] = "ab".
iv. titles[1] = "a" and titles[2] = "a".
v. titles[1] = "a" and titles[4] = "ab".
vi. titles[1] = "a" and titles[5] = "ac".
vii. titles[2] = "a" and titles[4] = "ab".
viii. titles[2] = "a" and titles[5] = "ac".
Input/Output
• [execution time limit] 4 seconds (py3)
• [memory limit] 1 GB

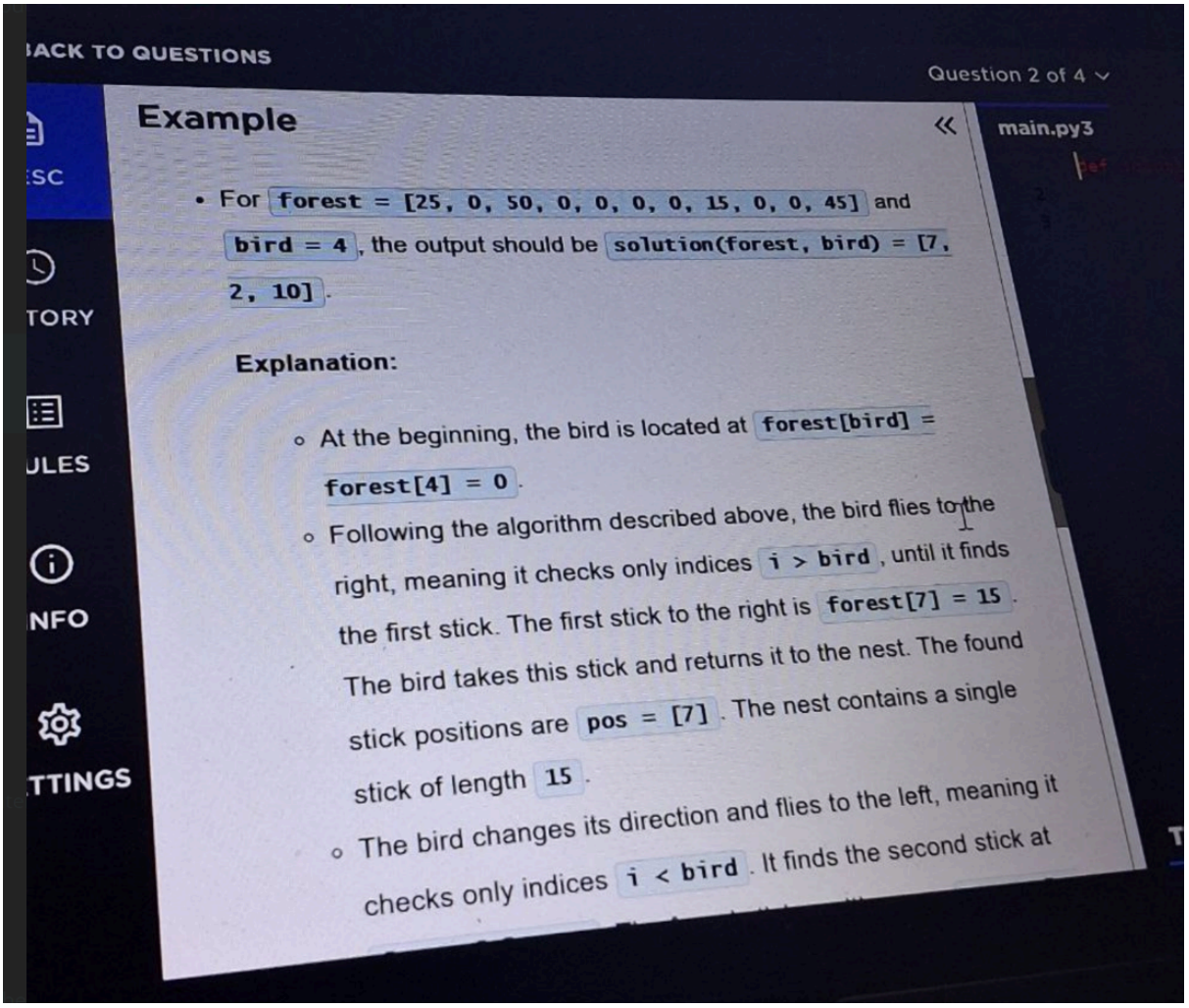

Input/Output
main.py3
• [execution time limit] 4 seconds (py3)
• [memory limit] 1 GB
• [input] array.string titles
An array of book names containing lowercase English letters.
Guaranteed constraints:
1 ≤ titles.length ≤ 10^5,
1 ≤ titles[i].length ≤ 10.
• [output] integer64
The number of pairs where either the book names are identical or one book name is the start of the other.

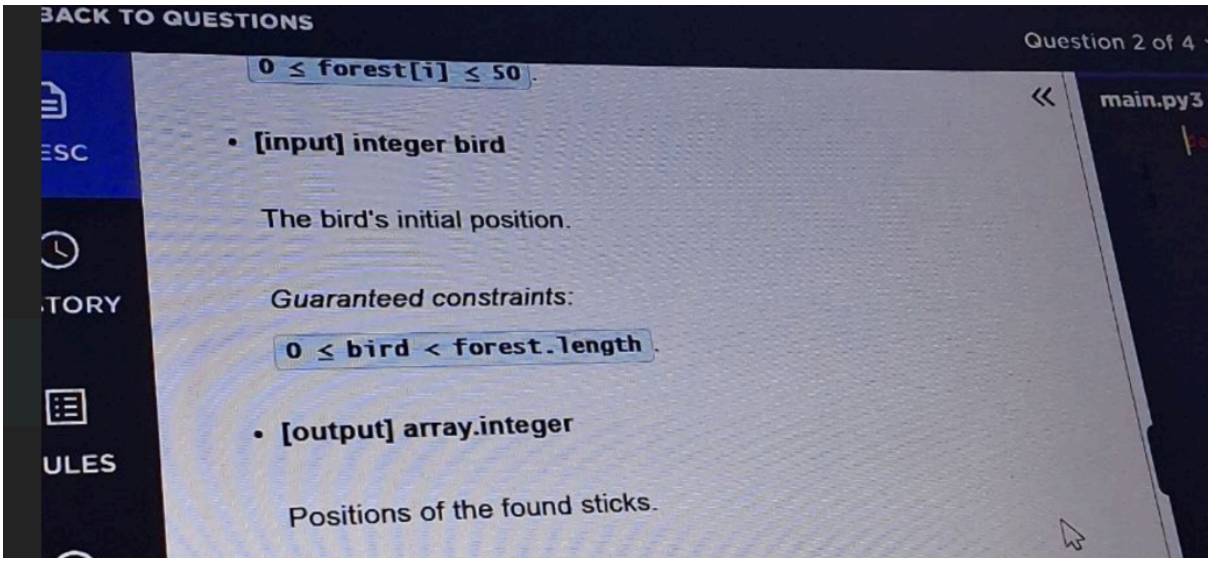
Q14.


ACK TO QUESTIONS
Question 2 of 4
main.py3
SC
TORY
ULES
NFO
TTINGS
: MORE
You are helping the bird build its nest. You are given an array forest, containing positive integers and zeros, and a non-negative integer bird, representing the bird's initial position. Each positive integer within the forest is considered to be a stick, where the forest[i] represents the length of the ith stick. Each zero within forest[i] represents that this place is empty.
Initially, the bird is located at forest[bird], which is guaranteed to be zero. The bird builds its nest, following the algorithm:
The bird flies to the right until it finds a stick;
The bird flies back to its initial position and attaches the found stick to the nest;
The bird repeats steps one and two, but changes the flight direction - it now flies to the left;
The bird will repeat steps one and two, changing to the opposite direction each time, until the total length of the sticks in the nest reaches 100.
Search

Q15


ACK TO QUESTIONS
Question 2 of 4
SC
TORY
JLES
NFO
TTINGS
main.py3
It is guaranteed that the total length of all sticks in the forest is greater or equal to 100. More formally, it is guaranteed that sum(forest) ≥ 100.
The forest is large, so we can safely guarantee that the bird will never reach the end of the forest during the process. More formally, if you follow the algorithm, you will never meet any of these situations:
The bird flies to the right, but forest[i] = 0 for every i > bird;
The bird flies to the left, but forest[i] = 0 for every i < bird;
Follow the process and return the array containing 0-based indices within the initial forest array of every stick found by the bird, sorted in the order in which it found them.
Note: You are not expected to provide the most optimal solution, but a solution with time complexity not worse than O(forest.length²) will fit within the execution time limit.
Example


BACK TO QUESTIONS
Question 2 of 4
Example
main.py3
For forest = [25, 0, 50, 0, 0, 0, 0, 15, 0, 0, 45] and bird = 4, the output should be solution(forest, bird) = [7, 2, 10].
Explanation:
At the beginning, the bird is located at forest[bird] = forest[4] = 0.
Following the algorithm described above, the bird flies to the right, meaning it checks only indices i > bird, until it finds the first stick. The first stick to the right is forest[7] = 15. The bird takes this stick and returns it to the nest. The found stick positions are pos = [7]. The nest contains a single stick of length 15.
The bird changes its direction and flies to the left, meaning it checks only indices i < bird. It finds the second stick at


BACK TO QUESTIONS
Question 2 of 4
0 ≤ forest[i] ≤ 50.
main.py3
[input] integer bird
The bird's initial position.
Guaranteed constraints:
0 ≤ bird < forest.length.
[output] array.integer
Positions of the found sticks.

Q15.

You are developing a text filtering feature for a messaging app to help users customize their message display.

Given a string `inputString` representing a user's message, remove all vowels (`'a'`, `'e'`, `'i'`, `'o'`, `'u'`, both uppercase and lowercase) and reverse the order of the remaining characters. Return the resulting string.

Note: You are not expected to provide the most optimal solution, but a solution with time complexity not worse than $O(inputString.length^2)$ *will fit within the execution time limit.*

Example

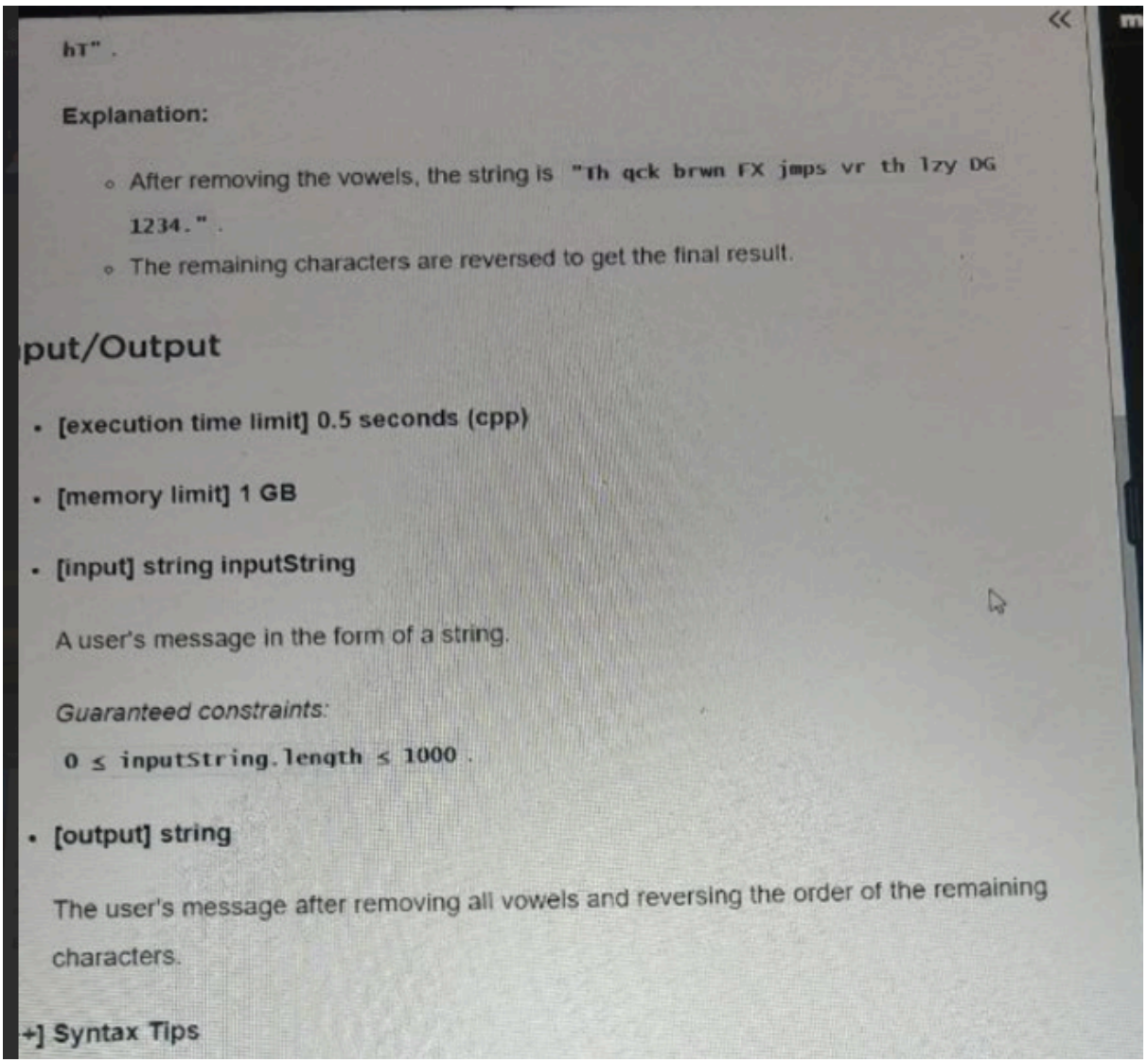
- For `inputString = "Hello world!"`, the output should be `solution(inputString) = "!dlrw llH"`.

  Explanation:

  - The vowels (`'e'`, `'o'`, `'o'`) are removed. `inputString` becomes `"Hll wrld!"`.
  - The remaining characters are reversed to get `"!dlrw llH"`.

- For `inputString = "The quick brown FOX jumps over the lazy DOG 1234."`, the output should be `solution(inputString) = ".4321 GD yzl ht rv spmj XF nwrb kcq hT"`.

  Explanation:



hT" .

Explanation:

- After removing the vowels, the string is `"Th qck brwn FX jmps vr th lzy DG 1234."` .
- The remaining characters are reversed to get the final result.

put/Output

- **[execution time limit] 0.5 seconds (cpp)**
- **[memory limit] 1 GB**
- **[input] string inputString**

  A user's message in the form of a string.

  Guaranteed constraints:

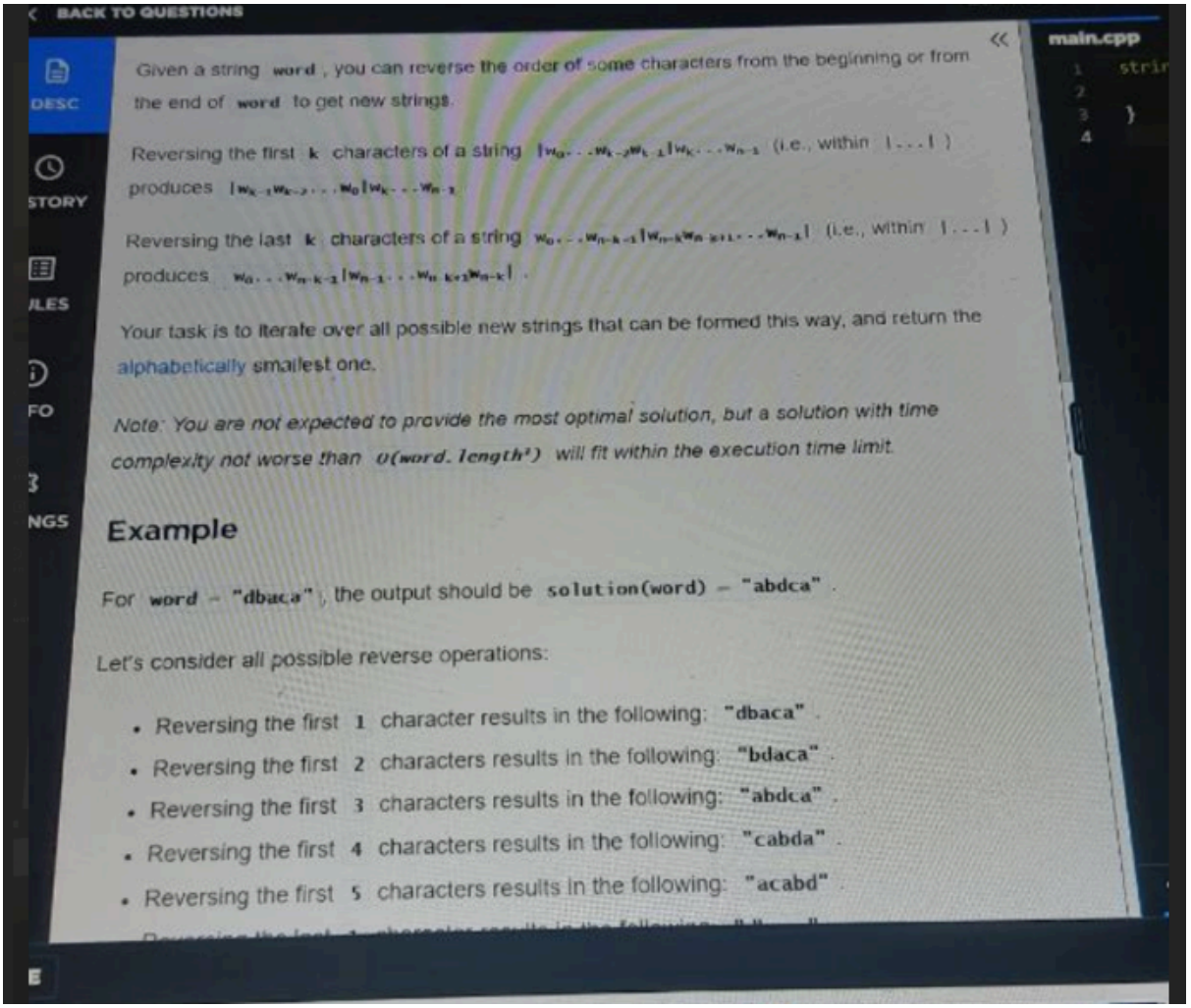
  `0 ≤ inputString.length ≤ 1000` .

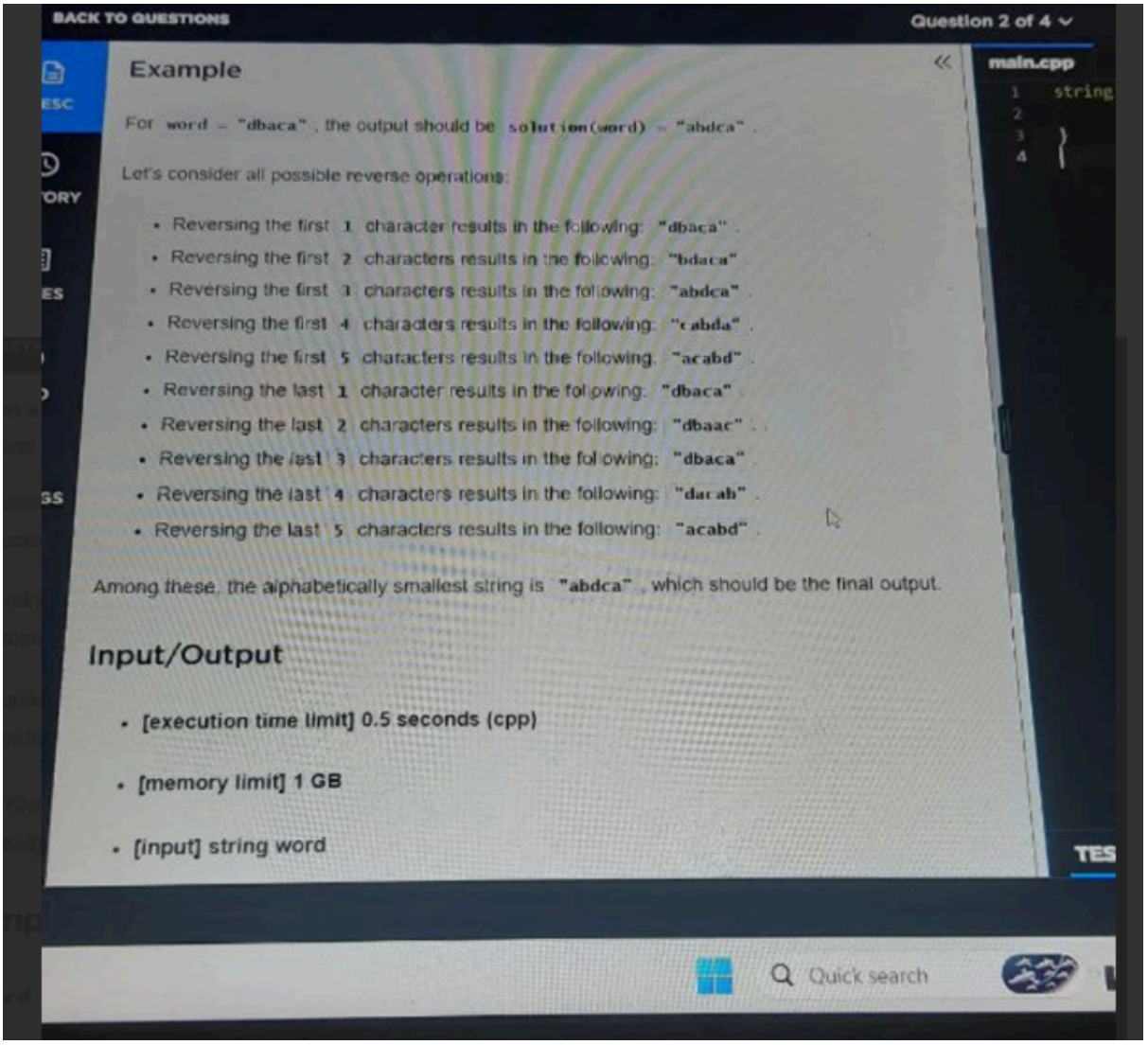
- **[output] string**

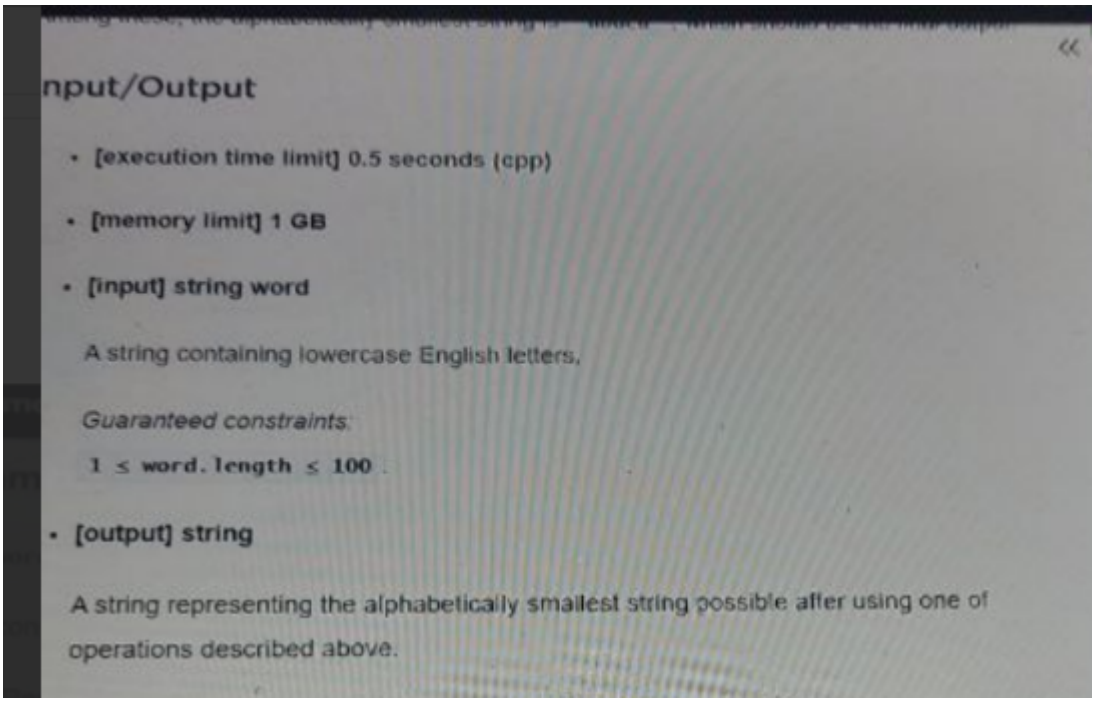
  The user's message after removing all vowels and reversing the order of the remaining characters.

+] Syntax Tips

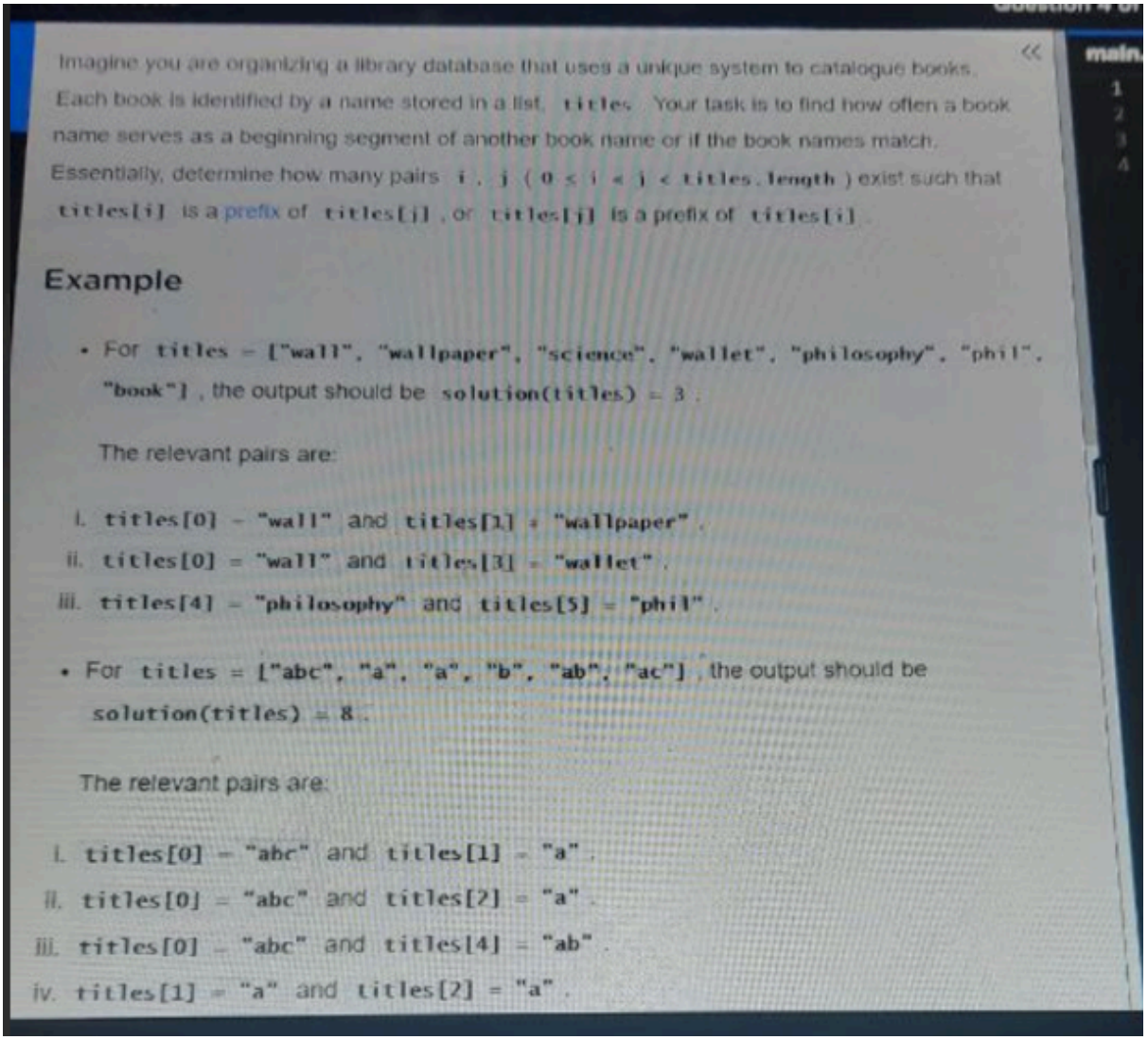
Q16.


BACK TO QUESTIONS
DESC
STORY
ALES
FO
NGS
E
Given a string word , you can reverse the order of some characters from the beginning or from the end of word to get new strings.
Reversing the first k characters of a string $|w_0 \dots w_{k-2} w_{k-1} | w_k \dots w_{n-1}$ (i.e., within $| \dots |$) produces $|w_{k-1} w_{k-2} \dots w_0 | w_k \dots w_{n-1}$.
Reversing the last k characters of a string $w_0 \dots w_{n-k-1} | w_{n-k} w_{n-k+1} \dots w_{n-1} |$ (i.e., within $| \dots |$) produces $w_0 \dots w_{n-k-1} | w_{n-1} \dots w_{n-k+1} w_{n-k} |$.
Your task is to iterate over all possible new strings that can be formed this way, and return the alphabetically smallest one.
Note: You are not expected to provide the most optimal solution, but a solution with time complexity not worse than O(word.length²) will fit within the execution time limit.
Example
For word = "dbaca", the output should be solution(word) = "abdca".
Let's consider all possible reverse operations:
Reversing the first 1 character results in the following: "dbaca".
Reversing the first 2 characters results in the following: "bdaca".
Reversing the first 3 characters results in the following: "abdca".
Reversing the first 4 characters results in the following: "cabda".
Reversing the first 5 characters results in the following: "acabd"
main.cpp
strin
}


BACK TO QUESTIONS
Question 2 of 4
Example
For word = "dbaca", the output should be solution(word) = "abdca".
Let's consider all possible reverse operations:
• Reversing the first 1 character results in the following: "dbaca".
• Reversing the first 2 characters results in the following: "bdaca".
• Reversing the first 3 characters results in the following: "abdca".
• Reversing the first 4 characters results in the following: "cabda".
• Reversing the first 5 characters results in the following: "acabd".
• Reversing the last 1 character results in the following: "dbaca".
• Reversing the last 2 characters results in the following: "dbaac".
• Reversing the last 3 characters results in the following: "dbaca".
• Reversing the last 4 characters results in the following: "dacab".
• Reversing the last 5 characters results in the following: "acabd".
Among these, the alphabetically smallest string is "abdca", which should be the final output.
Input/Output
• [execution time limit] 0.5 seconds (cpp)
• [memory limit] 1 GB
• [input] string word
main.cpp
string
Quick search


nput/Output
• [execution time limit] 0.5 seconds (cpp)
• [memory limit] 1 GB
• [input] string word
A string containing lowercase English letters.
Guaranteed constraints:
1 ≤ word.length ≤ 100.
• [output] string
A string representing the alphabetically smallest string possible after using one of operations described above.

Q17.



Imagine you are organizing a library database that uses a unique system to catalogue books. Each book is identified by a name stored in a list, `titles`. Your task is to find how often a book name serves as a beginning segment of another book name or if the book names match. Essentially, determine how many pairs `i`, `j` (`0 ≤ i < j < titles.length`) exist such that `titles[i]` is a prefix of `titles[j]`, or `titles[j]` is a prefix of `titles[i]`.

Example

- For `titles = ["wall", "wallpaper", "science", "wallet", "philosophy", "phil", "book"]`, the output should be `solution(titles) = 3`.

  The relevant pairs are:

  i. `titles[0] = "wall"` and `titles[1] = "wallpaper"`.
  ii. `titles[0] = "wall"` and `titles[3] = "wallet"`.
  iii. `titles[4] = "philosophy"` and `titles[5] = "phil"`.

- For `titles = ["abc", "a", "a", "b", "ab", "ac"]`, the output should be `solution(titles) = 8`.

  The relevant pairs are:

  i. `titles[0] = "abc"` and `titles[1] = "a"`.
  ii. `titles[0] = "abc"` and `titles[2] = "a"`.
  iii. `titles[0] = "abc"` and `titles[4] = "ab"`.
  iv. `titles[1] = "a"` and `titles[2] = "a"`.

Q18.

The relevant pairs are:

i. `titles[0] = "abc"` and `titles[1] = "a"`.
ii. `titles[0] = "abc"` and `titles[2] = "a"`.
iii. `titles[0] = "abc"` and `titles[4] = "ab"`.
iv. `titles[1] = "a"` and `titles[2] = "a"`.
v. `titles[1] = "a"` and `titles[4] = "ab"`.
vi. `titles[1] = "a"` and `titles[5] = "ac"`.
vii. `titles[2] = "a"` and `titles[4] = "ab"`.
viii. `titles[2] = "a"` and `titles[5] = "ac"`.

Input/Output

- **[execution time limit] 0.5 seconds (cpp)**
- **[memory limit] 1 GB**
- **[input] array.string titles**

  An array of book names containing lowercase English letters.

  Guaranteed constraints:

  $1 \le$ `titles.length` $\le 10^5$,

  $1 \le$ `titles[i].length` ≤ 10.

- [output] integer64

Input/Output

- **[execution time limit] 0.5 seconds (cpp)**
- **[memory limit] 1 GB**
- **[input] array.string titles**

  An array of book names containing lowercase English letters.

  Guaranteed constraints:

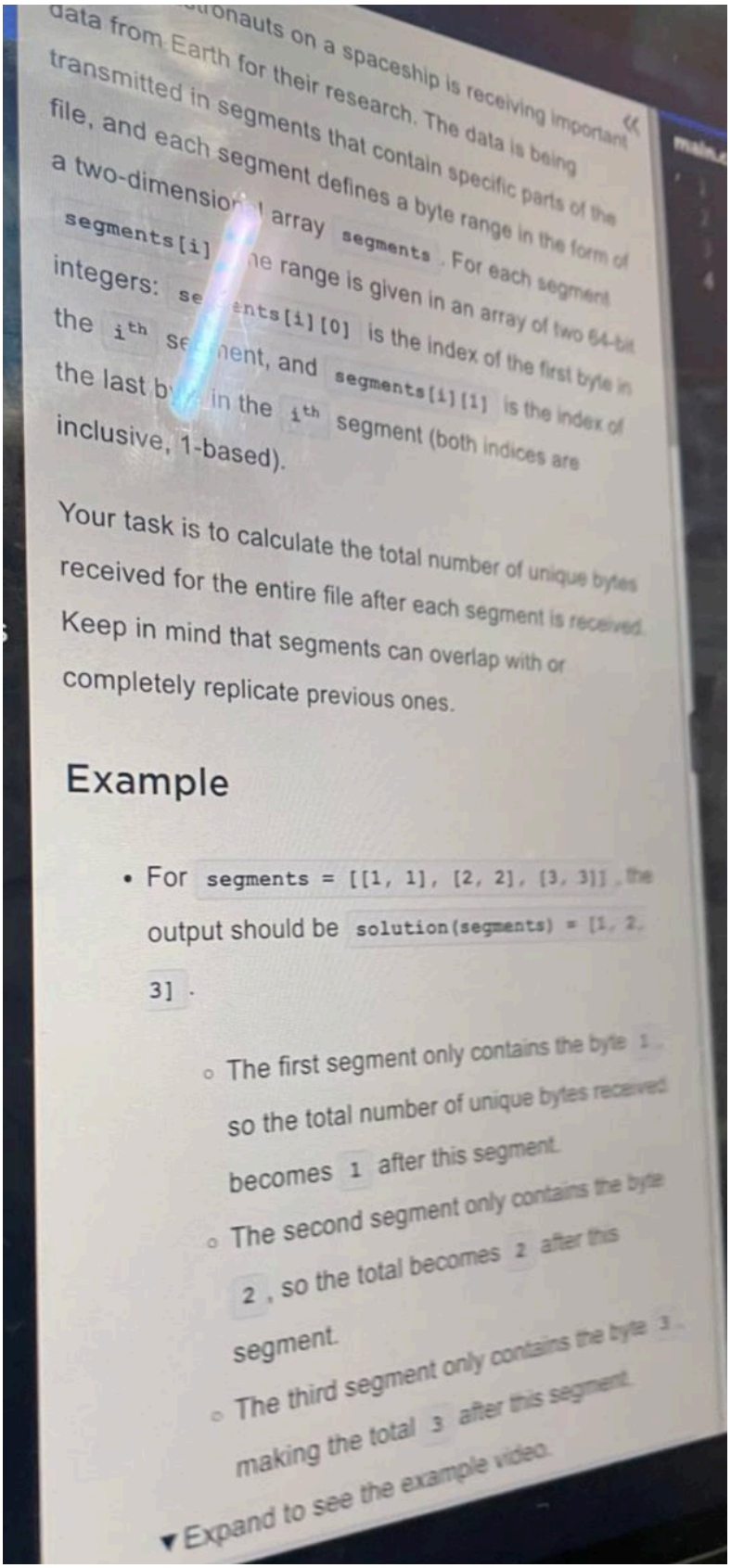
  $1 \le$ `titles.length` $\le 10^5$,

  $1 \le$ `titles[i].length` ≤ 10.

- **[output] integer64**

  The number of pairs where either the book names are identical or one book name is the start of the other.

Q19.



...tronauts on a spaceship is receiving important data from Earth for their research. The data is being transmitted in segments that contain specific parts of the file, and each segment defines a byte range in the form of a two-dimensional array `segments`. For each segment `segments[i]`, the range is given in an array of two 64-bit integers: `segments[i][0]` is the index of the first byte in the i^{th} segment, and `segments[i][1]` is the index of the last byte in the i^{th} segment (both indices are inclusive, 1-based).

Your task is to calculate the total number of unique bytes received for the entire file after each segment is received. Keep in mind that segments can overlap with or completely replicate previous ones.

Example

- For `segments = [[1, 1], [2, 2], [3, 3]]`, the output should be `solution(segments) = [1, 2, 3]`.
  - The first segment only contains the byte `1`, so the total number of unique bytes received becomes `1` after this segment.
  - The second segment only contains the byte `2`, so the total becomes `2` after this segment.
  - The third segment only contains the byte `3`, making the total `3` after this segment.

▼ Expand to see the example video.

TO QUESTIONS

DESC

HISTORY

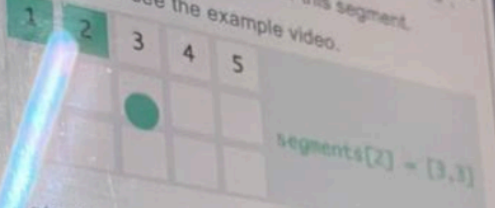
RULES

INFO

SETTINGS

- The third segment only contains the byte `3`, making the total `3` after this segment.

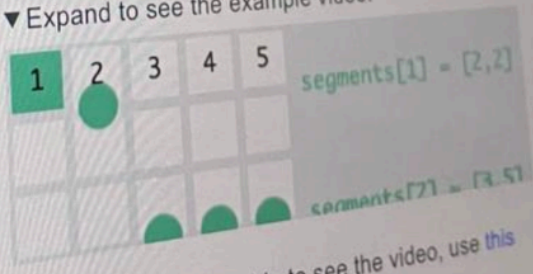
▼ Expand to see the example video.



Note: If you are not able to see the video, use this link to access it.

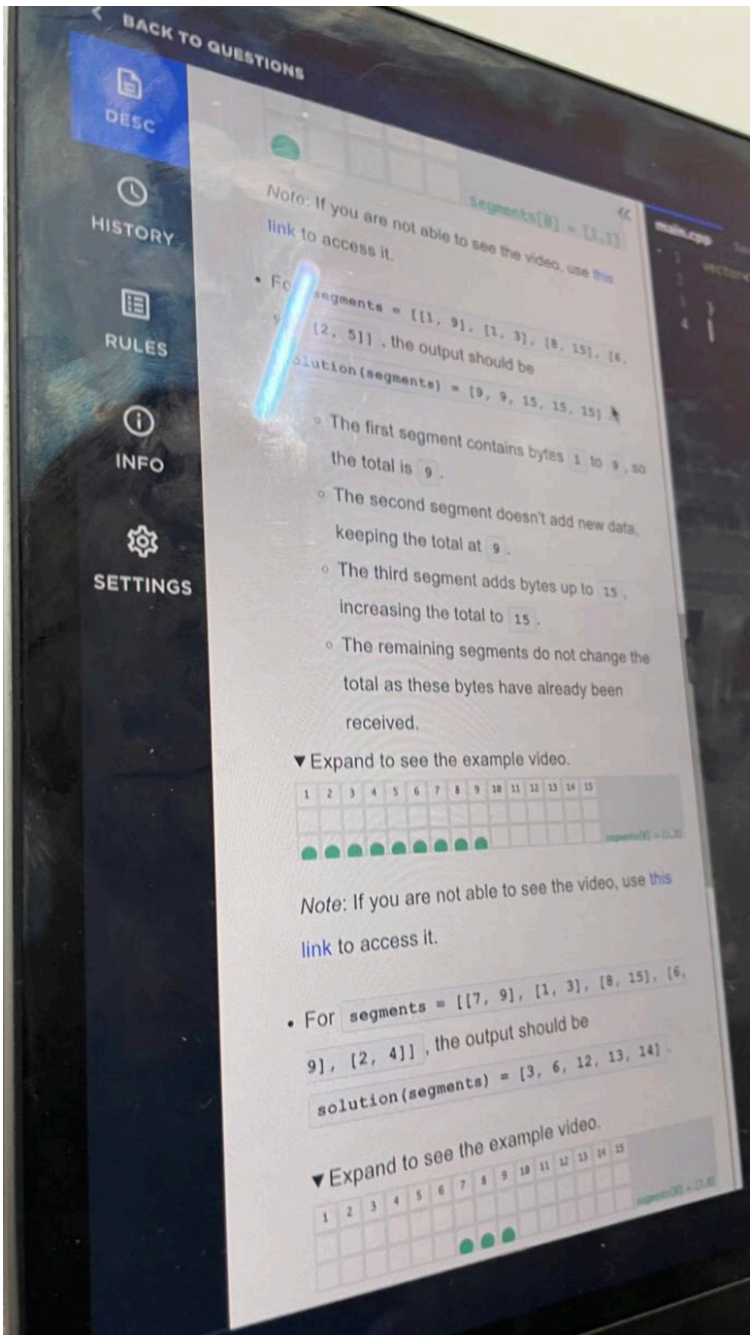
- For `segments = [[1, 1], [2, 2], [3, 5]]`, the output should be `solution(segments) = [1, 2, 5]`.
  - The first segment only contains the byte `1`, so the total number of unique bytes received becomes `1`.
  - The second segment adds byte `2`, making the total `2`.
  - The third segment includes bytes `3` to `5`, increasing the total to `5`.

▼ Expand to see the example video.



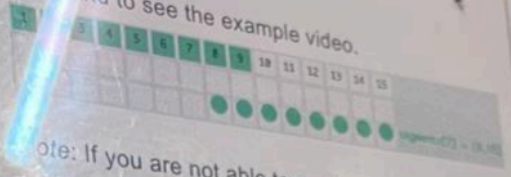
Note: If you are not able to see the video, use this link to access it.

- For `segments = [[1, 9], [1, 3], [8, 15], [6, 9], [2, 5]]`, the output should be


BACK TO QUESTIONS
DESC
HISTORY
RULES
INFO
SETTINGS
Note: If you are not able to see the video, use this link to access it.
For segments = [[1, 9], [1, 3], [8, 15], [6, [2, 5]] , the output should be
solution(segments) = [9, 9, 15, 15, 15]
The first segment contains bytes 1 to 9 , so the total is 9 .
The second segment doesn't add new data, keeping the total at 9 .
The third segment adds bytes up to 15 , increasing the total to 15 .
The remaining segments do not change the total as these bytes have already been received.
▼ Expand to see the example video.
Note: If you are not able to see the video, use this link to access it.
For segments = [[7, 9], [1, 3], [8, 15], [6, 9], [2, 4]] , the output should be solution(segments) = [3, 6, 12, 13, 14] .
▼ Expand to see the example video.

- The third segment adds bytes up to `15`, increasing the total to `15`.
- The remaining segments do not change the total as these bytes have already been received.

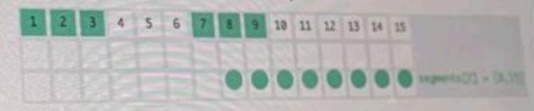
▼Expand to see the example video.



Note: If you are not able to see the video, use this link to access it.

- For `segments = [[7, 9], [1, 3], [8, 15], [6, 9], [2, 4]]`, the output should be `solution(segments) = [3, 6, 12, 13, 14]`.

▼Expand to see the example video.

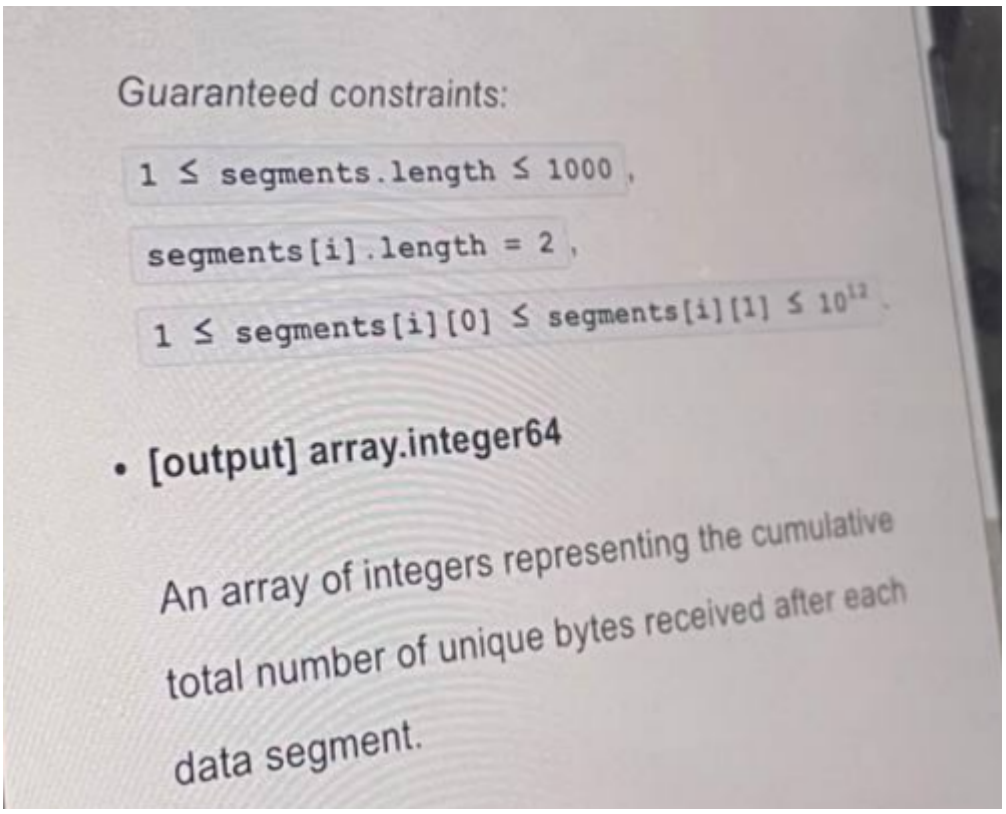


Note: If you are not able to see the video, use this link to access it.

Input/Output

- **[execution time limit] 0.5 seconds (cpp)**
- **[memory limit] 1 GB**
- **[input] array.array.integer64 segments**

An array of arrays, each representing the indices of the first and last bytes contained in each data



Guaranteed constraints:

`1 ≤ segments.length ≤ 1000`,

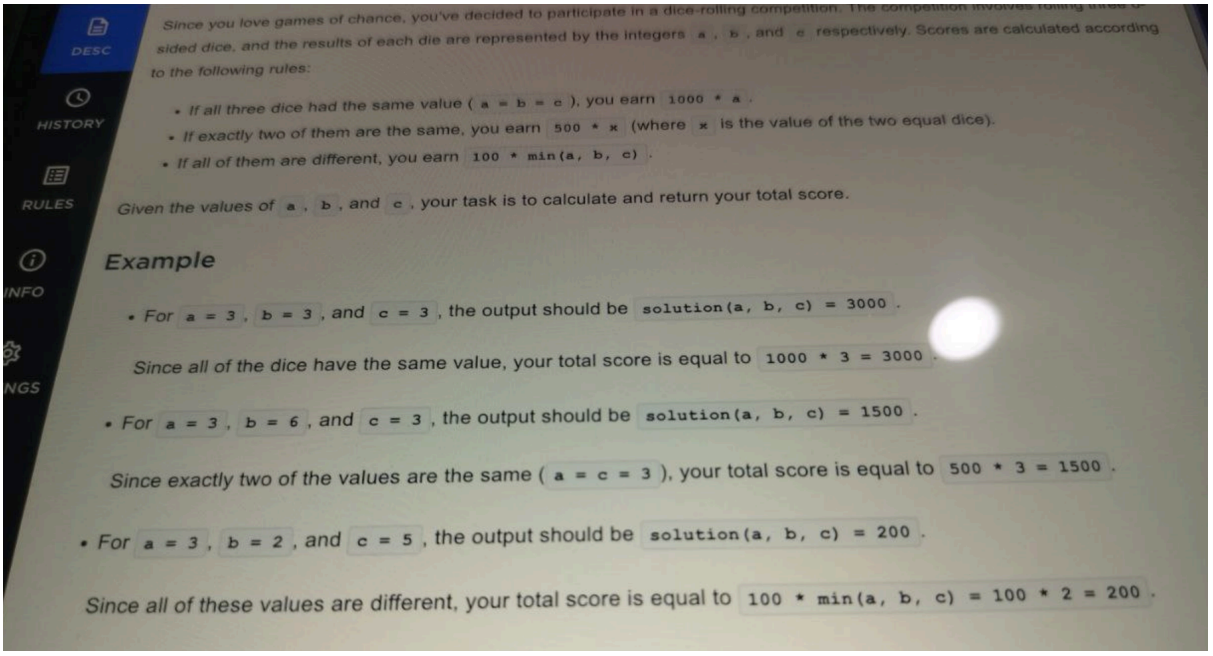
`segments[i].length = 2`,

`1 ≤ segments[i][0] ≤ segments[i][1] ≤ 10^12`.

- **[output] array.integer64**

An array of integers representing the cumulative total number of unique bytes received after each data segment.

Q20.



DESC

HISTORY

RULES

INFO

NGS

Since you love games of chance, you've decided to participate in a dice-rolling competition. The competition involves rolling three six-sided dice, and the results of each die are represented by the integers `a`, `b`, and `c` respectively. Scores are calculated according to the following rules:

- If all three dice had the same value (`a = b = c`), you earn `1000 * a`.
- If exactly two of them are the same, you earn `500 * x` (where `x` is the value of the two equal dice).
- If all of them are different, you earn `100 * min(a, b, c)`.

Given the values of `a`, `b`, and `c`, your task is to calculate and return your total score.

Example

- For `a = 3`, `b = 3`, and `c = 3`, the output should be `solution(a, b, c) = 3000`.

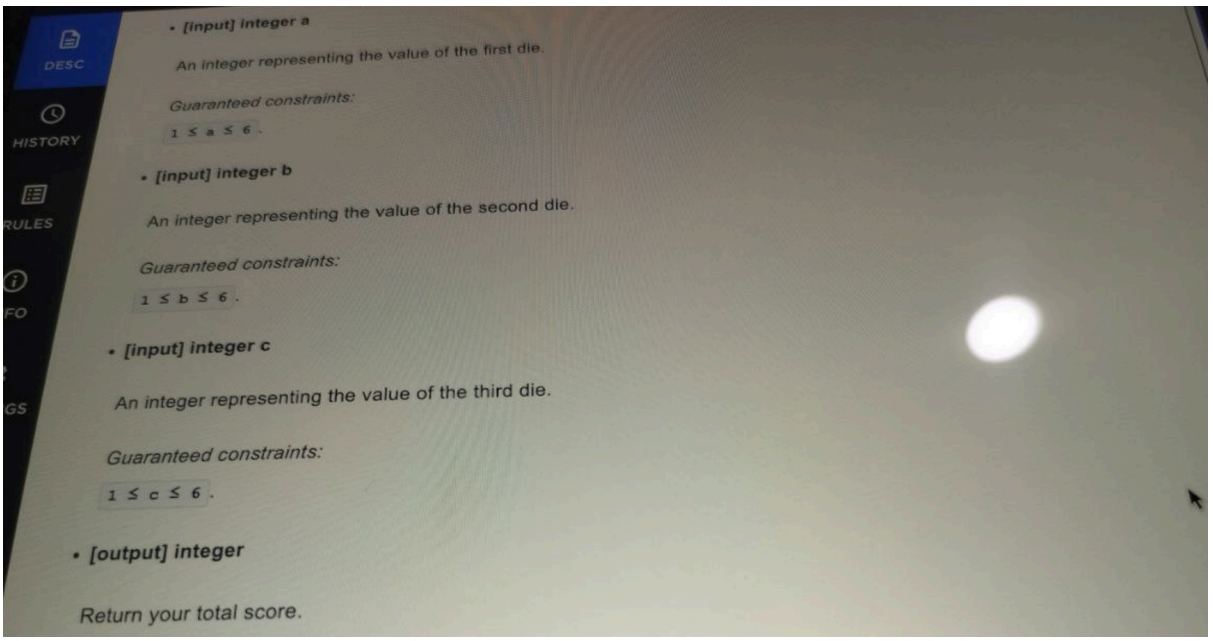
  Since all of the dice have the same value, your total score is equal to `1000 * 3 = 3000`.

- For `a = 3`, `b = 6`, and `c = 3`, the output should be `solution(a, b, c) = 1500`.

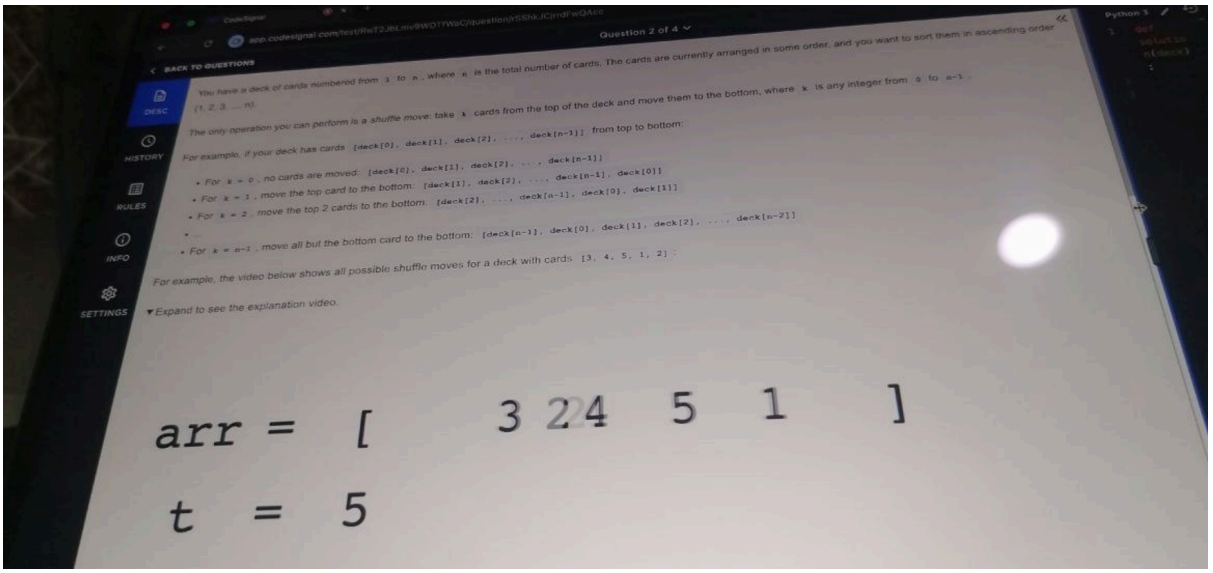
  Since exactly two of the values are the same (`a = c = 3`), your total score is equal to `500 * 3 = 1500`.

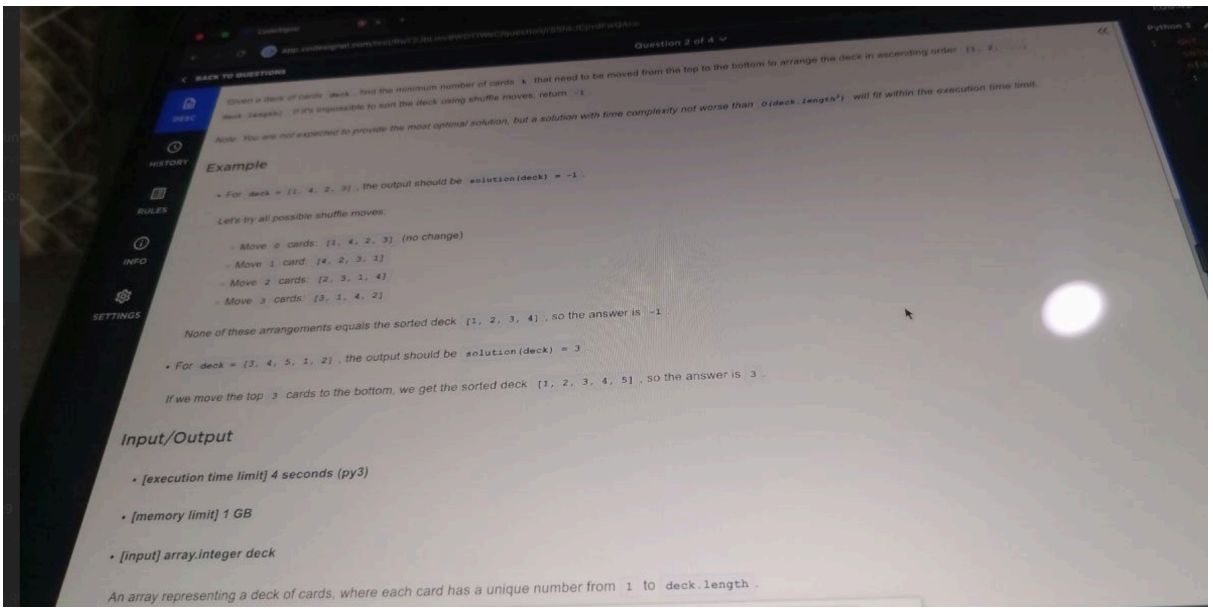
- For `a = 3`, `b = 2`, and `c = 5`, the output should be `solution(a, b, c) = 200`.

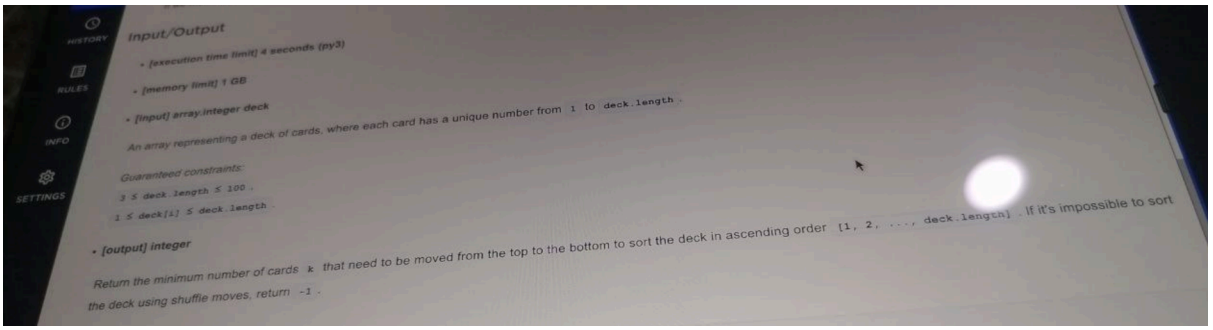
  Since all of these values are different, your total score is equal to `100 * min(a, b, c) = 100 * 2 = 200`.


DESC
HISTORY
RULES
INFO
SETTINGS
• [input] integer a
An integer representing the value of the first die.
Guaranteed constraints:
1 ≤ a ≤ 6 .
• [input] integer b
An integer representing the value of the second die.
Guaranteed constraints:
1 ≤ b ≤ 6 .
• [input] integer c
An integer representing the value of the third die.
Guaranteed constraints:
1 ≤ c ≤ 6 .
• [output] integer
Return your total score.

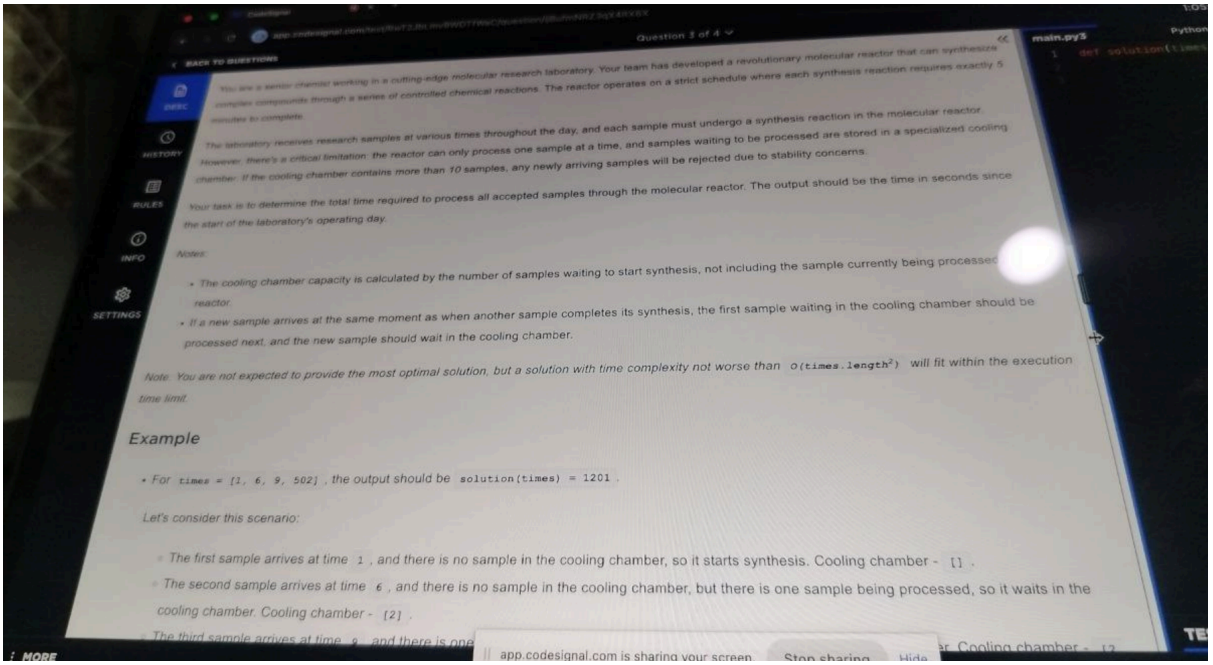
Q21.

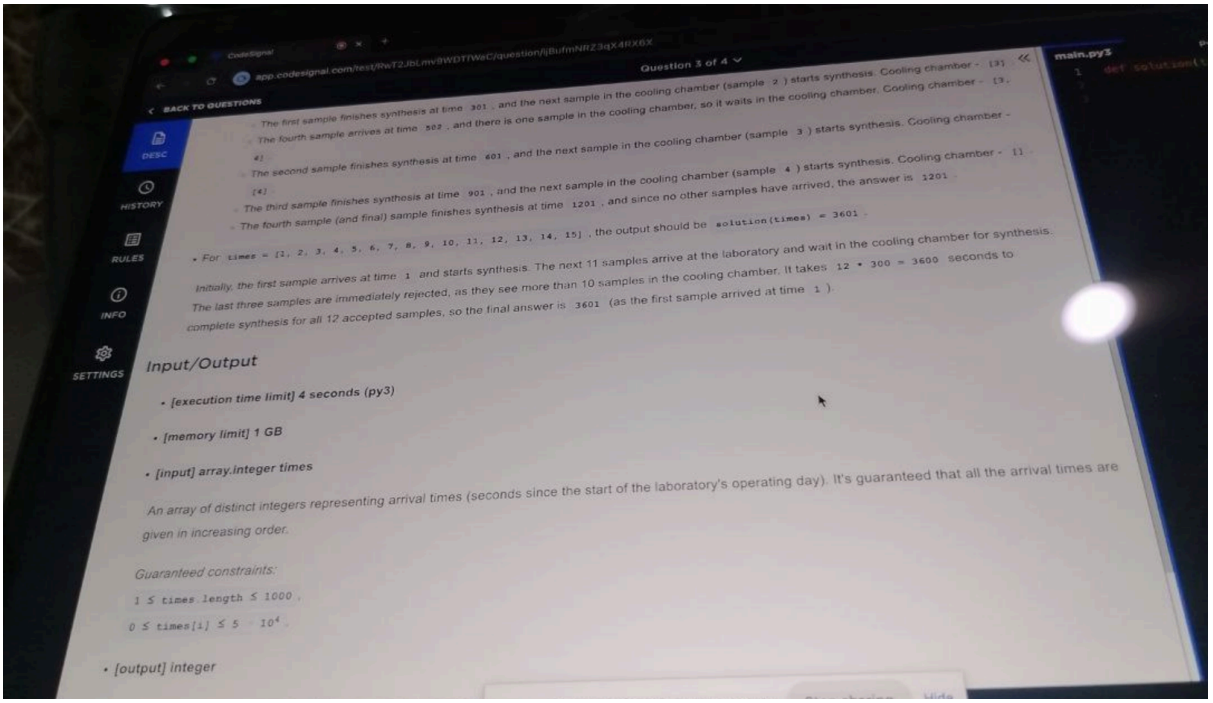

BACK TO QUESTIONS
Question 2 of 4
DESC
HISTORY
RULES
INFO
SETTINGS
arr = [3 2 4 5 1]
t = 5


Question 2 of 4
BACK TO QUESTIONS
DESC
HISTORY
RULES
INFO
SETTINGS
Example
For deck = [1, 4, 2, 3], the output should be solution(deck) = -1.
Let's try all possible shuffle moves:
Move 0 cards: [1, 4, 2, 3] (no change)
Move 1 card: [4, 2, 3, 1]
Move 2 cards: [2, 3, 1, 4]
Move 3 cards: [3, 1, 4, 2]
None of these arrangements equals the sorted deck [1, 2, 3, 4], so the answer is -1.
For deck = [3, 4, 5, 1, 2], the output should be solution(deck) = 3.
If we move the top 3 cards to the bottom, we get the sorted deck [1, 2, 3, 4, 5], so the answer is 3.
Input/Output
[execution time limit] 4 seconds (py3)
[memory limit] 1 GB
[input] array.integer deck
An array representing a deck of cards, where each card has a unique number from 1 to deck.length.

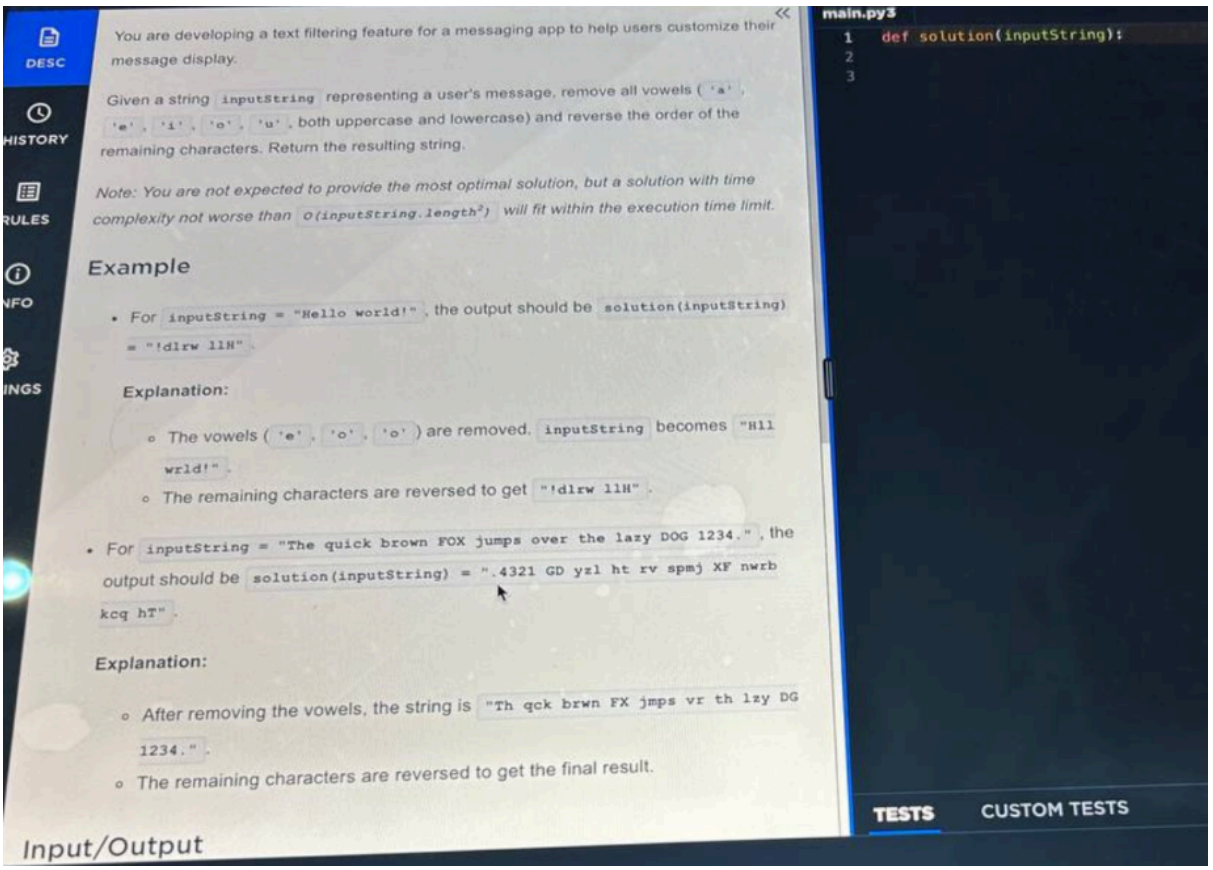

HISTORY
RULES
INFO
SETTINGS
Input/Output
[execution time limit] 4 seconds (py3)
[memory limit] 1 GB
[input] array.integer deck
An array representing a deck of cards, where each card has a unique number from 1 to deck.length.
Guaranteed constraints:
3 ≤ deck.length ≤ 100,
1 ≤ deck[i] ≤ deck.length.
[output] integer
Return the minimum number of cards k that need to be moved from the top to the bottom to sort the deck in ascending order [1, 2, ..., deck.length]. If it's impossible to sort the deck using shuffle moves, return -1.

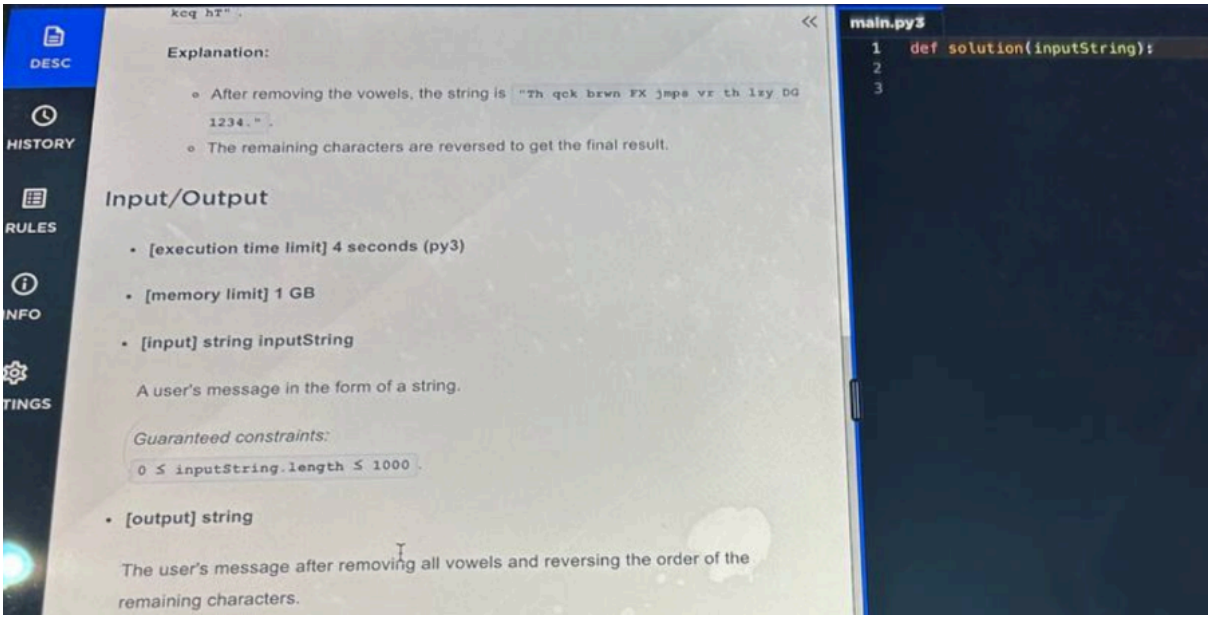
Q22.


Question 3 of 4
BACK TO QUESTIONS
DESC
HISTORY
RULES
INFO
SETTINGS
main.py3
Python
You are a senior chemist working in a cutting-edge molecular research laboratory. Your team has developed a revolutionary molecular reactor that can synthesize complex compounds through a series of controlled chemical reactions. The reactor operates on a strict schedule where each synthesis reaction requires exactly 5 minutes to complete.
The laboratory receives research samples at various times throughout the day, and each sample must undergo a synthesis reaction in the molecular reactor. However, there's a critical limitation: the reactor can only process one sample at a time, and samples waiting to be processed are stored in a specialized cooling chamber. If the cooling chamber contains more than 10 samples, any newly arriving samples will be rejected due to stability concerns.
Your task is to determine the total time required to process all accepted samples through the molecular reactor. The output should be the time in seconds since the start of the laboratory's operating day.
Notes:
The cooling chamber capacity is calculated by the number of samples waiting to start synthesis, not including the sample currently being processed in the reactor.
If a new sample arrives at the same moment as when another sample completes its synthesis, the first sample waiting in the cooling chamber should be processed next, and the new sample should wait in the cooling chamber.
Note: You are not expected to provide the most optimal solution, but a solution with time complexity not worse than O(times.length²) will fit within the execution time limit.
Example
For times = [1, 6, 9, 502], the output should be solution(times) = 1201.
Let's consider this scenario:
The first sample arrives at time 1, and there is no sample in the cooling chamber, so it starts synthesis. Cooling chamber - [].
The second sample arrives at time 6, and there is no sample in the cooling chamber, but there is one sample being processed, so it waits in the cooling chamber. Cooling chamber - [2].
The third sample arrives at time 9, and there is one
app.codesignal.com is sharing your screen.
Stop sharing
Hide
MORE

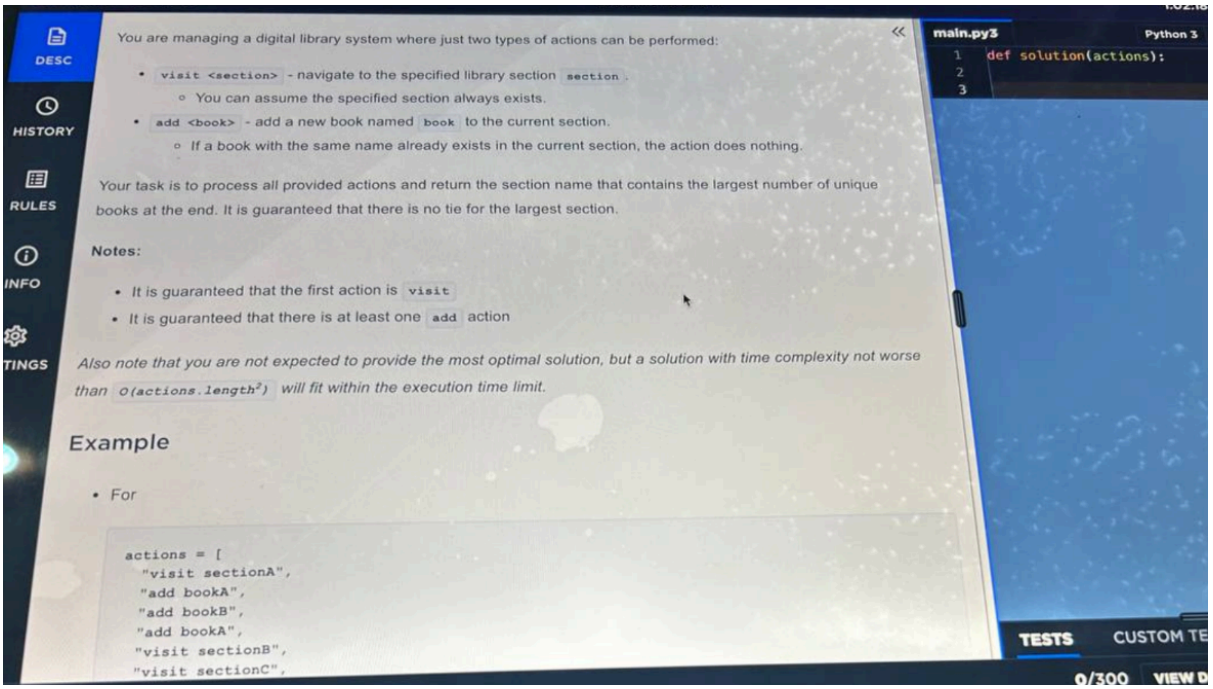

BACK TO QUESTIONS
Question 3 of 4
DESC
HISTORY
RULES
INFO
SETTINGS
main.py3
Input/Output
[execution time limit] 4 seconds (py3)
[memory limit] 1 GB
[input] array.integer times
An array of distinct integers representing arrival times (seconds since the start of the laboratory's operating day). It's guaranteed that all the arrival times are given in increasing order.
Guaranteed constraints:
1 ≤ times.length ≤ 1000,
0 ≤ times[i] ≤ 5 · 10^4.
[output] integer

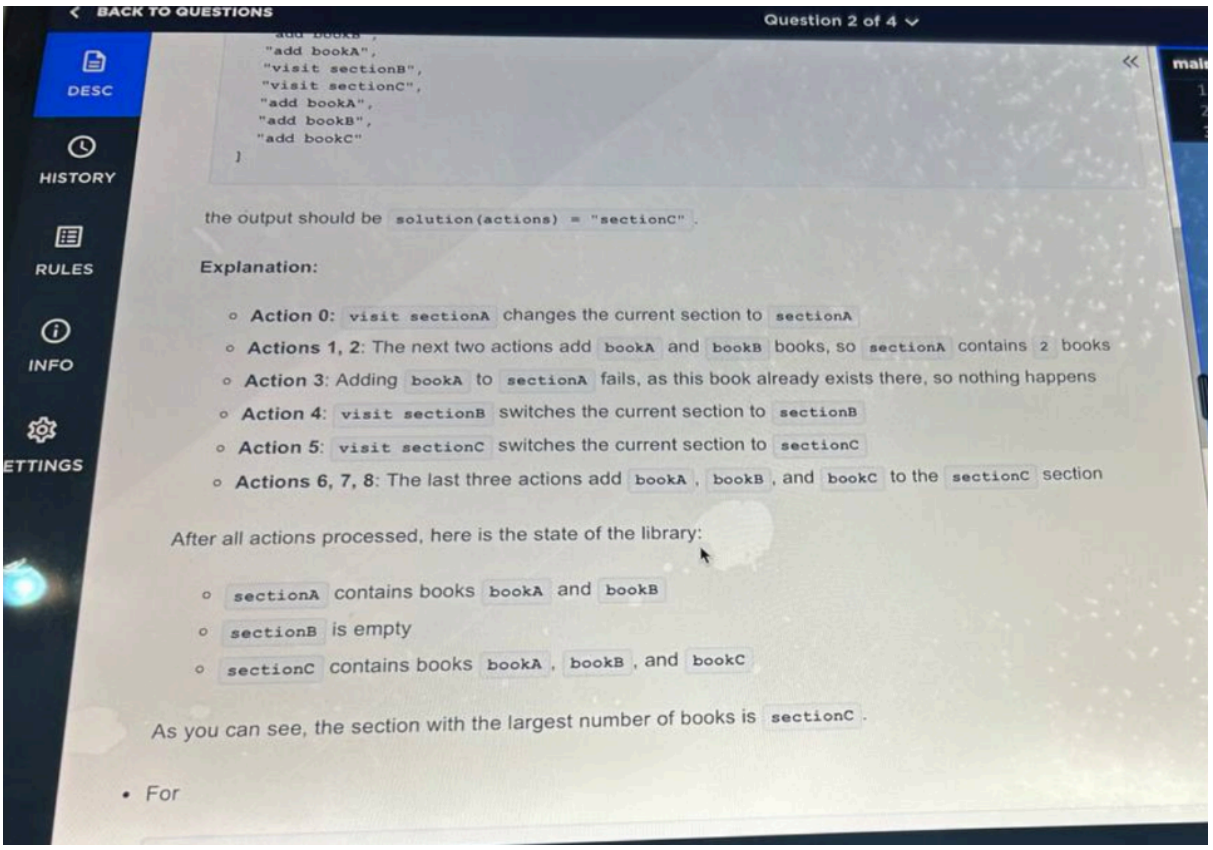
Q23.

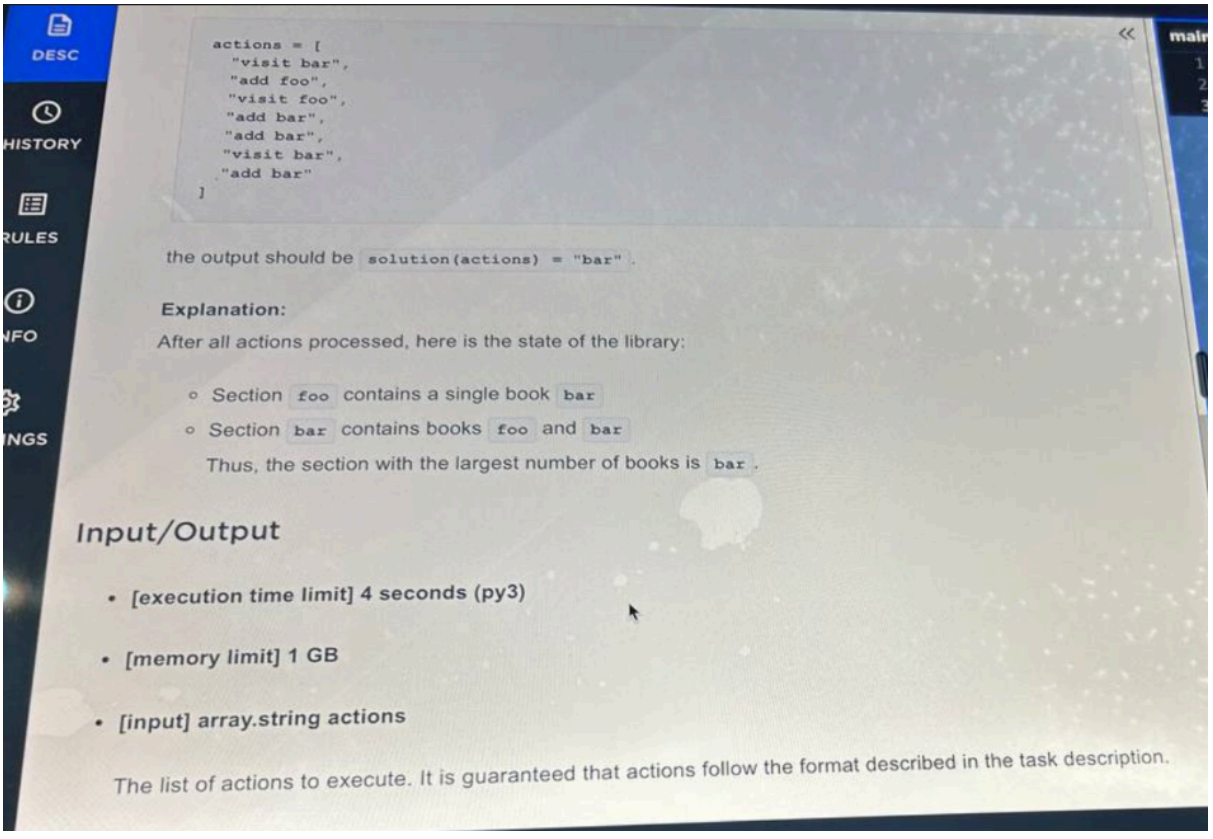

DESC
HISTORY
RULES
INFO
INGS
main.py3
1 def solution(inputString):
2
3
You are developing a text filtering feature for a messaging app to help users customize their message display.
Given a string inputString representing a user's message, remove all vowels ('a', 'e', 'i', 'o', 'u', both uppercase and lowercase) and reverse the order of the remaining characters. Return the resulting string.
Note: You are not expected to provide the most optimal solution, but a solution with time complexity not worse than O(inputString.length²) will fit within the execution time limit.
Example
For inputString = "Hello world!", the output should be solution(inputString) = "!dlrw llH".
Explanation:
The vowels ('e', 'o', 'o') are removed. inputString becomes "Hll wrld!".
The remaining characters are reversed to get "!dlrw llH".
For inputString = "The quick brown FOX jumps over the lazy DOG 1234.", the output should be solution(inputString) = ".4321 GD yzl ht rv spmj XF nwrb kcq hT".
Explanation:
After removing the vowels, the string is "Th qck brwn FX jmps vr th lzy DG 1234.".
The remaining characters are reversed to get the final result.
Input/Output
TESTS
CUSTOM TESTS

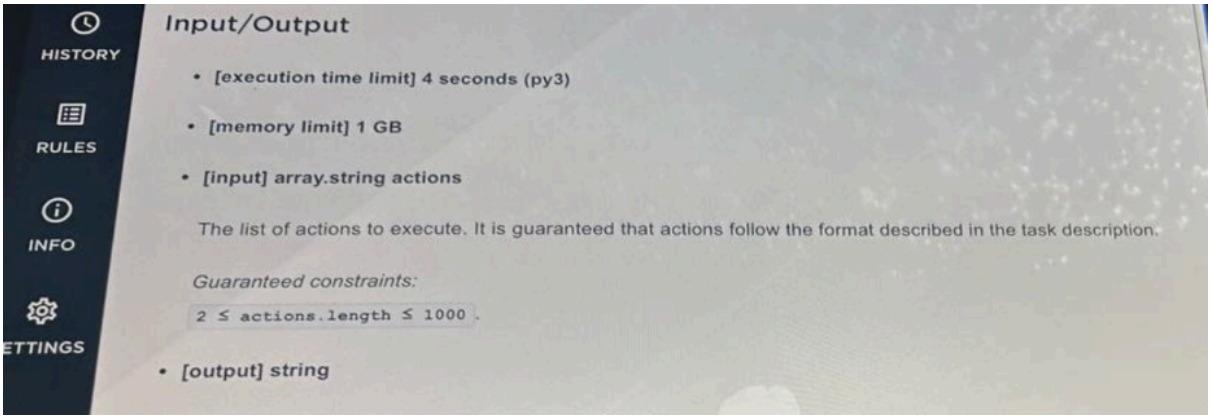

DESC
HISTORY
RULES
INFO
TINGS
kcq hT" .
Explanation:
After removing the vowels, the string is "Th qck brwn FX jmps vr th lzy DG 1234." .
The remaining characters are reversed to get the final result.
Input/Output
[execution time limit] 4 seconds (py3)
[memory limit] 1 GB
[input] string inputString
A user's message in the form of a string.
Guaranteed constraints:
0 ≤ inputString.length ≤ 1000 .
[output] string
The user's message after removing all vowels and reversing the order of the remaining characters.
main.py3
1 def solution(inputString):
2
3

Q24.

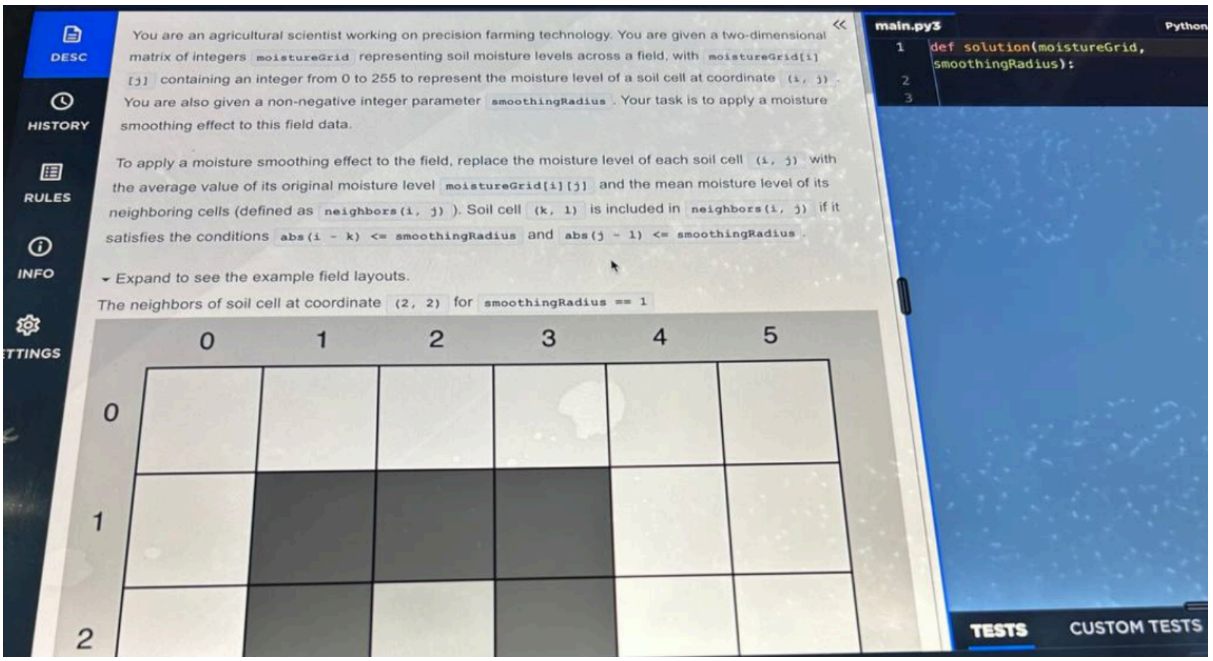

DESC
HISTORY
RULES
INFO
TINGS
You are managing a digital library system where just two types of actions can be performed:
visit <section> - navigate to the specified library section section .
You can assume the specified section always exists.
add <book> - add a new book named book to the current section.
If a book with the same name already exists in the current section, the action does nothing.
Your task is to process all provided actions and return the section name that contains the largest number of unique books at the end. It is guaranteed that there is no tie for the largest section.
Notes:
It is guaranteed that the first action is visit
It is guaranteed that there is at least one add action
Also note that you are not expected to provide the most optimal solution, but a solution with time complexity not worse than O(actions.length²) will fit within the execution time limit.
Example
For
actions = [
"visit sectionA",
"add bookA",
"add bookB",
"add bookA",
"visit sectionB",
"visit sectionC",
main.py3
Python 3
1 def solution(actions):
2
3
TESTS
CUSTOM TE
0/300
VIEW D

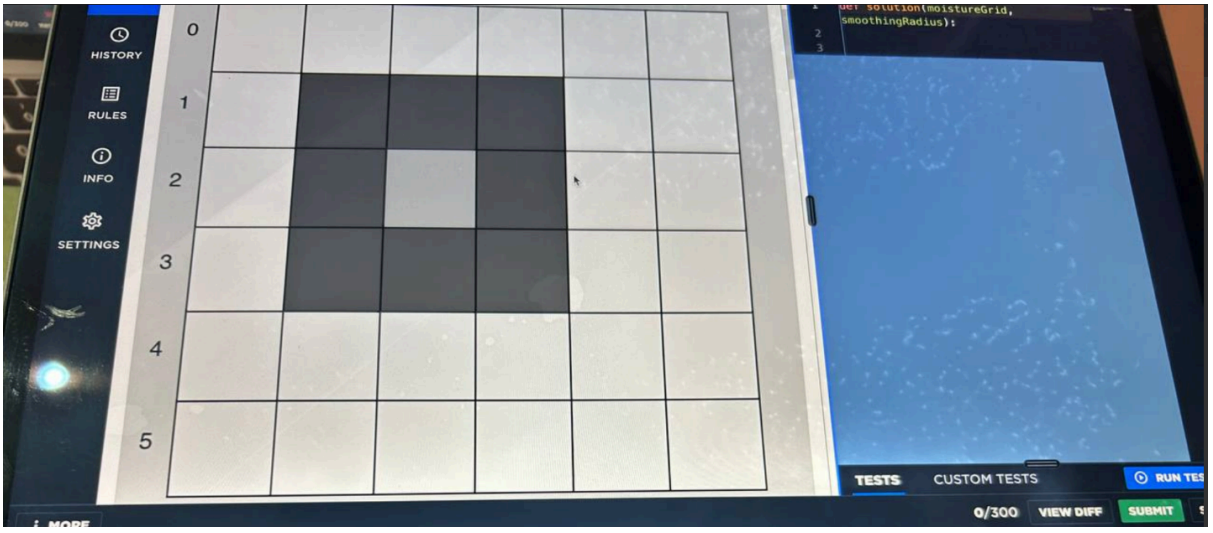

BACK TO QUESTIONS
Question 2 of 4
DESC
HISTORY
RULES
INFO
ETTINGS
"add bookA",
"visit sectionB",
"visit sectionC",
"add bookA",
"add bookB",
"add bookC"
]
the output should be solution(actions) = "sectionC".
Explanation:
Action 0: visit sectionA changes the current section to sectionA
Actions 1, 2: The next two actions add bookA and bookB books, so sectionA contains 2 books
Action 3: Adding bookA to sectionA fails, as this book already exists there, so nothing happens
Action 4: visit sectionB switches the current section to sectionB
Action 5: visit sectionC switches the current section to sectionC
Actions 6, 7, 8: The last three actions add bookA, bookB, and bookC to the sectionC section
After all actions processed, here is the state of the library:
sectionA contains books bookA and bookB
sectionB is empty
sectionC contains books bookA, bookB, and bookC
As you can see, the section with the largest number of books is sectionC.
For

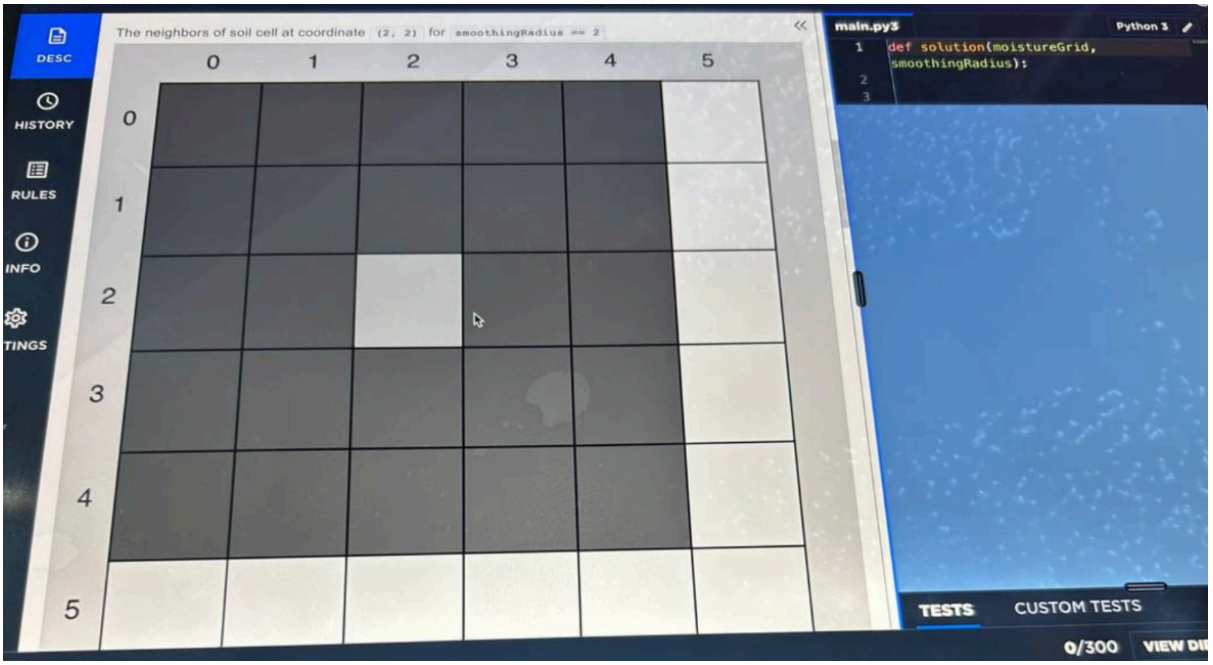

DESC
HISTORY
RULES
INFO
NGS
actions = [
"visit bar",
"add foo",
"visit foo",
"add bar",
"add bar",
"visit bar",
"add bar"
]
the output should be solution(actions) = "bar".
Explanation:
After all actions processed, here is the state of the library:
Section foo contains a single book bar
Section bar contains books foo and bar
Thus, the section with the largest number of books is bar.
Input/Output
[execution time limit] 4 seconds (py3)
[memory limit] 1 GB
[input] array.string actions
The list of actions to execute. It is guaranteed that actions follow the format described in the task description.

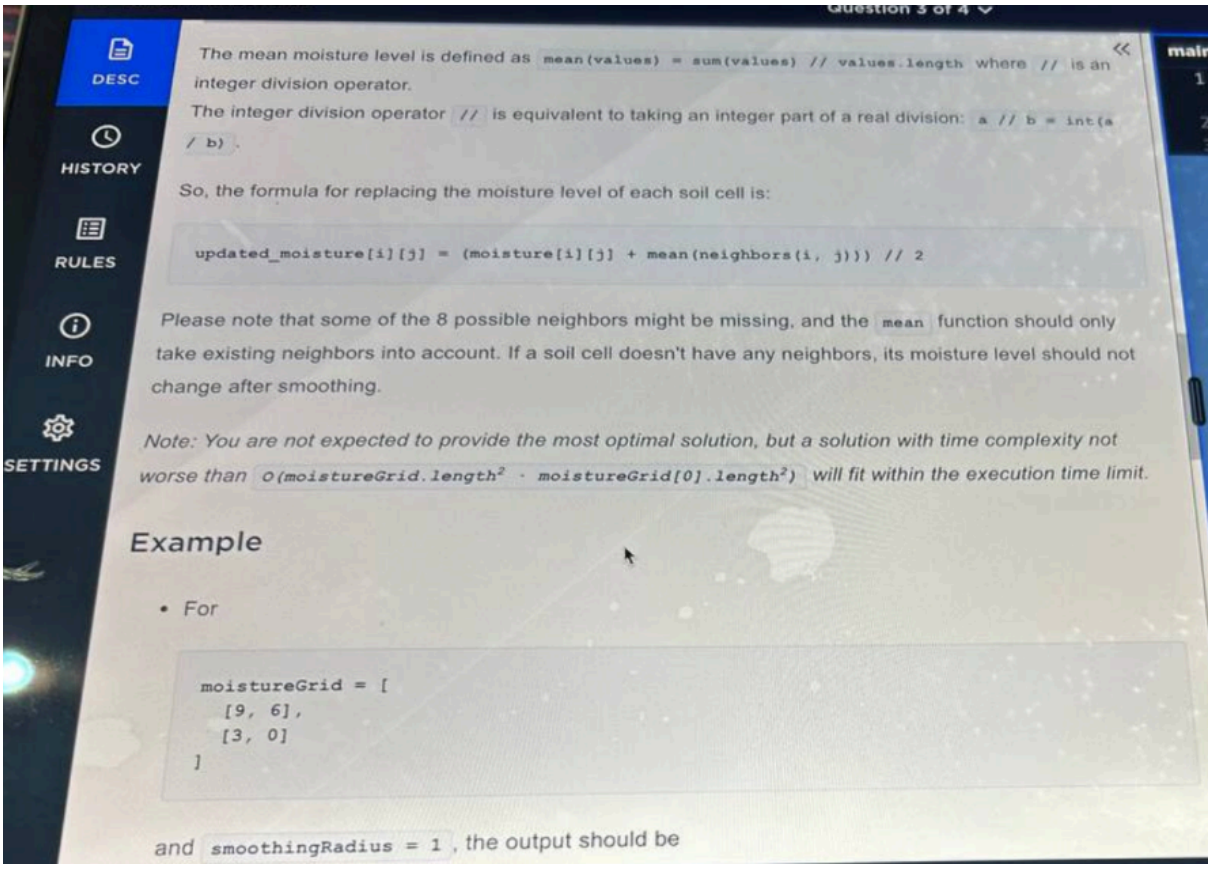

HISTORY
RULES
INFO
ETTINGS
Input/Output
• [execution time limit] 4 seconds (py3)
• [memory limit] 1 GB
• [input] array.string actions
The list of actions to execute. It is guaranteed that actions follow the format described in the task description.
Guaranteed constraints:
2 ≤ actions.length ≤ 1000 .
• [output] string

Q25.


DESC
HISTORY
RULES
INFO
TTINGS
You are an agricultural scientist working on precision farming technology. You are given a two-dimensional matrix of integers moistureGrid representing soil moisture levels across a field, with moistureGrid[i][j] containing an integer from 0 to 255 to represent the moisture level of a soil cell at coordinate (i, j). You are also given a non-negative integer parameter smoothingRadius. Your task is to apply a moisture smoothing effect to this field data.
To apply a moisture smoothing effect to the field, replace the moisture level of each soil cell (i, j) with the average value of its original moisture level moistureGrid[i][j] and the mean moisture level of its neighboring cells (defined as neighbors(i, j)). Soil cell (k, l) is included in neighbors(i, j) if it satisfies the conditions abs(i - k) <= smoothingRadius and abs(j - l) <= smoothingRadius.
Expand to see the example field layouts.
The neighbors of soil cell at coordinate (2, 2) for smoothingRadius == 1
0 1 2 3 4 5
0
1
2
main.py3
Python
1 def solution(moistureGrid,
smoothingRadius):
2
3
TESTS
CUSTOM TESTS


HISTORY
RULES
INFO
SETTINGS
0 1 2 3 4 5
def solution(moistureGrid,
smoothingRadius):
TESTS
CUSTOM TESTS
RUN TES
0/300
VIEW DIFF
SUBMIT


DESC
HISTORY
RULES
INFO
TINGS
The neighbors of soil cell at coordinate (2, 2) for smoothingRadius == 2
0 1 2 3 4 5
0
1
2
3
4
5
main.py3
Python 3
1 def solution(moistureGrid,
smoothingRadius):
2
3
TESTS
CUSTOM TESTS
0/300
VIEW DI


Question 3 of 4
DESC
HISTORY
RULES
INFO
SETTINGS
The mean moisture level is defined as mean(values) = sum(values) // values.length where // is an integer division operator.
The integer division operator // is equivalent to taking an integer part of a real division: a // b = int(a / b).
So, the formula for replacing the moisture level of each soil cell is:
updated_moisture[i][j] = (moisture[i][j] + mean(neighbors(i, j))) // 2
Please note that some of the 8 possible neighbors might be missing, and the mean function should only take existing neighbors into account. If a soil cell doesn't have any neighbors, its moisture level should not change after smoothing.
Note: You are not expected to provide the most optimal solution, but a solution with time complexity not worse than O(moistureGrid.length² · moistureGrid[0].length²) will fit within the execution time limit.
Example
For
moistureGrid = [
[9, 6],
[3, 0]
]
and smoothingRadius = 1, the output should be

DESC
HISTORY
RULES
INFO
TINGS

- For

```
moistureGrid = [
  [9, 6],
  [3, 0]
]
```

and `smoothingRadius = 1`, the output should be

```
solution(moistureGrid, smoothingRadius) = [
  [6, 5],
  [4, 3]
]
```

Explanation:

- All soil cells in the original field have 3 neighbors.
- `moistureGrid[0][0] = 9`, and its neighbors are `[6, 3, 0]` with a mean of `(6 + 3 + 0) // 3 = 3`. So, it should be updated to `mean(9, 3) = (9 + 3) // 2 = 6`.
- `moistureGrid[0][1] = 6`, and its neighbors are `[9, 3, 0]` with a mean of `12 // 3 = 4`. So, it should be updated to `mean(6, 4) = 10 // 2 = 5`.
- `moistureGrid[1][0] = 3`, and its neighbors are `[9, 6, 0]` with a mean of `5`. So, it should be updated to `mean(3, 5) = 4`.
- `moistureGrid[1][1] = 0`, and its neighbors are `[9, 6, 3]` with a mean of `6`. So, it should be updated to `mean(0, 6) = 3`.

- For

main.py3

1 2 3

DESC
HISTORY
RULES
INFO
INGS

- For

```
moistureGrid = [
  [0, 0, 0],
  [0, 255, 0],
  [0, 0, 0]
]
```

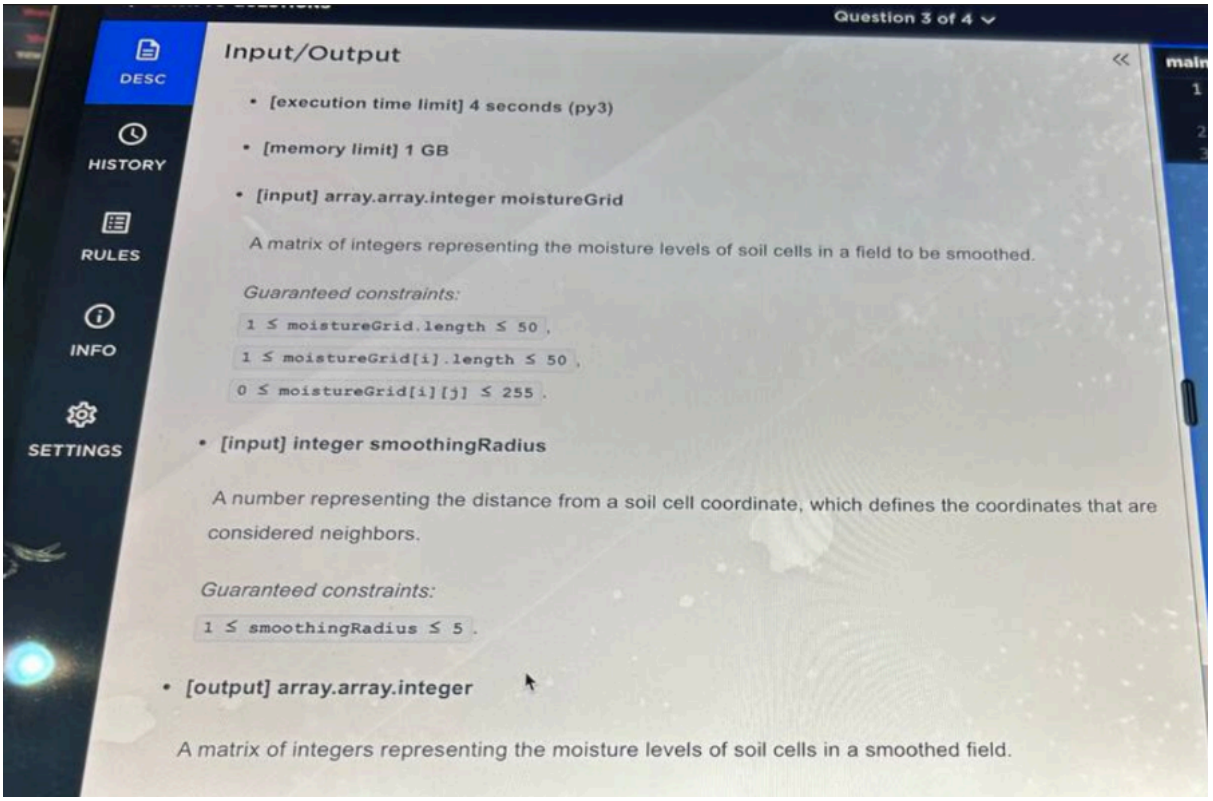
and `smoothingRadius = 2`, the output should be

```
solution(moistureGrid, smoothingRadius) = [
  [15, 15, 15],
  [15, 127, 15],
  [15, 15, 15]
]
```

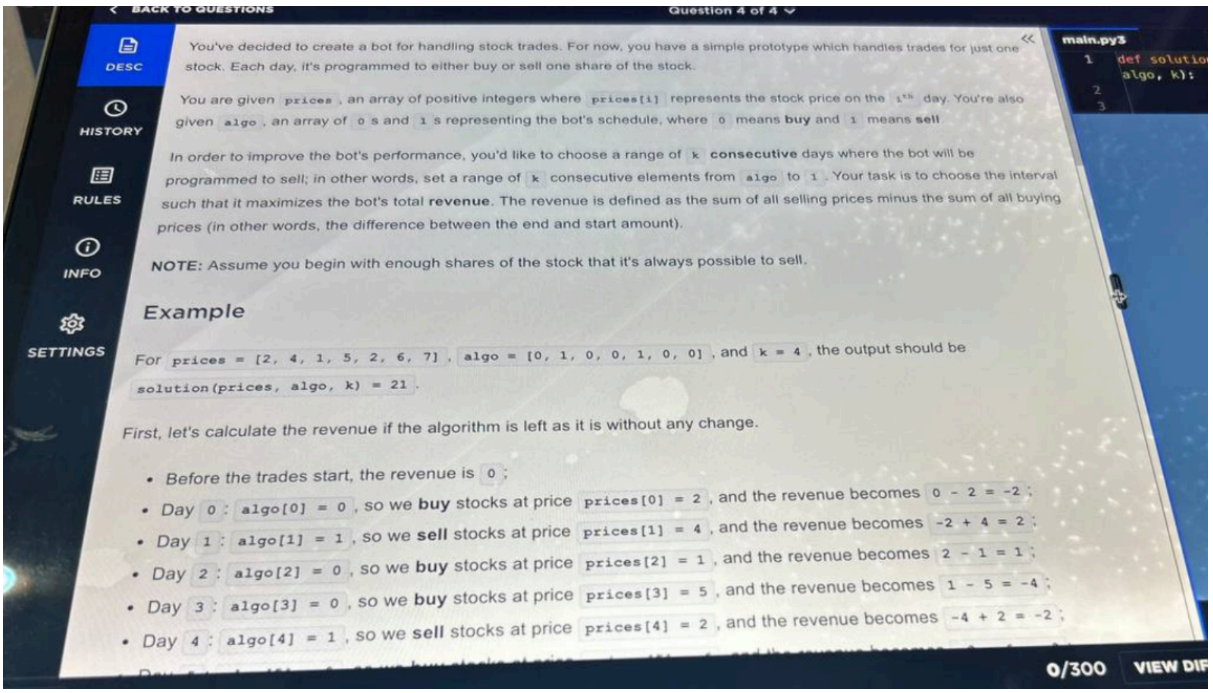
Explanation:

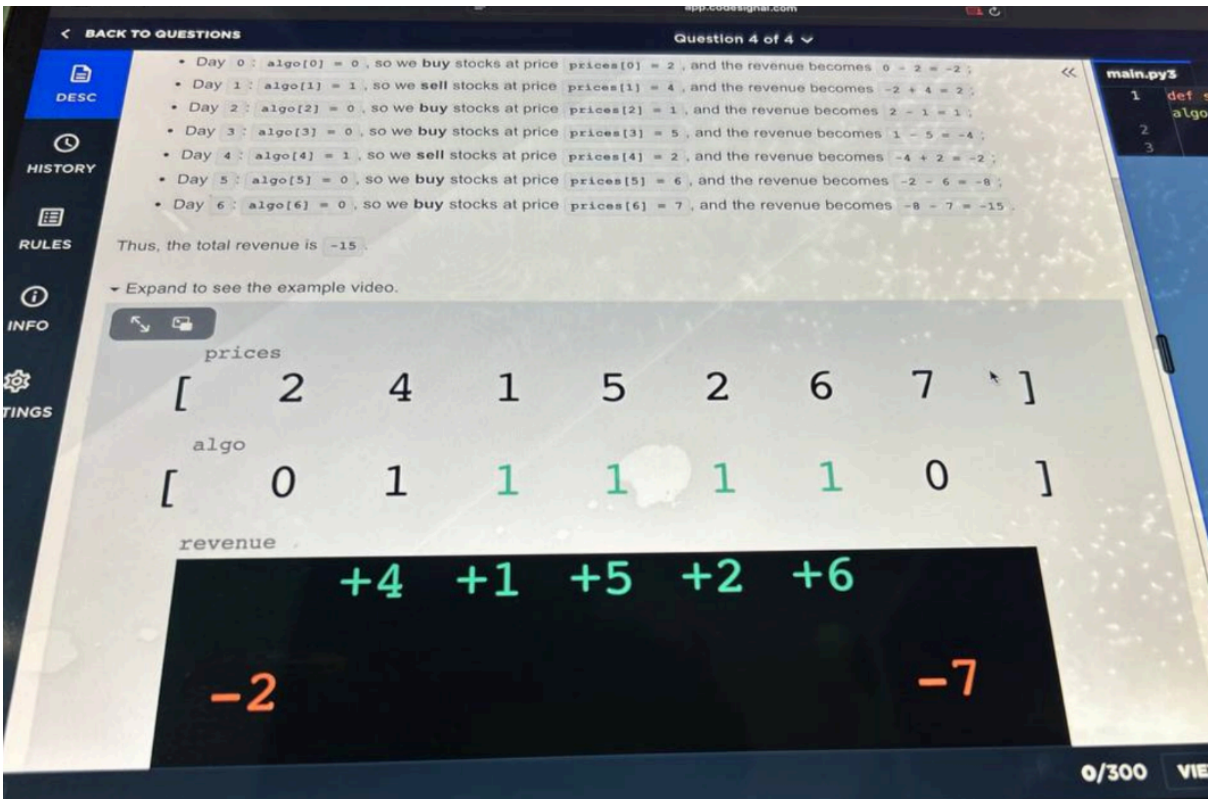
- `smoothingRadius` is equal to 2, so, all the soil cells in this example are the neighbors of each other.
- For the central soil cell `moistureGrid[1][1] = 255`, there are 8 neighbors `[0, 0, 0, 0, 0, 0, 0, 0]` with a mean of `0 // 8 = 0`. So, it should be updated to `mean(255, 0) = 255 // 2 = 127`.
- For all 8 border soil cells, there are 8 neighbors with a mean of `255 // 8 = 255 // 8 = 31`. So, they should be updated to `mean(0, 31) = 31 // 2 = int(15.5) = 15`.

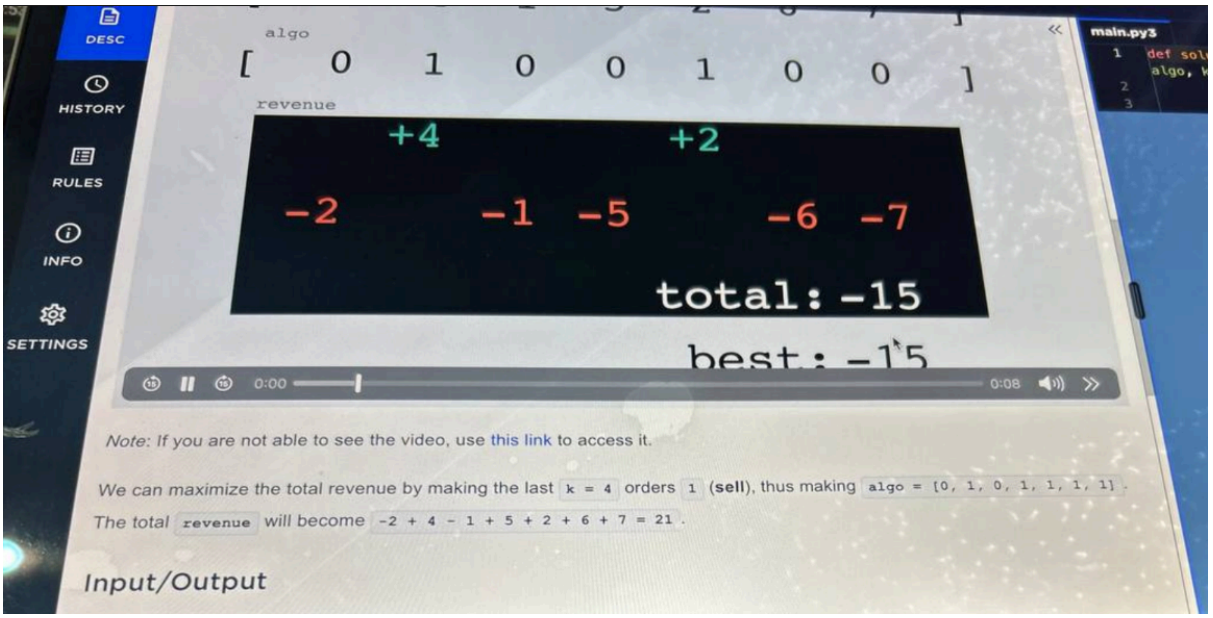
Input/Output

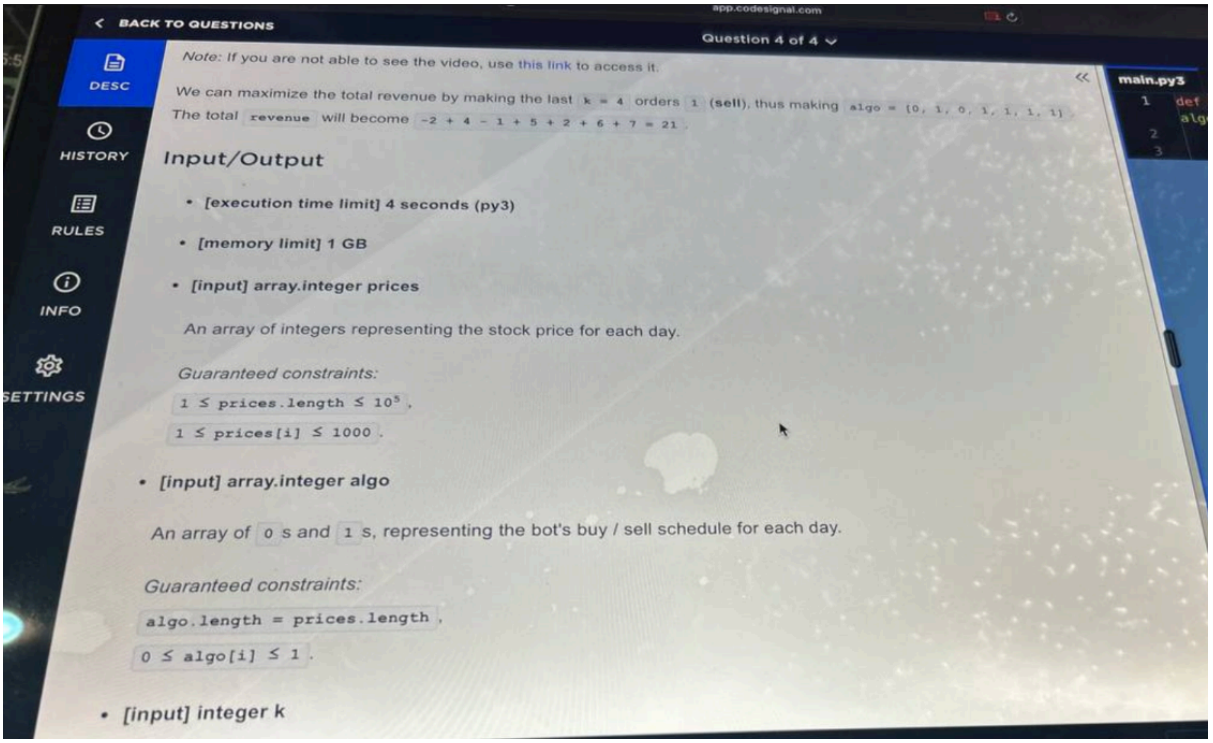

Question 3 of 4
DESC
HISTORY
RULES
INFO
SETTINGS
Input/Output
• [execution time limit] 4 seconds (py3)
• [memory limit] 1 GB
• [input] array.array.integer moistureGrid
A matrix of integers representing the moisture levels of soil cells in a field to be smoothed.
Guaranteed constraints:
1 ≤ moistureGrid.length ≤ 50,
1 ≤ moistureGrid[i].length ≤ 50,
0 ≤ moistureGrid[i][j] ≤ 255.
• [input] integer smoothingRadius
A number representing the distance from a soil cell coordinate, which defines the coordinates that are considered neighbors.
Guaranteed constraints:
1 ≤ smoothingRadius ≤ 5.
• [output] array.array.integer
A matrix of integers representing the moisture levels of soil cells in a smoothed field.

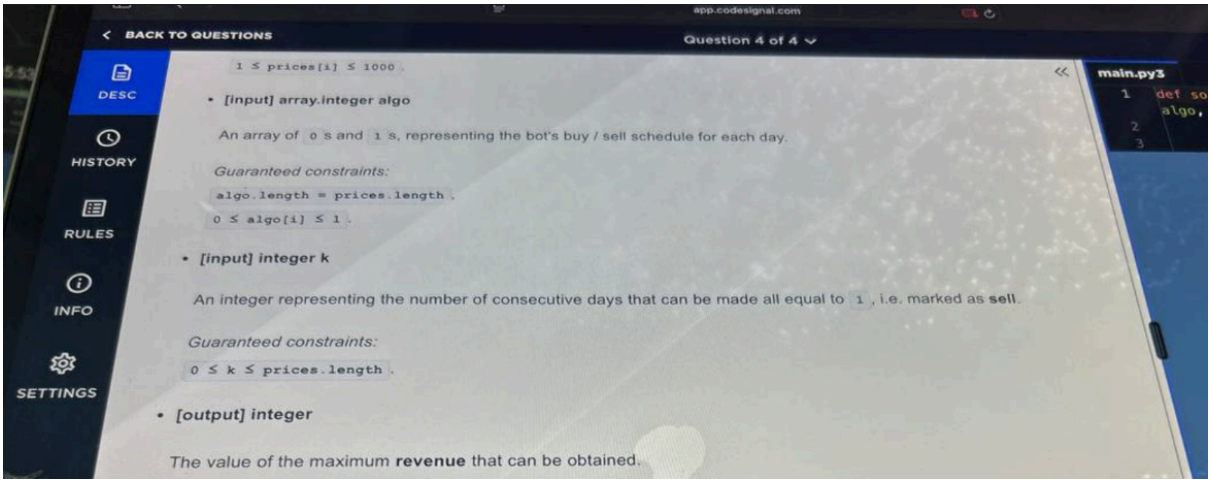
Q26.


BACK TO QUESTIONS
Question 4 of 4
DESC
HISTORY
RULES
INFO
SETTINGS
You've decided to create a bot for handling stock trades. For now, you have a simple prototype which handles trades for just one stock. Each day, it's programmed to either buy or sell one share of the stock.
You are given prices, an array of positive integers where prices[i] represents the stock price on the iᵗʰ day. You're also given algo, an array of 0s and 1s representing the bot's schedule, where 0 means buy and 1 means sell.
In order to improve the bot's performance, you'd like to choose a range of k consecutive days where the bot will be programmed to sell; in other words, set a range of k consecutive elements from algo to 1. Your task is to choose the interval such that it maximizes the bot's total revenue. The revenue is defined as the sum of all selling prices minus the sum of all buying prices (in other words, the difference between the end and start amount).
NOTE: Assume you begin with enough shares of the stock that it's always possible to sell.
Example
For prices = [2, 4, 1, 5, 2, 6, 7], algo = [0, 1, 0, 0, 1, 0, 0], and k = 4, the output should be solution(prices, algo, k) = 21.
First, let's calculate the revenue if the algorithm is left as it is without any change.
• Before the trades start, the revenue is 0;
• Day 0: algo[0] = 0, so we buy stocks at price prices[0] = 2, and the revenue becomes 0 - 2 = -2;
• Day 1: algo[1] = 1, so we sell stocks at price prices[1] = 4, and the revenue becomes -2 + 4 = 2;
• Day 2: algo[2] = 0, so we buy stocks at price prices[2] = 1, and the revenue becomes 2 - 1 = 1;
• Day 3: algo[3] = 0, so we buy stocks at price prices[3] = 5, and the revenue becomes 1 - 5 = -4;
• Day 4: algo[4] = 1, so we sell stocks at price prices[4] = 2, and the revenue becomes -4 + 2 = -2;
main.py3
def solutio
algo, k):
0/300
VIEW DIF


BACK TO QUESTIONS
Question 4 of 4
DESC
HISTORY
RULES
INFO
SETTINGS
• Day 0 : algo[0] = 0 , so we buy stocks at price prices[0] = 2 , and the revenue becomes 0 - 2 = -2 ;
• Day 1 : algo[1] = 1 , so we sell stocks at price prices[1] = 4 , and the revenue becomes -2 + 4 = 2 ;
• Day 2 : algo[2] = 0 , so we buy stocks at price prices[2] = 1 , and the revenue becomes 2 - 1 = 1 ;
• Day 3 : algo[3] = 0 , so we buy stocks at price prices[3] = 5 , and the revenue becomes 1 - 5 = -4 ;
• Day 4 : algo[4] = 1 , so we sell stocks at price prices[4] = 2 , and the revenue becomes -4 + 2 = -2 ;
• Day 5 : algo[5] = 0 , so we buy stocks at price prices[5] = 6 , and the revenue becomes -2 - 6 = -8 ;
• Day 6 : algo[6] = 0 , so we buy stocks at price prices[6] = 7 , and the revenue becomes -8 - 7 = -15 .
Thus, the total revenue is -15 .
Expand to see the example video.
prices
[2 4 1 5 2 6 7]
algo
[0 1 1 1 1 1 0]
revenue
+4 +1 +5 +2 +6
-2
-7
main.py3
0/300


DESC
HISTORY
RULES
INFO
SETTINGS
algo
[0 1 0 0 1 0 0]
revenue
+4 +2
-2 -1 -5 -6 -7
total: -15
best: -15
0:00
0:08
main.py3
Note: If you are not able to see the video, use this link to access it.
We can maximize the total revenue by making the last k = 4 orders 1 (sell), thus making algo = [0, 1, 0, 1, 1, 1, 1] .
The total revenue will become -2 + 4 - 1 + 5 + 2 + 6 + 7 = 21 .
Input/Output


app.codesignal.com
BACK TO QUESTIONS
Question 4 of 4
DESC
HISTORY
RULES
INFO
SETTINGS
Note: If you are not able to see the video, use this link to access it.
We can maximize the total revenue by making the last k = 4 orders 1 (sell), thus making algo = [0, 1, 0, 1, 1, 1, 1]. The total revenue will become -2 + 4 - 1 + 5 + 2 + 6 + 7 = 21.
Input/Output
[execution time limit] 4 seconds (py3)
[memory limit] 1 GB
[input] array.integer prices
An array of integers representing the stock price for each day.
Guaranteed constraints:
1 ≤ prices.length ≤ 10^5,
1 ≤ prices[i] ≤ 1000.
[input] array.integer algo
An array of 0 s and 1 s, representing the bot's buy / sell schedule for each day.
Guaranteed constraints:
algo.length = prices.length,
0 ≤ algo[i] ≤ 1.
[input] integer k
main.py3
1 def
alg
2
3


app.codesignal.com
BACK TO QUESTIONS
Question 4 of 4
DESC
HISTORY
RULES
INFO
SETTINGS
1 ≤ prices[i] ≤ 1000.
[input] array.integer algo
An array of 0 s and 1 s, representing the bot's buy / sell schedule for each day.
Guaranteed constraints:
algo.length = prices.length,
0 ≤ algo[i] ≤ 1.
[input] integer k
An integer representing the number of consecutive days that can be made all equal to 1, i.e. marked as sell.
Guaranteed constraints:
0 ≤ k ≤ prices.length.
[output] integer
The value of the maximum revenue that can be obtained.
main.py3
1 def so
algo,
2
3

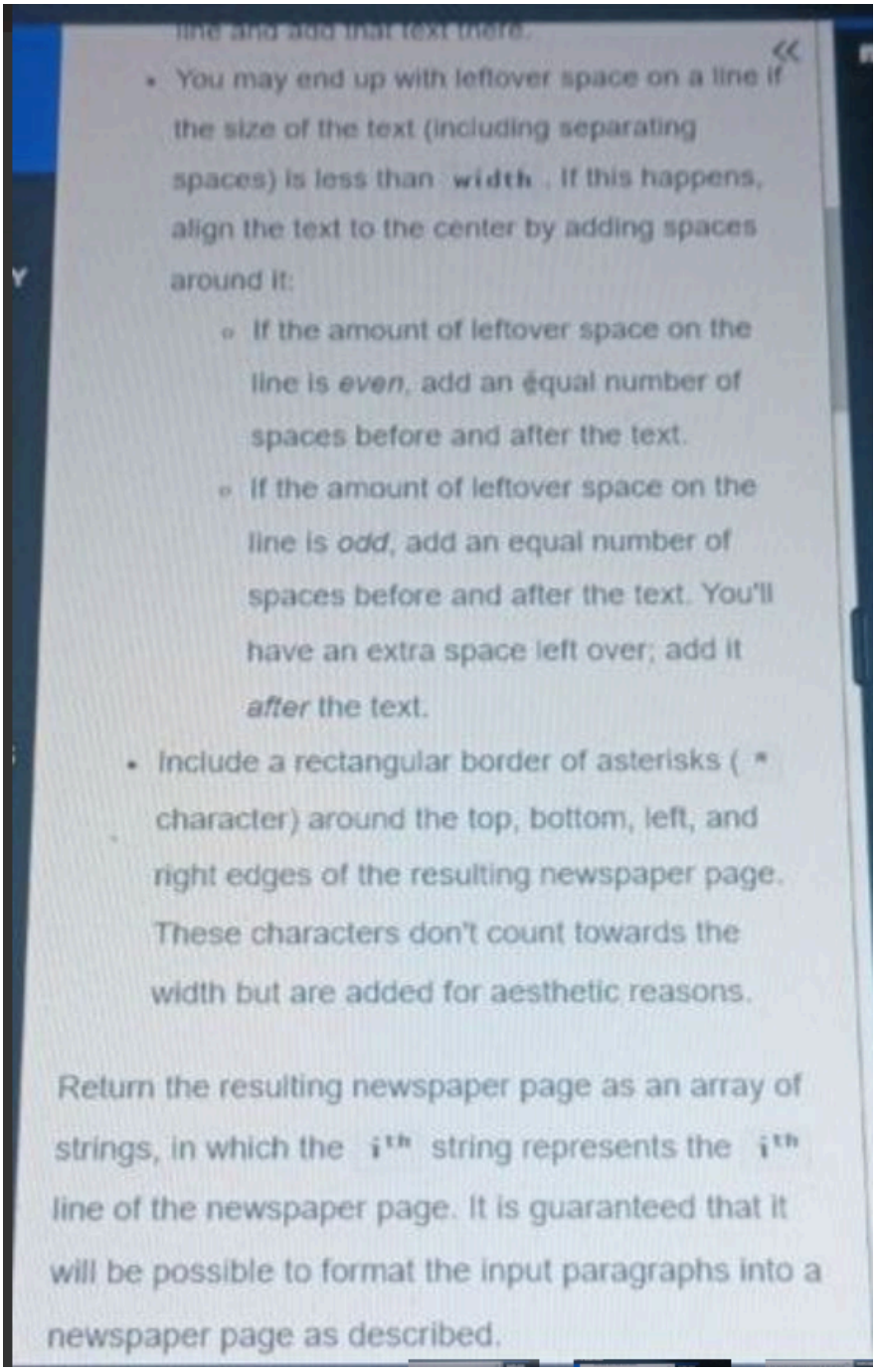
Q27.

Imagine that you are formatting the text that will appear on newspaper page, which requires you to format the text on the page. The text is provided to you in the following format:

- `paragraphs` is an array of paragraphs, where each paragraph is represented by an array containing chunks of text.
- `width` represents the maximum number of characters that can fit on each line of the newspaper page.

Your task is to produce a newspaper page according to the following specifications:

- For each paragraph `paragraphs[i]`, start a new line and add the text from `paragraphs[i][j]` in order. When the portions of text is together on a line, they should be separated by 1 space.
- You cannot break up the portions of text. So, if adding the next portion would cause you to exceed `width`, you will need to start a new line and add that text there.
- You may end up with leftover space on a line if



line and add that text there.

- You may end up with leftover space on a line if the size of the text (including separating spaces) is less than `width`. If this happens, align the text to the center by adding spaces around it:
  - If the amount of leftover space on the line is *even*, add an equal number of spaces before and after the text.
  - If the amount of leftover space on the line is *odd*, add an equal number of spaces before and after the text. You'll have an extra space left over; add it *after* the text.
- Include a rectangular border of asterisks (`*` character) around the top, bottom, left, and right edges of the resulting newspaper page. These characters don't count towards the width but are added for aesthetic reasons.

Return the resulting newspaper page as an array of strings, in which the i^{th} string represents the i^{th} line of the newspaper page. It is guaranteed that it will be possible to format the input paragraphs into a newspaper page as described.

Return the resulting newspaper page as an array of strings, in which the i^{th} string represents the i^{th} line of the newspaper page. It is guaranteed that it will be possible to format the input paragraphs into a newspaper page as described.

Note: You are not expected to provide the most optimal solution, but a solution with time complexity not worse than ***O(paragraphs.length · paragraphs[0].length · width)*** *will fit within the execution time limit.*

Example

For `paragraphs = [["hello", "world"], ["How", "areYou", "doing"], ["Please look", "and align", "to center"]]` and `width = 16`, the output should be

```
solution(paragraphs, width) = ["**********
                               "* hello wo
                               "*How areYou
                               "* Please l
                               "*  and ali
                               "*  to cent
                               "**********
```

Let's consider all paragraphs:

- `paragraphs[0] = ["hello", "world"]`
  - Both portions of text fit on one line with a combined length of `11`, including the separating space.
  - Since `width = 16`, there are `16 - 11 = 5` leftover spaces on the line. We'll need to align the text to center.
  - Since `5` is odd, the line should have `2` leading spaces and `3` trailing spaces.
  - So the result for this paragraph is the line `"  hello world   "`.
- `paragraphs[1] = ["How", "areYou", "doing"]`
  - All three portions of text fit on one line with a length of `16`, including separating spaces.
  - There are no leftover spaces, so the result is `"How areYou doing"`.
- `paragraphs[2] = ["Please look", "and align", "to center"]`

- `paragraphs[2] = ["Please look", "and align", "to center"]`
  - The text `"Please look"` and `"and align"` are too long to combine (the result would have a length of `21 > 16`), so we'll start a new line with `"and align"`.
  - Furthermore, the portions of text `"and align"` and `"to center"` would have a combined length of `19 > 16` which is also too long, so `"to center"` will also be on a new line.
  - Per line, the amount of leftover space is `5`, `7`, and `7`. The result of this paragraph is therefore:

```
"  Please look    "
"   and align     "
"   to center     "
```

To create the border, add an asterisk (`*` character) to the beginning and end of each line, then add a string of `width + 2` asterisks before the first line and after the last line.

- **[input] array.array.string paragraphs**

  An array of arrays of strings representing text for the newspaper page.

  Guaranteed constraints:

  $1 \le$ `paragraphs.length` ≤ 20,

  $1 \le$ `paragraphs[i].length` ≤ 10,

  $1 \le$ `paragraphs[i][j].length` $\le$ `width`.

- **[input] integer width**

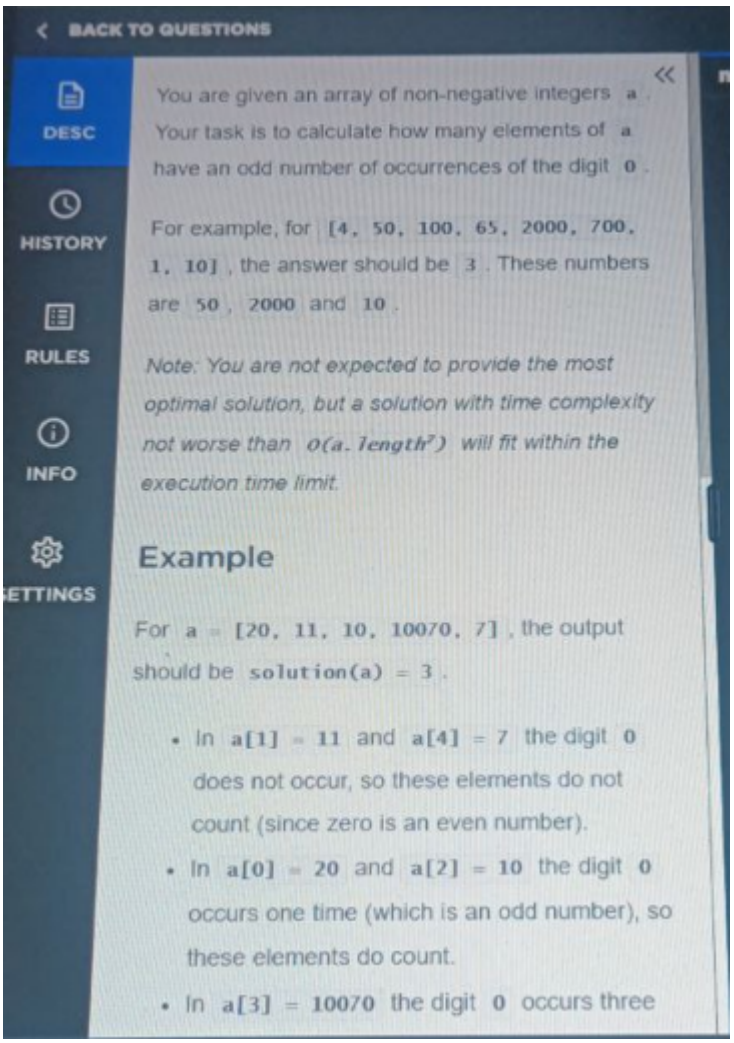
  An integer representing the width of the newspaper page.

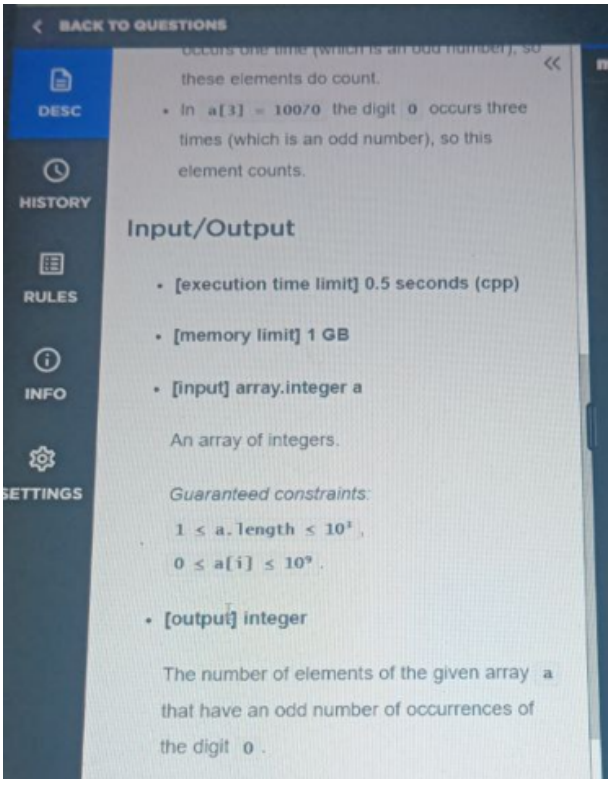
  Guaranteed constraints:

  $5 \le$ `width` ≤ 50.

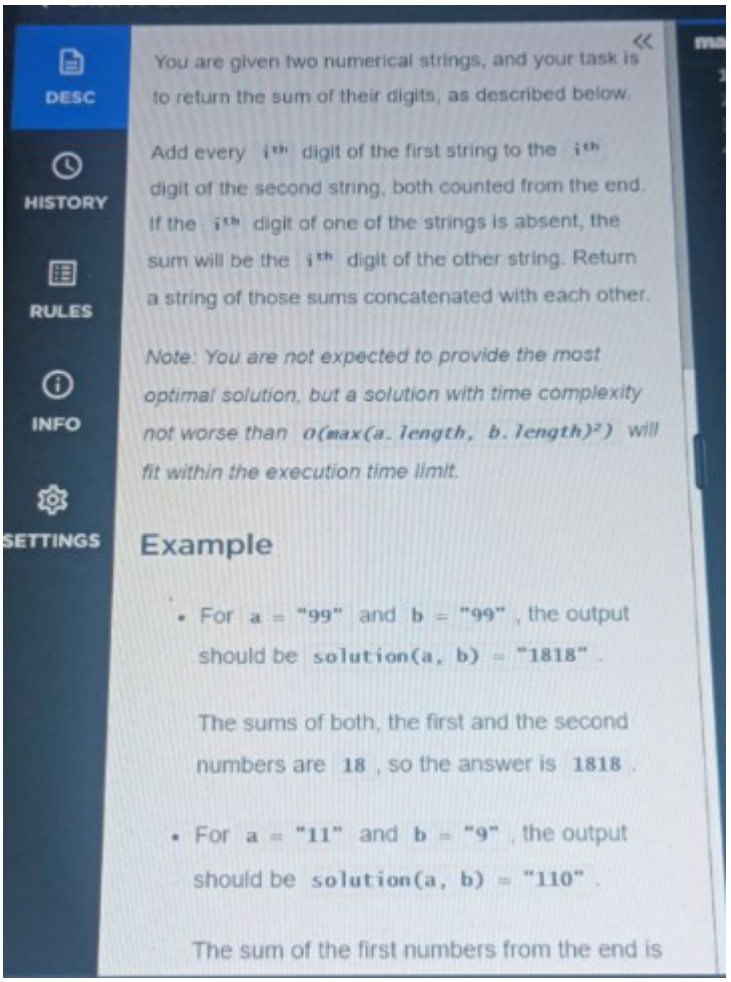
- **[output] array.string**

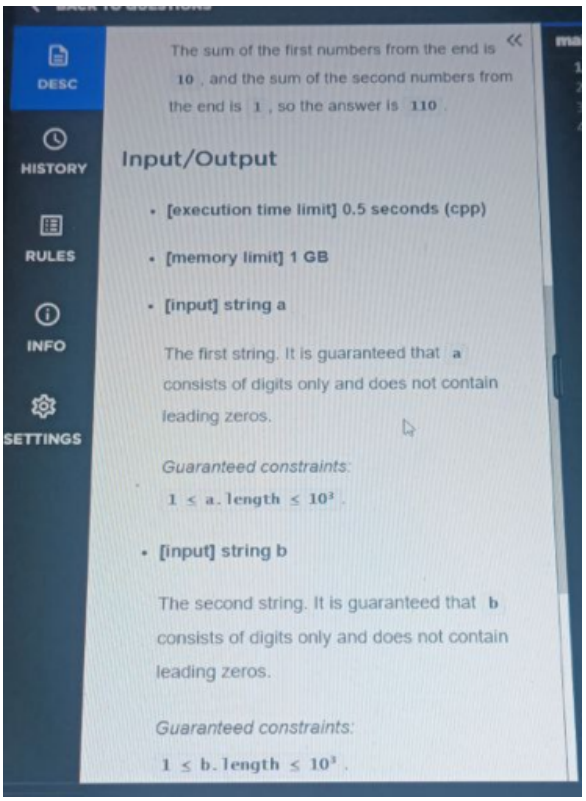
Q28.

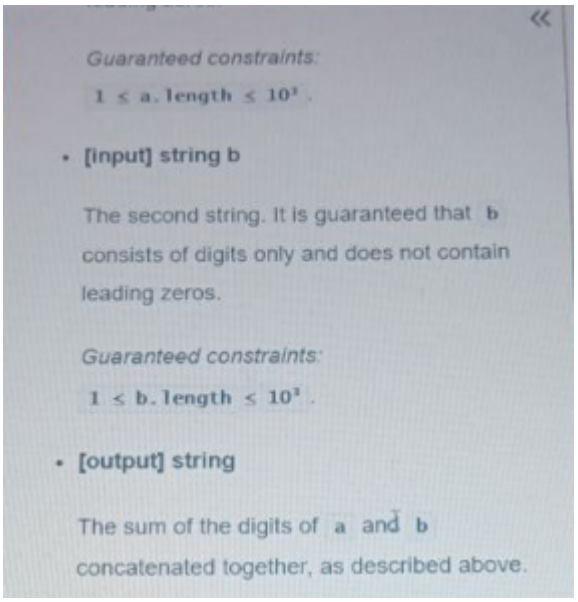

BACK TO QUESTIONS
DESC
HISTORY
RULES
INFO
SETTINGS
You are given an array of non-negative integers a. Your task is to calculate how many elements of a have an odd number of occurrences of the digit 0.
For example, for [4, 50, 100, 65, 2000, 700, 1, 10], the answer should be 3. These numbers are 50, 2000 and 10.
Note: You are not expected to provide the most optimal solution, but a solution with time complexity not worse than O(a.length^2) will fit within the execution time limit.
Example
For a = [20, 11, 10, 10070, 7], the output should be solution(a) = 3.
In a[1] = 11 and a[4] = 7 the digit 0 does not occur, so these elements do not count (since zero is an even number).
In a[0] = 20 and a[2] = 10 the digit 0 occurs one time (which is an odd number), so these elements do count.
In a[3] = 10070 the digit 0 occurs three


BACK TO QUESTIONS
DESC
HISTORY
RULES
INFO
SETTINGS
these elements do count.
In a[3] = 10070 the digit 0 occurs three times (which is an odd number), so this element counts.
Input/Output
[execution time limit] 0.5 seconds (cpp)
[memory limit] 1 GB
[input] array.integer a
An array of integers.
Guaranteed constraints:
1 ≤ a.length ≤ 10^1,
0 ≤ a[i] ≤ 10^9.
[output] integer
The number of elements of the given array a that have an odd number of occurrences of the digit 0.

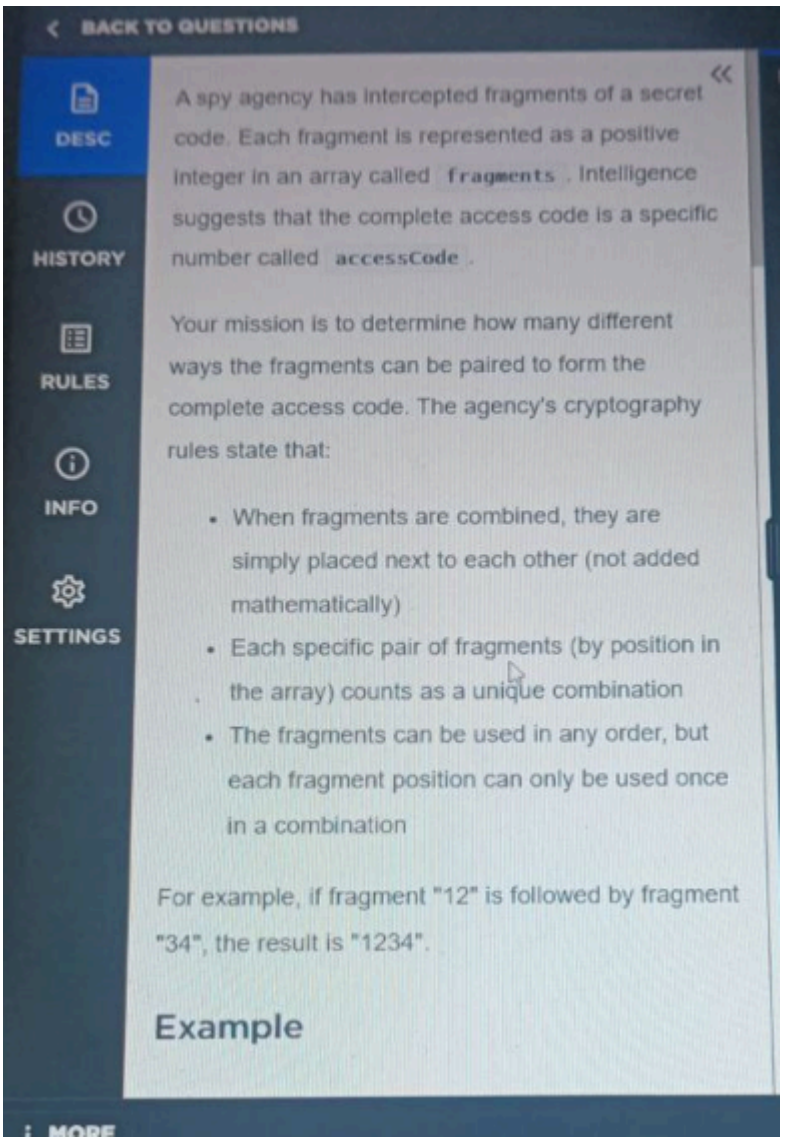
Q29.

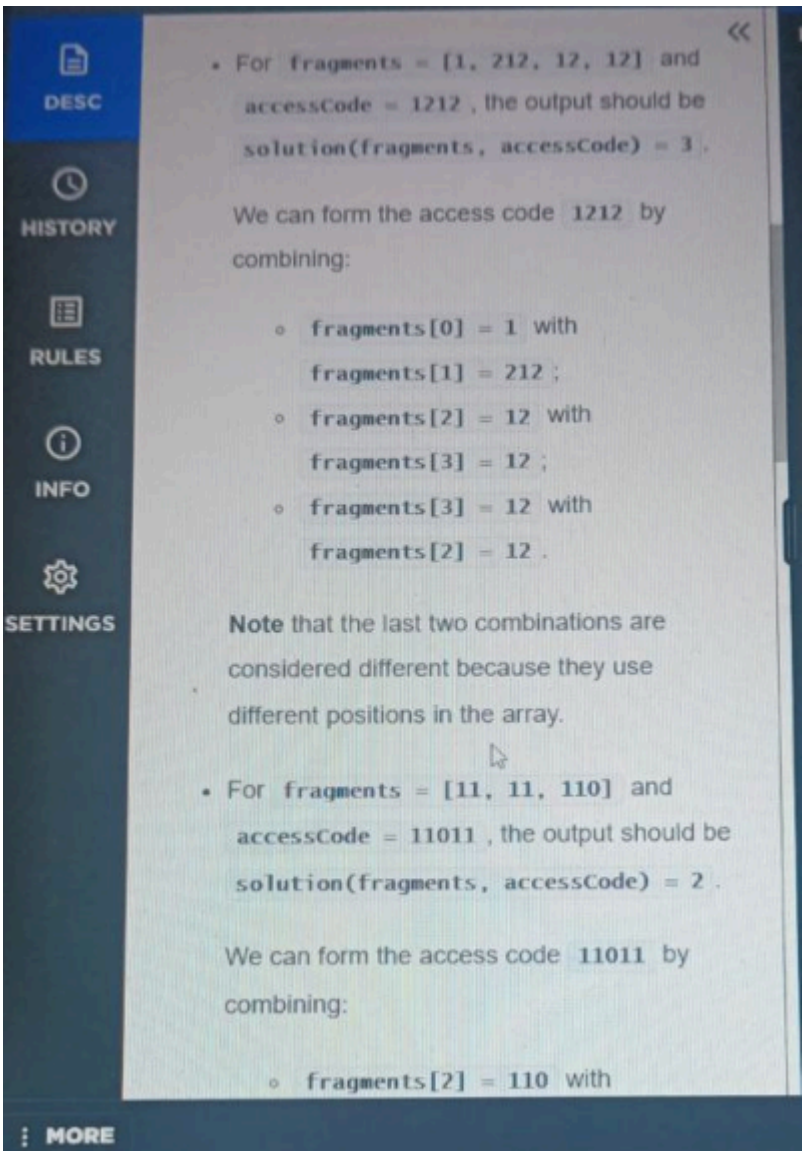

DESC
HISTORY
RULES
INFO
SETTINGS
You are given two numerical strings, and your task is to return the sum of their digits, as described below.
Add every i^{th} digit of the first string to the i^{th} digit of the second string, both counted from the end. If the i^{th} digit of one of the strings is absent, the sum will be the i^{th} digit of the other string. Return a string of those sums concatenated with each other.
Note: You are not expected to provide the most optimal solution, but a solution with time complexity not worse than $O(max(a.length, b.length)^2)$ will fit within the execution time limit.
Example
For a = "99" and b = "99", the output should be solution(a, b) = "1818".
The sums of both, the first and the second numbers are 18, so the answer is 1818.
For a = "11" and b = "9", the output should be solution(a, b) = "110".
The sum of the first numbers from the end is


DESC
HISTORY
RULES
INFO
SETTINGS
The sum of the first numbers from the end is 10, and the sum of the second numbers from the end is 1, so the answer is 110.
Input/Output
[execution time limit] 0.5 seconds (cpp)
[memory limit] 1 GB
[input] string a
The first string. It is guaranteed that a consists of digits only and does not contain leading zeros.
Guaranteed constraints:
1 ≤ a.length ≤ 10^3.
[input] string b
The second string. It is guaranteed that b consists of digits only and does not contain leading zeros.
Guaranteed constraints:
1 ≤ b.length ≤ 10^3.


Guaranteed constraints:
1 ≤ a.length ≤ 10^3.
[input] string b
The second string. It is guaranteed that b consists of digits only and does not contain leading zeros.
Guaranteed constraints:
1 ≤ b.length ≤ 10^3.
[output] string
The sum of the digits of a and b concatenated together, as described above.

Q 30.


BACK TO QUESTIONS
DESC
HISTORY
RULES
INFO
SETTINGS
A spy agency has intercepted fragments of a secret code. Each fragment is represented as a positive integer in an array called fragments. Intelligence suggests that the complete access code is a specific number called accessCode.
Your mission is to determine how many different ways the fragments can be paired to form the complete access code. The agency's cryptography rules state that:
When fragments are combined, they are simply placed next to each other (not added mathematically)
Each specific pair of fragments (by position in the array) counts as a unique combination
The fragments can be used in any order, but each fragment position can only be used once in a combination
For example, if fragment "12" is followed by fragment "34", the result is "1234".
Example
MORE



- For `fragments = [1, 212, 12, 12]` and `accessCode = 1212`, the output should be `solution(fragments, accessCode) = 3`.

  We can form the access code `1212` by combining:

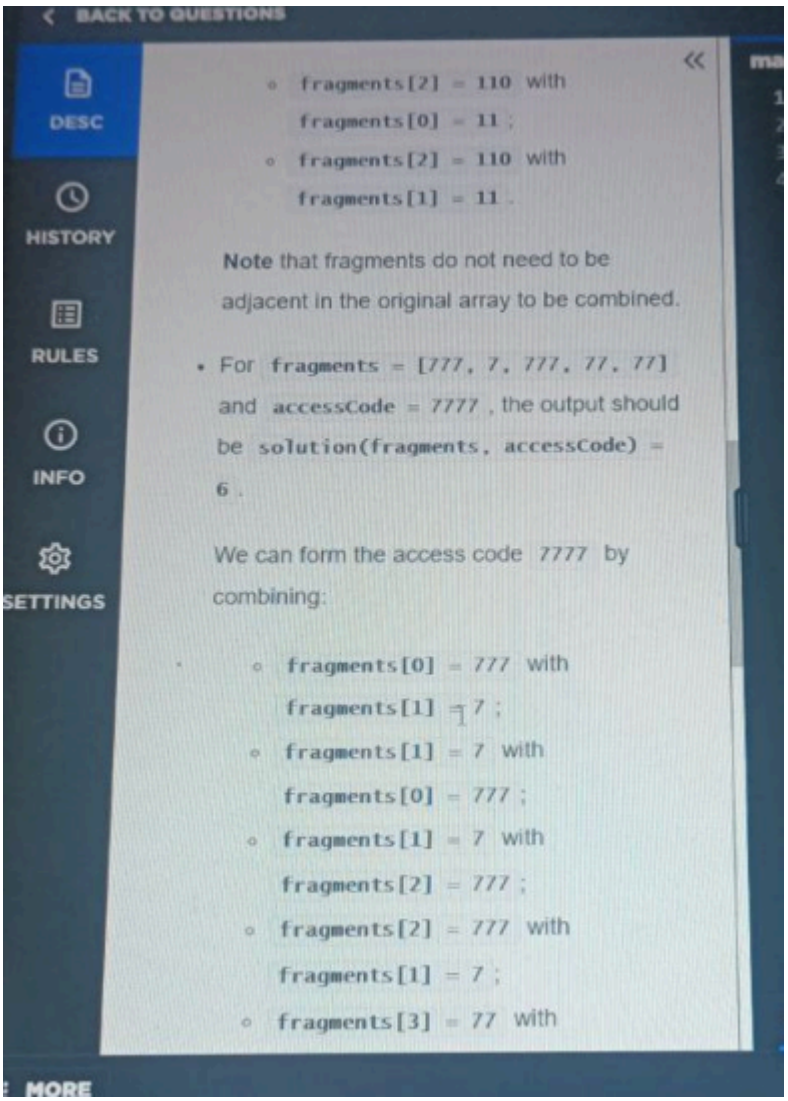
  - `fragments[0] = 1` with `fragments[1] = 212`;
  - `fragments[2] = 12` with `fragments[3] = 12`;
  - `fragments[3] = 12` with `fragments[2] = 12`.

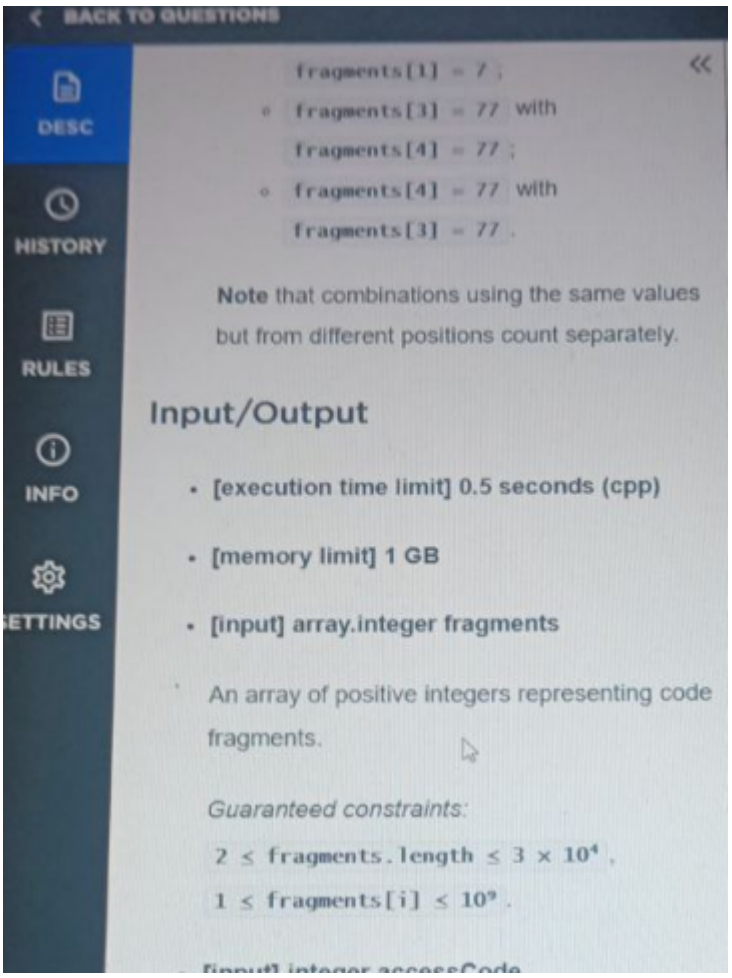
  Note that the last two combinations are considered different because they use different positions in the array.

- For `fragments = [11, 11, 110]` and `accessCode = 11011`, the output should be `solution(fragments, accessCode) = 2`.

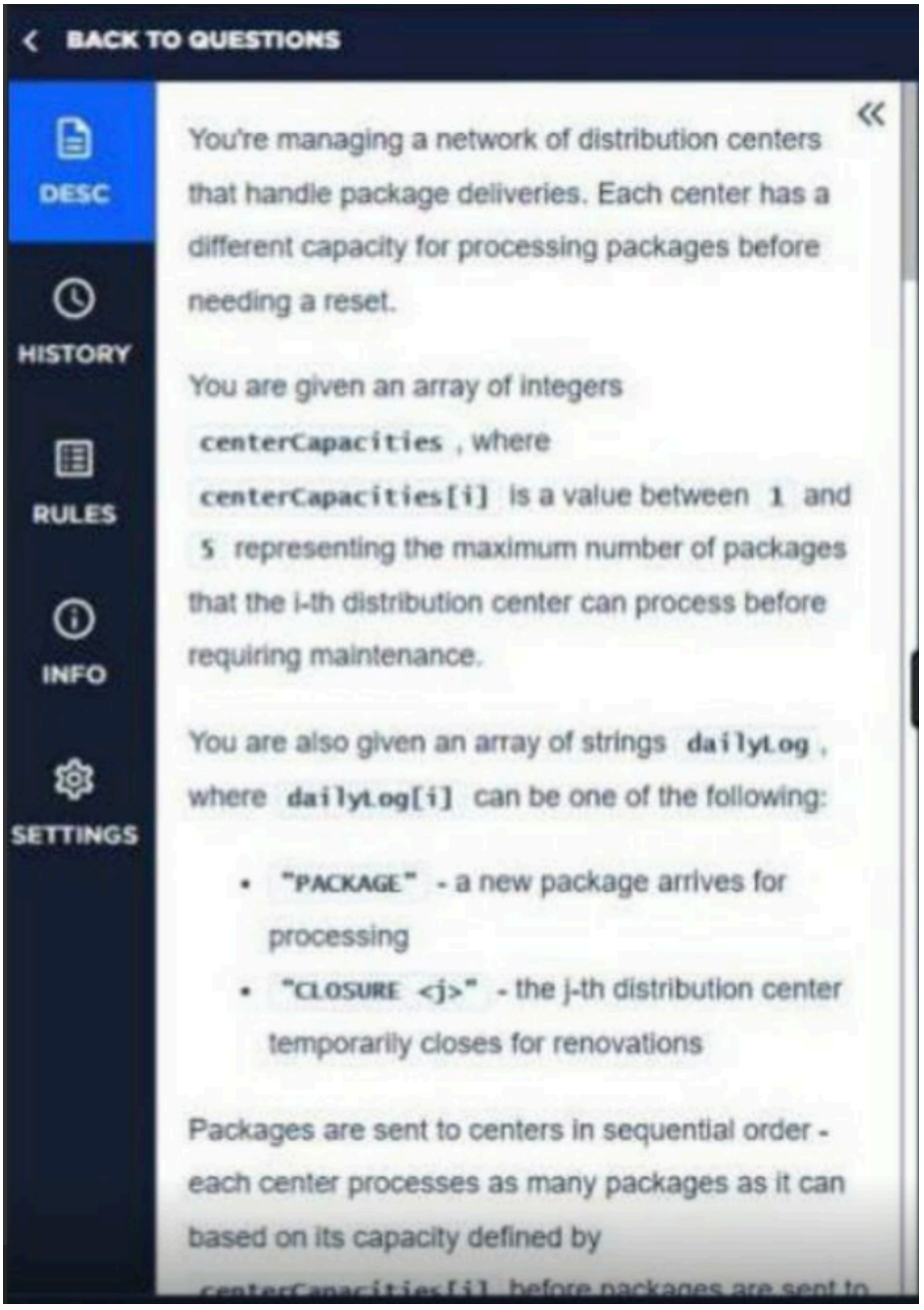
  We can form the access code `11011` by combining:

  - `fragments[2] = 110` with


BACK TO QUESTIONS
DESC
HISTORY
RULES
INFO
SETTINGS
MORE
fragments[2] = 110 with fragments[0] = 11;
fragments[2] = 110 with fragments[1] = 11.
Note that fragments do not need to be adjacent in the original array to be combined.
For fragments = [777, 7, 777, 77, 77] and accessCode = 7777, the output should be solution(fragments, accessCode) = 6.
We can form the access code 7777 by combining:
fragments[0] = 777 with fragments[1] = 7;
fragments[1] = 7 with fragments[0] = 777;
fragments[1] = 7 with fragments[2] = 777;
fragments[2] = 777 with fragments[1] = 7;
fragments[3] = 77 with


BACK TO QUESTIONS
DESC
HISTORY
RULES
INFO
SETTINGS
fragments[1] = 7 ;
fragments[3] = 77 with
fragments[4] = 77 ;
fragments[4] = 77 with
fragments[3] = 77 .
Note that combinations using the same values but from different positions count separately.
Input/Output
[execution time limit] 0.5 seconds (cpp)
[memory limit] 1 GB
[input] array.integer fragments
An array of positive integers representing code fragments.
Guaranteed constraints:
2 ≤ fragments.length ≤ 3 × 10^4 ,
1 ≤ fragments[i] ≤ 10^9 .

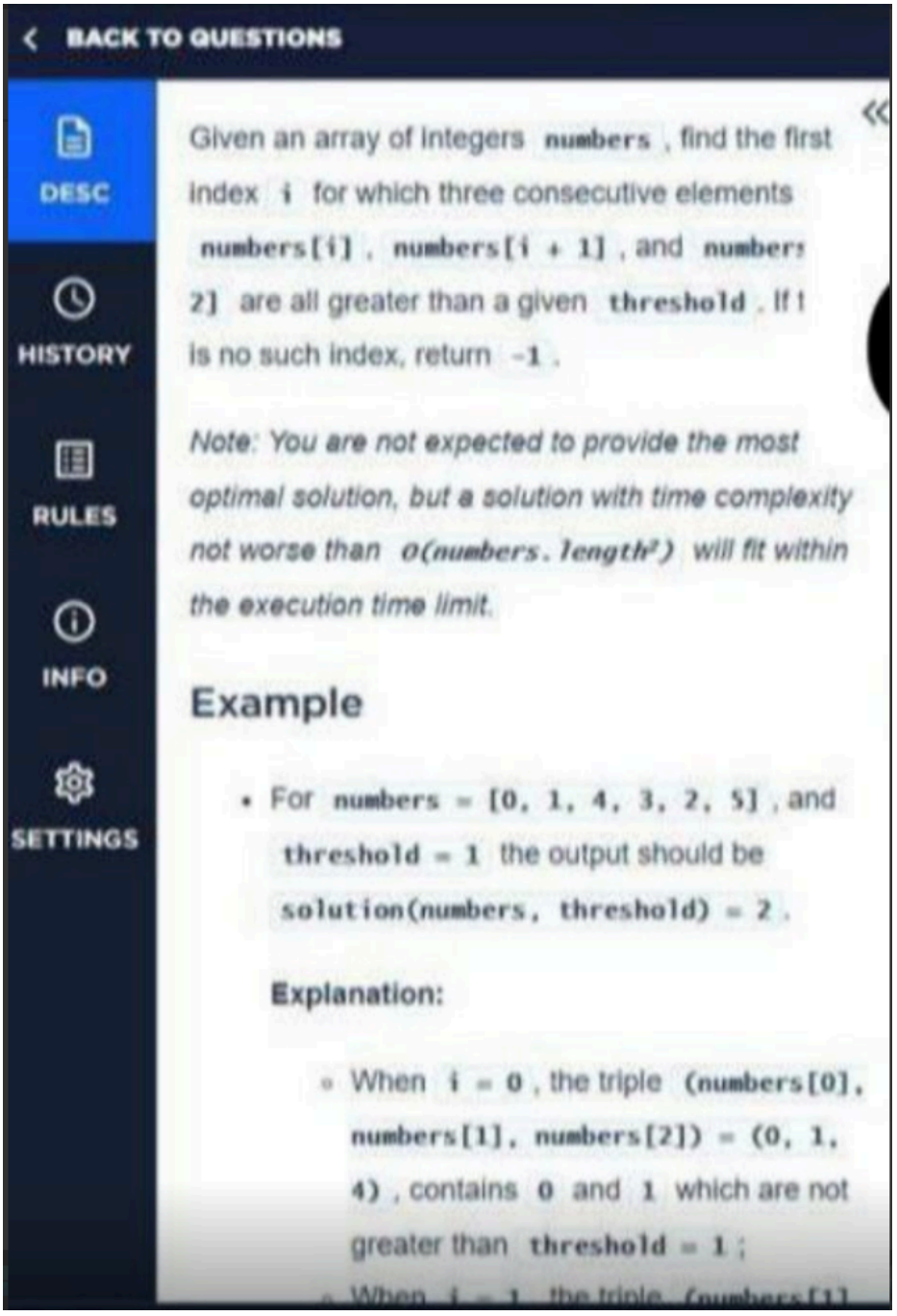
Q31.

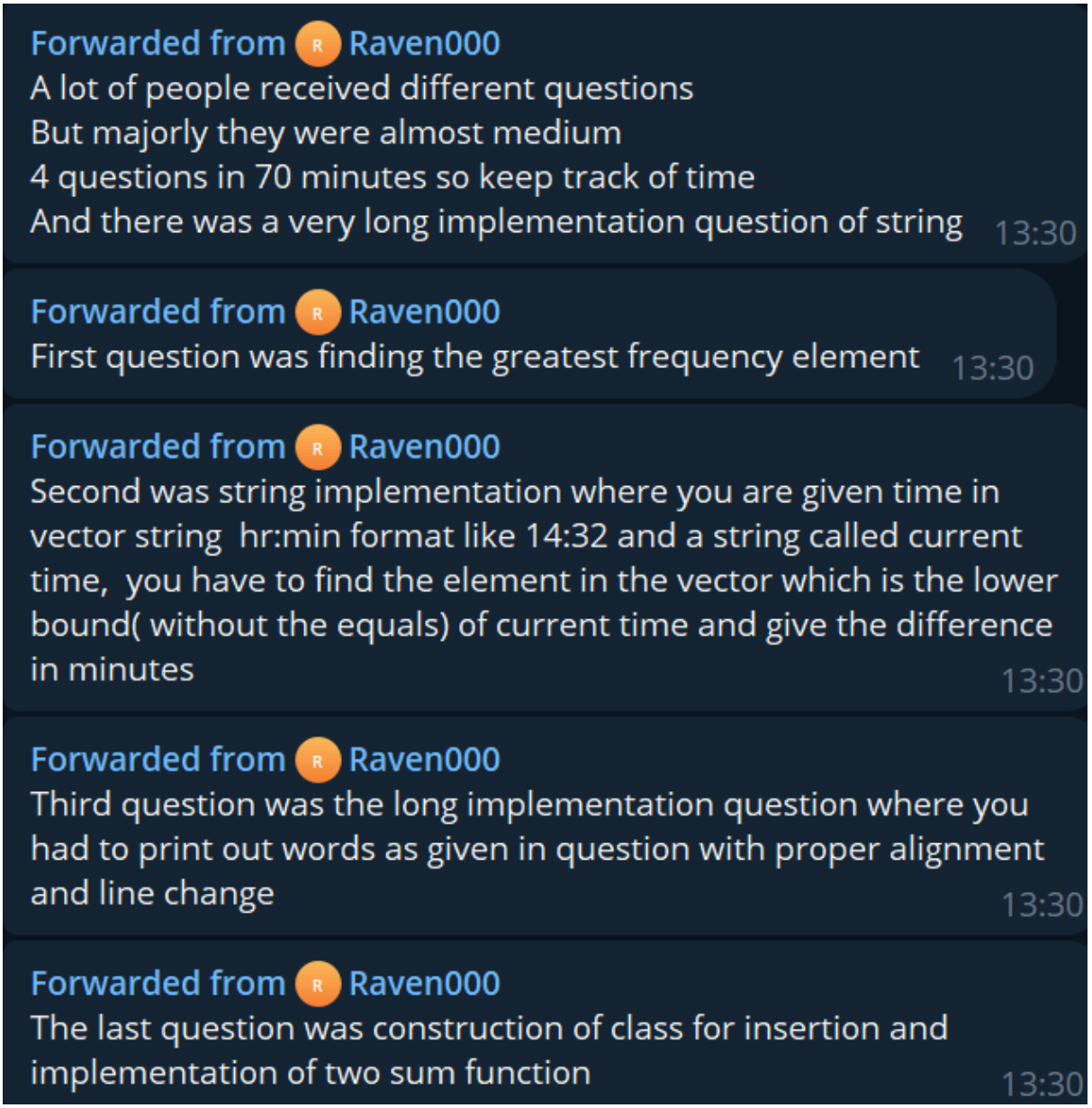

BACK TO QUESTIONS
DESC
HISTORY
RULES
INFO
SETTINGS
You're managing a network of distribution centers that handle package deliveries. Each center has a different capacity for processing packages before needing a reset.
You are given an array of integers centerCapacities , where centerCapacities[i] is a value between 1 and 5 representing the maximum number of packages that the i-th distribution center can process before requiring maintenance.
You are also given an array of strings dailyLog , where dailyLog[i] can be one of the following:
"PACKAGE" - a new package arrives for processing
"CLOSURE <j>" - the j-th distribution center temporarily closes for renovations
Packages are sent to centers in sequential order - each center processes as many packages as it can based on its capacity defined by

You are given `numbers`, an array of non-negative integers. Your task is to perform the following algorithm on this array:

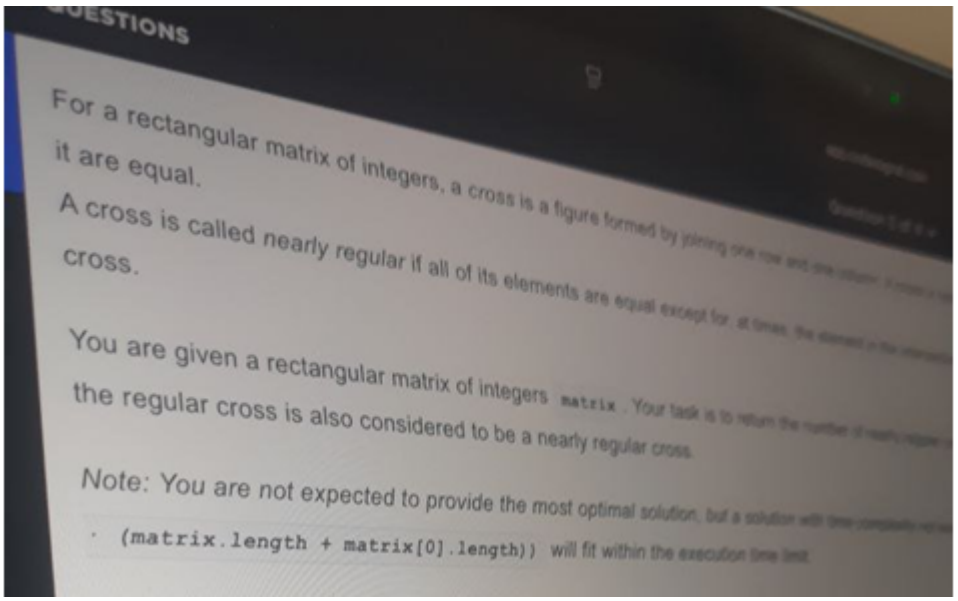
- **Step 1.** Find the index `i` of the leftmost non-zero element `numbers[i] = x ≠ 0`. If there is no such element, finish the algorithm.
- **Step 2.** Starting at index `i` and going to the right, attempt to subtract `x` from each element.
  - If the element is strictly less than `x`, move on to step 3;
  - Otherwise, subtract `x` from the element and move on to the next element;
  - If you reach the end of the array, move on to step 3.
- **Step 3.** Add `x` to the final result.
- **Step 4.** Go back to step 1.

Return the resulting sum obtained from step 3. It is guaranteed that the algorithm is finite and will finish at some point.


BACK TO QUESTIONS
DESC
HISTORY
RULES
INFO
SETTINGS
Given an array of integers numbers , find the first index i for which three consecutive elements numbers[i] , numbers[i + 1] , and numbers[i + 2] are all greater than a given threshold . If there is no such index, return -1 .
Note: You are not expected to provide the most optimal solution, but a solution with time complexity not worse than O(numbers.length²) will fit within the execution time limit.
Example
For numbers = [0, 1, 4, 3, 2, 5] , and threshold = 1 the output should be solution(numbers, threshold) = 2 .
Explanation:
When i = 0 , the triple (numbers[0], numbers[1], numbers[2]) = (0, 1, 4) , contains 0 and 1 which are not greater than threshold = 1 ;


Forwarded from Raven000
A lot of people received different questions
But majorly they were almost medium
4 questions in 70 minutes so keep track of time
And there was a very long implementation question of string 13:30
Forwarded from Raven000
First question was finding the greatest frequency element 13:30
Forwarded from Raven000
Second was string implementation where you are given time in vector string hr:min format like 14:32 and a string called current time, you have to find the element in the vector which is the lower bound(without the equals) of current time and give the difference in minutes 13:30
Forwarded from Raven000
Third question was the long implementation question where you had to print out words as given in question with proper alignment and line change 13:30
Forwarded from Raven000
The last question was construction of class for insertion and implementation of two sum function 13:30

Q32.



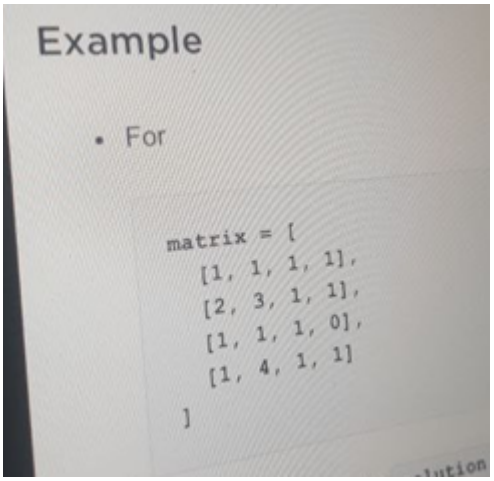
QUESTIONS

For a rectangular matrix of integers, a cross is a figure formed by joining one row and one column. A cross is nearly regular if all of its elements are equal except for, at most, one element in the intersection of the row and column that form the cross. The element in the intersection of the row and column that form the cross may be any value.

A cross is called nearly regular if all of its elements are equal except for, at most, one element in the intersection of the row and column that form the cross.

You are given a rectangular matrix of integers `matrix`. Your task is to return the number of nearly regular crosses in `matrix`. Note that the regular cross is also considered to be a nearly regular cross.

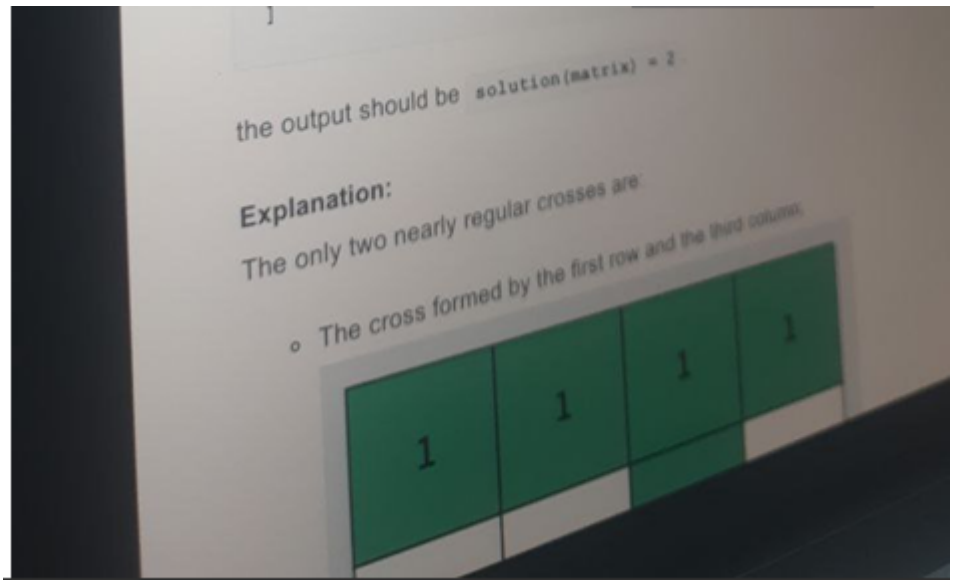
Note: You are not expected to provide the most optimal solution, but a solution with time complexity not exceeding `(matrix.length + matrix[0].length)` will fit within the execution time limit.



Example

- For

```
matrix = [
  [1, 1, 1, 1],
  [2, 3, 1, 1],
  [1, 1, 1, 0],
  [1, 4, 1, 1]
]
```



]

the output should be `solution(matrix) = 2`.

Explanation:

The only two nearly regular crosses are:

- The cross formed by the first row and the third column: